

計算機算術

Computer Arithmetic

Problem: Designing MIPS ALU

- ◆ Requirements: must support the following arithmetic and logic operations
 - **add, sub**: two's complement adder/subtractor with overflow detection
 - **and, or, nor**: logical AND, logical OR, logical NOR
 - **slt** (set on less than): two's complement adder with inverter, check sign bit of result

Functional Specification

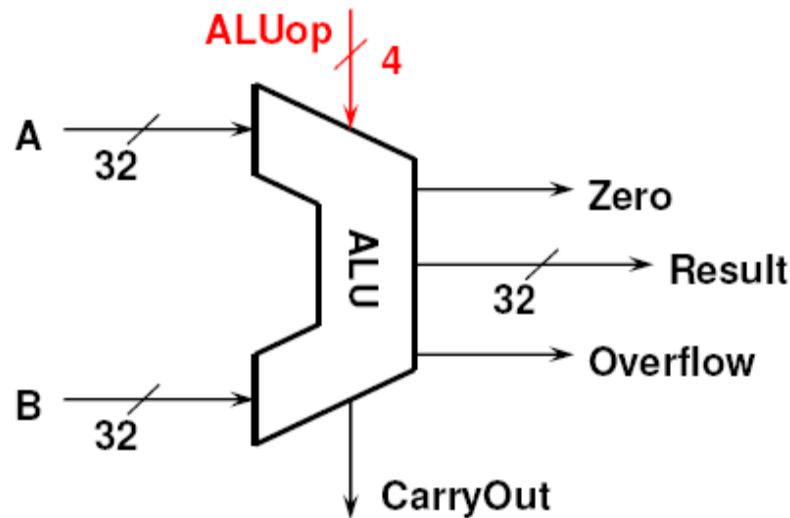


Fig. B.5.14

ALU Control (ALUop)

0000

0001

0010

0110

0111

Function

and

or

add

subtract

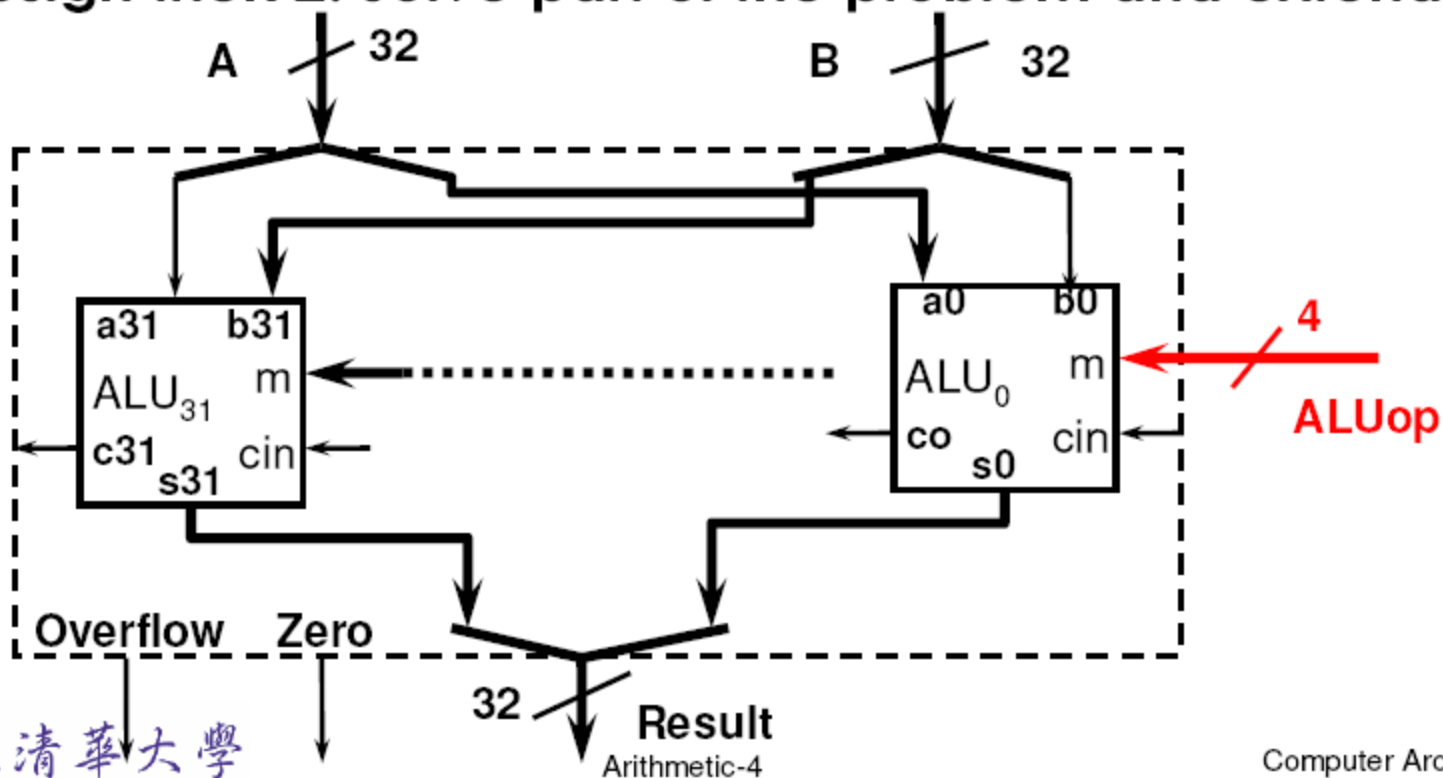
set-on-less-than

nor

Fig. B.5.13

A Bit-slice ALU

- ◆ Design trick 1: divide and conquer
 - Break the problem into simpler problems, solve them and glue together the solution
- ◆ Design trick 2: solve part of the problem and extend



A 1-bit ALU

- ◆ Design trick 3: take pieces you know (or can imagine) and try to put them together

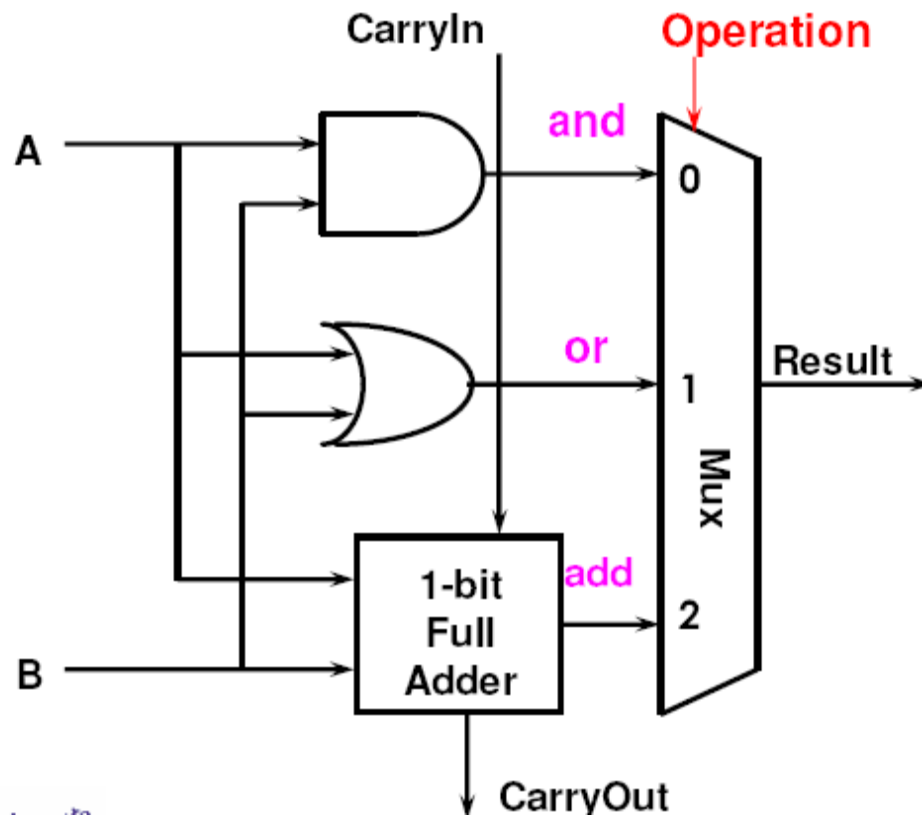
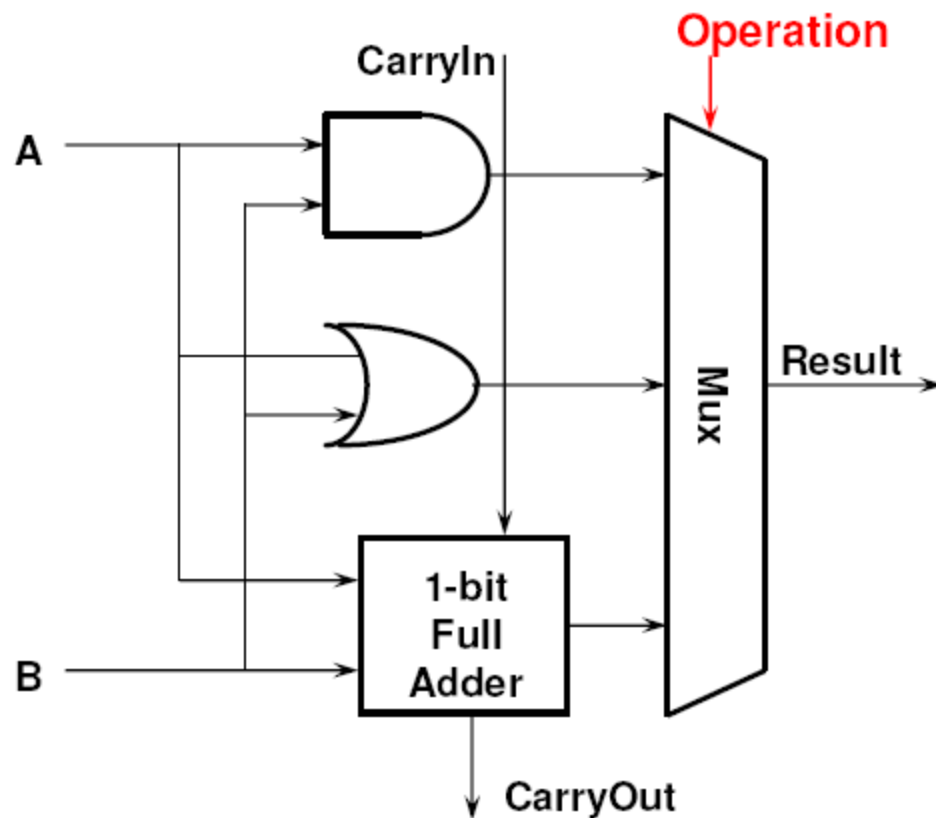


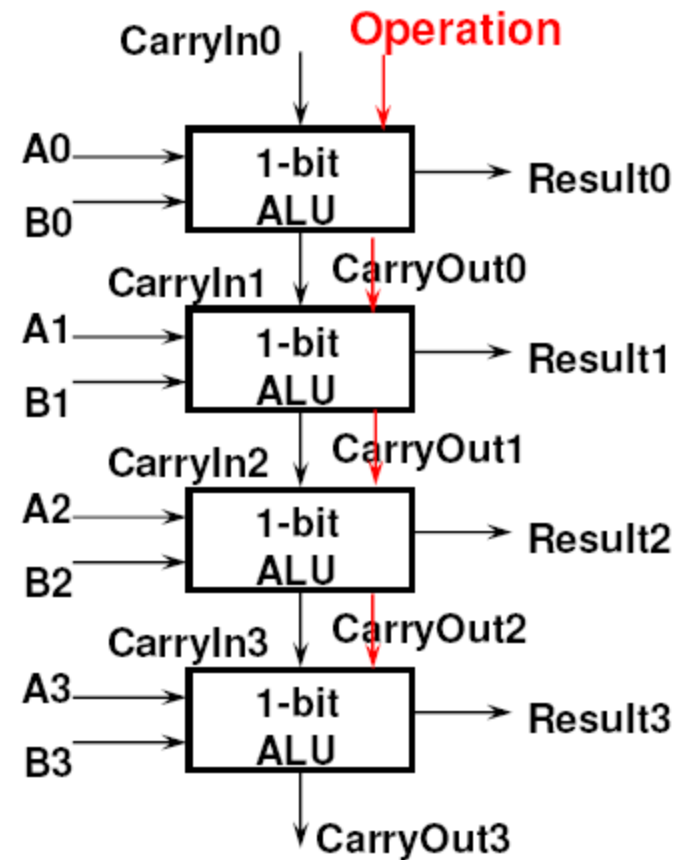
Fig. B.5.6

A 4-bit ALU

1-bit ALU



4-bit ALU



How about Subtraction?

- ◆ 2's complement: take inverse of every bit and add 1 (at c_{in} of first stage)
 - $A + B' + 1 = A + (B' + 1) = A + (-B) = A - B$
 - Bit-wise inverse of B is B'

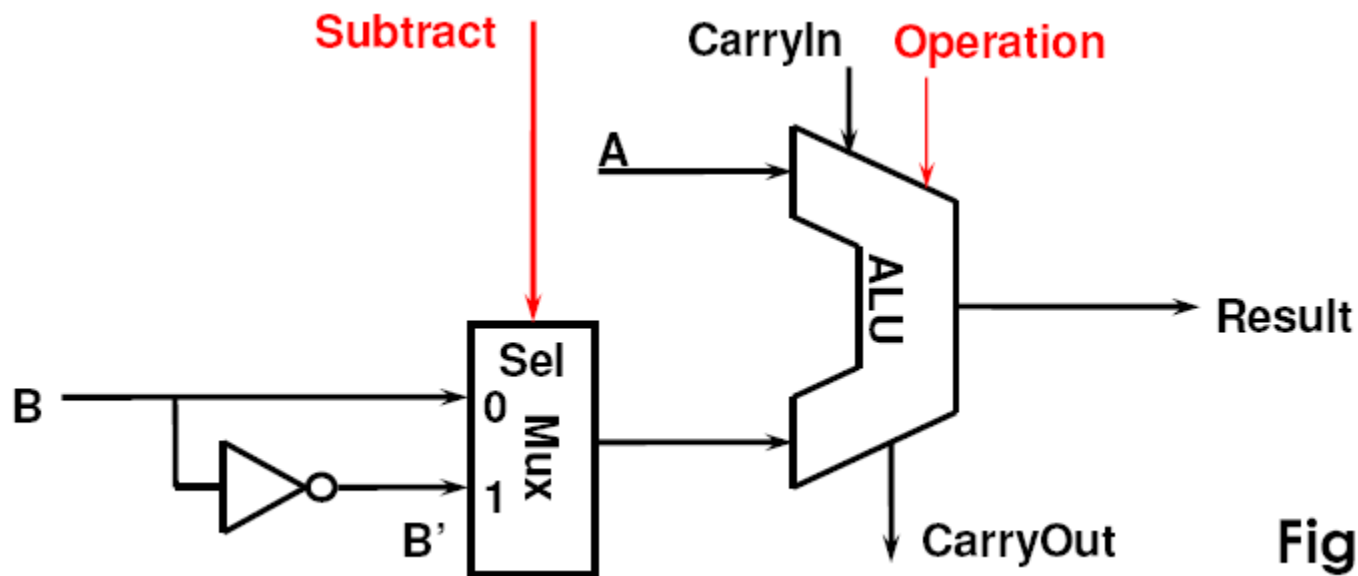
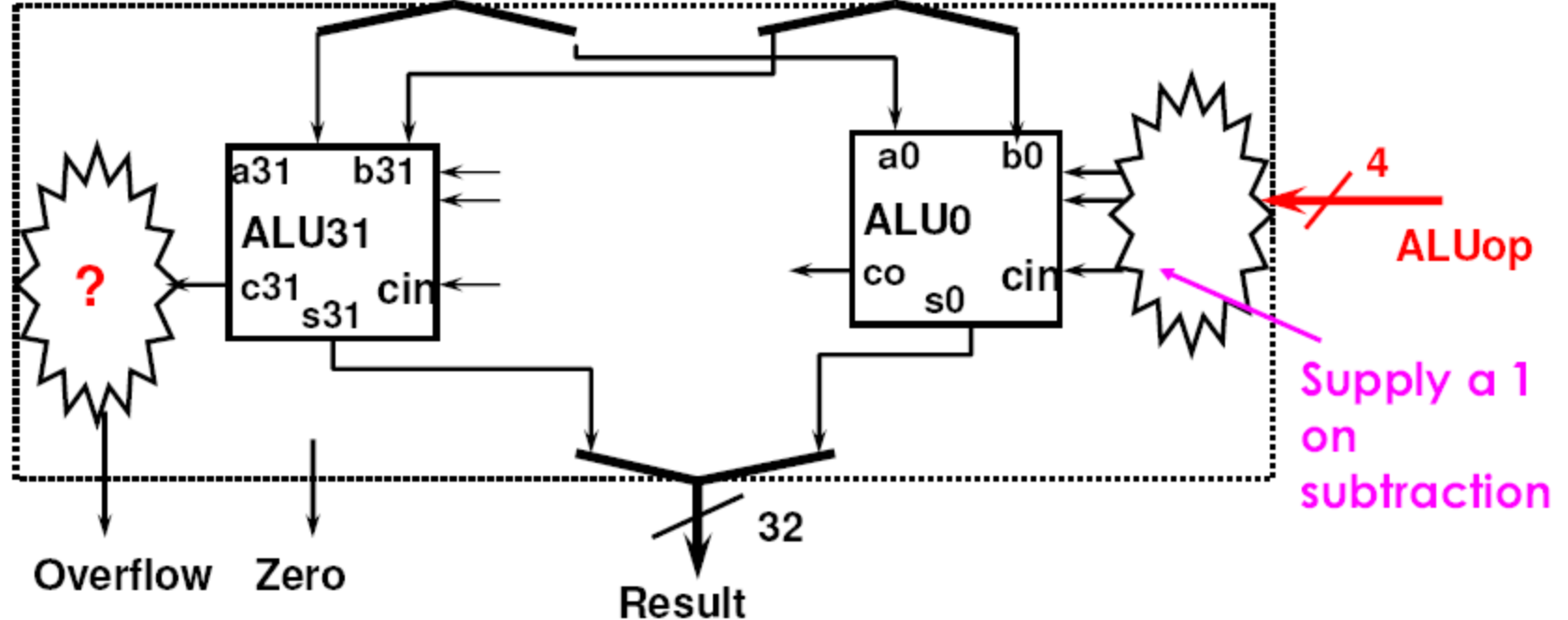


Fig. B.5.8



Nor Operation

- ♦ $A \text{ nor } B = (\text{not } A) \text{ and } (\text{not } B)$

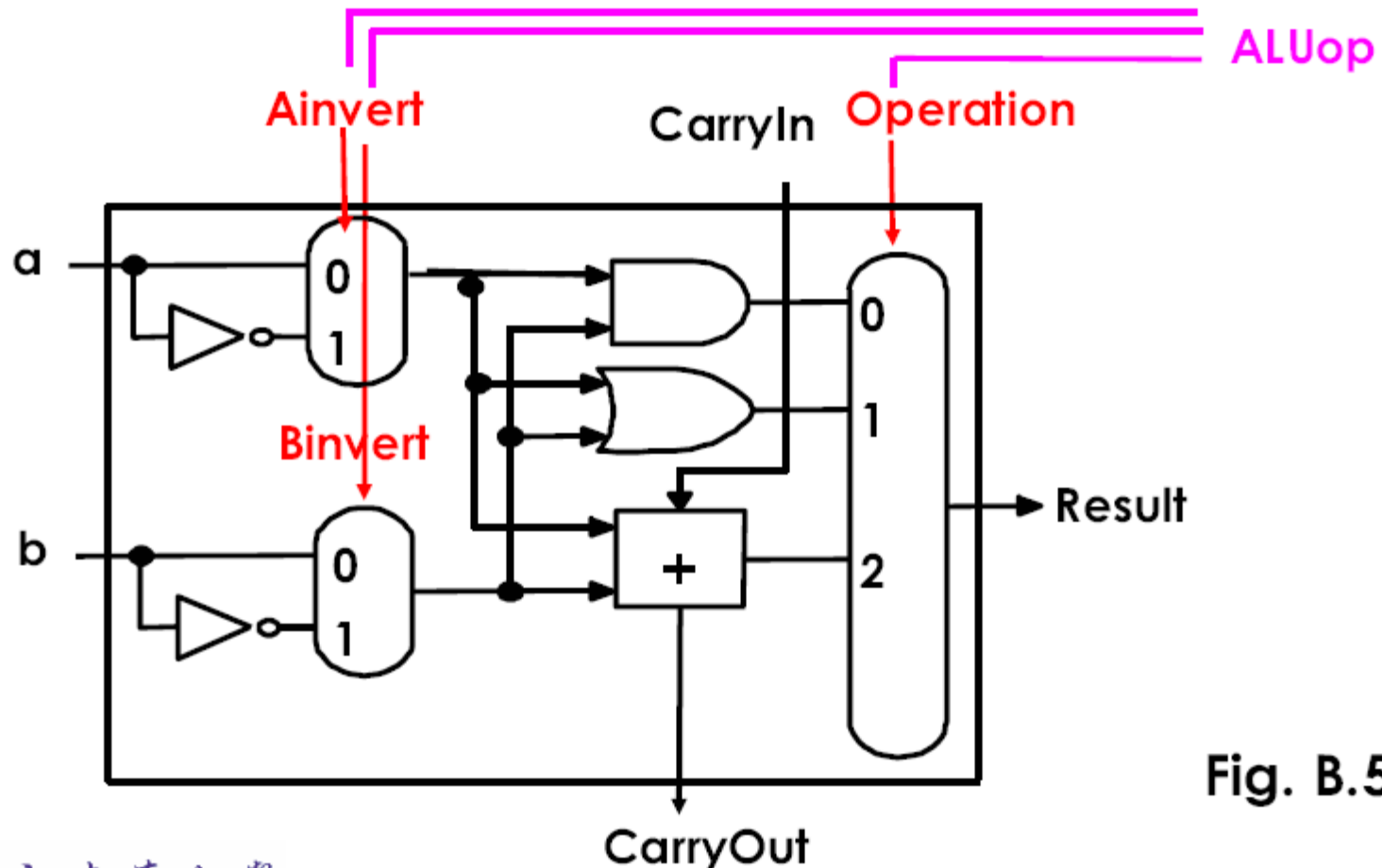


Fig. B.5.9

Overflow

Decimal	Binary	Decimal	2's complement
0	0000	0	0000
1	0001	-1	1111
2	0010	-2	1110
3	0011	-3	1101
4	0100	-4	1100
5	0101	-5	1011
6	0110	-6	1010
7	0111	-7	1001
		-8	1000

Ex: $7 + 3 = 10$ but ...

$$\begin{array}{r}
 \begin{array}{ccccccc}
 0 & 1 & 1 & 1 & & & \\
 \swarrow & \nwarrow & \nwarrow & \nwarrow & & & \\
 0 & 0 & 1 & 1 & 1 & 1 & 7 \\
 + & 0 & 0 & 1 & 1 & 3 & \\
 \hline
 & 1 & 0 & 1 & 0 & -6 &
 \end{array}
 \end{array}$$

$-4 - 5 = -9$ but ...

$$\begin{array}{r}
 \begin{array}{ccccccc}
 1 & 0 & 0 & 0 & & & \\
 \swarrow & \nwarrow & \nwarrow & \nwarrow & & & \\
 1 & 1 & 1 & 0 & 0 & -4 \\
 + & 1 & 0 & 1 & 1 & -5 & \\
 \hline
 & 0 & 1 & 1 & 1 & 7 &
 \end{array}
 \end{array}$$

Overflow Detection

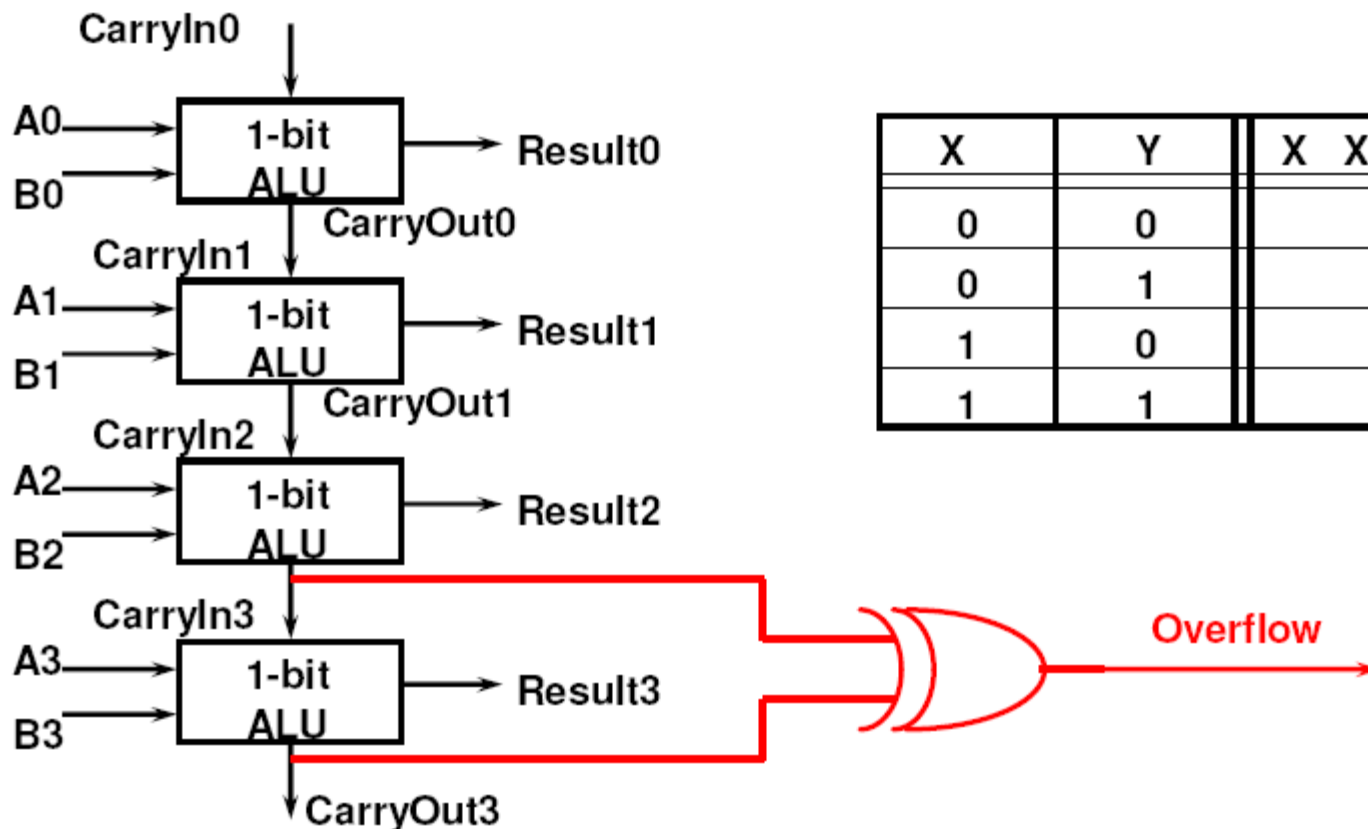
- ◆ **Overflow: result too big/small to represent**
 - $-8 \leq 4\text{-bit binary number} \leq 7$
 - When adding operands with different signs, overflow cannot occur!
 - Overflow occurs when adding:
 - 2 positive numbers and the sum is negative
 - 2 negative numbers and the sum is positive $\Rightarrow$ sign bit is set with the value of the result
 - Overflow if: Carry into MSB $\neq$ Carry out of MSB

Diagram illustrating a 4-bit ripple-carry adder. The inputs are 0011 (3) and 0001 (1). The sum is 0100 (4). The carry-in and carry-out are both 1, highlighted in a red box.

$$\begin{array}{r}
 1000 \\
 + 1101 \\
 \hline
 0111
 \end{array}$$

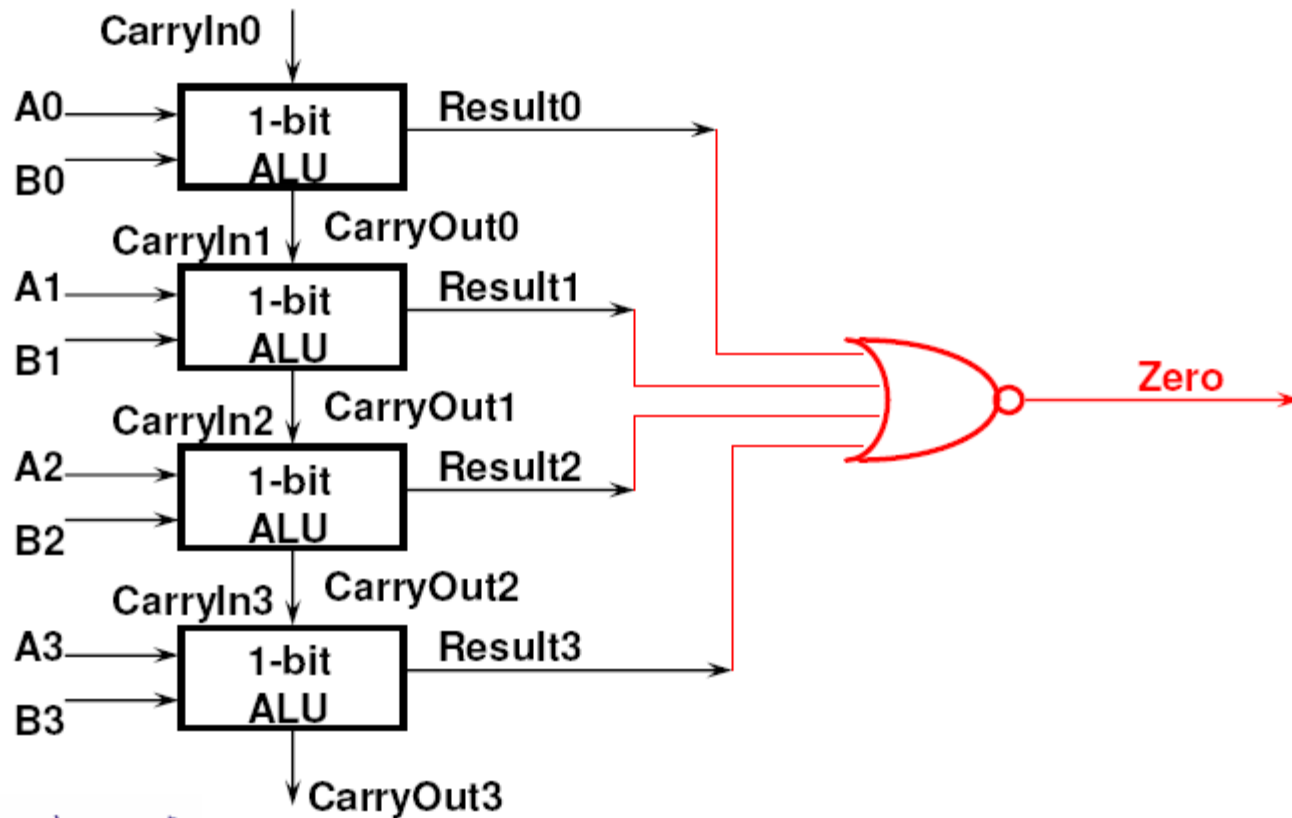
Overflow Detection Logic

- ◆ $\text{Overflow} = \text{CarryIn}[N-1] \text{ XOR } \text{CarryOut}[N-1]$



Zero Detection Logic

- Zero Detection Logic is a one BIG NOR gate



Set on Less Than (I)

◆ 1-bit in ALU

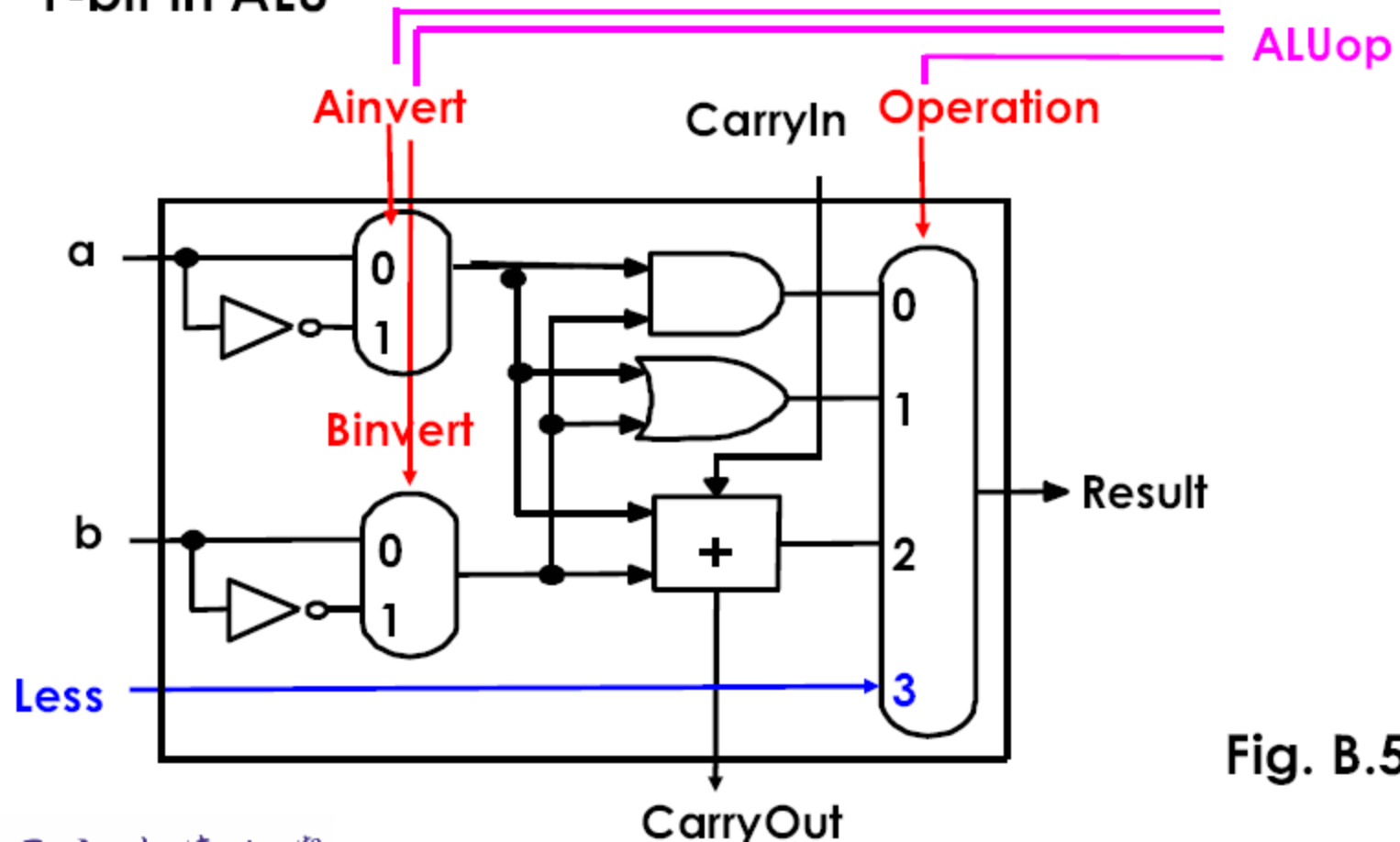


Fig. B.5.10a

Set on Less than (II)

◆ Sign bit in ALU

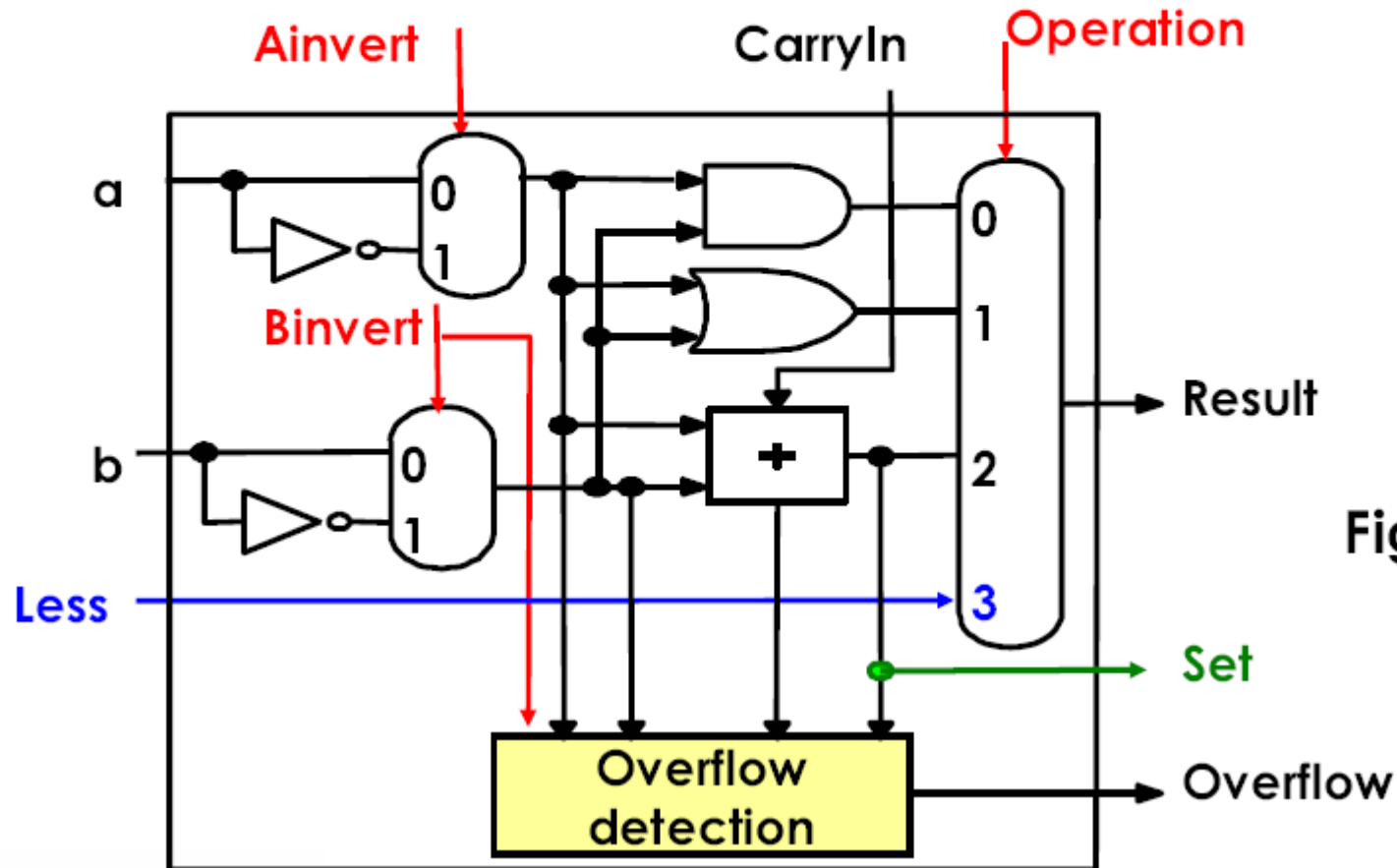
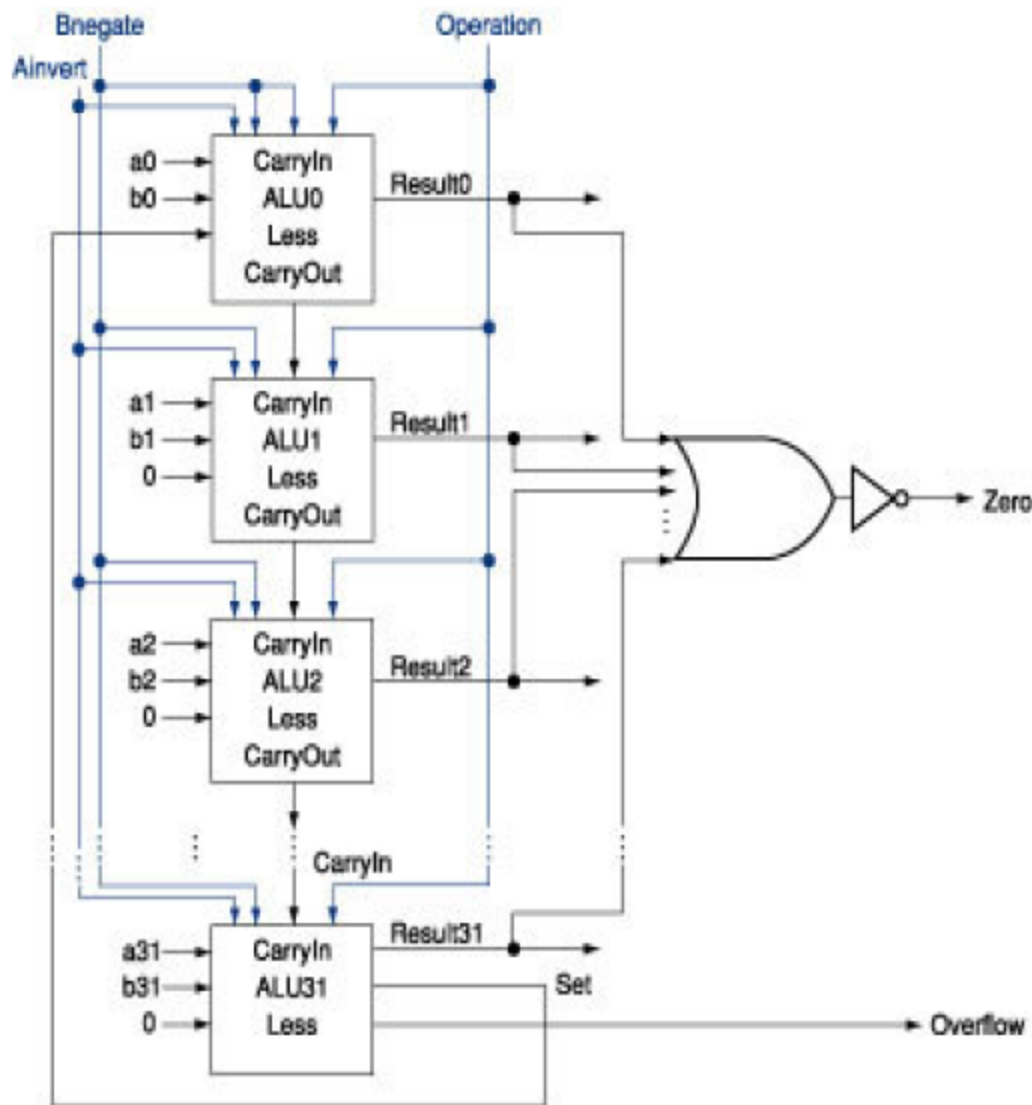


Fig. 5.10b

A Ripple Carry Adder

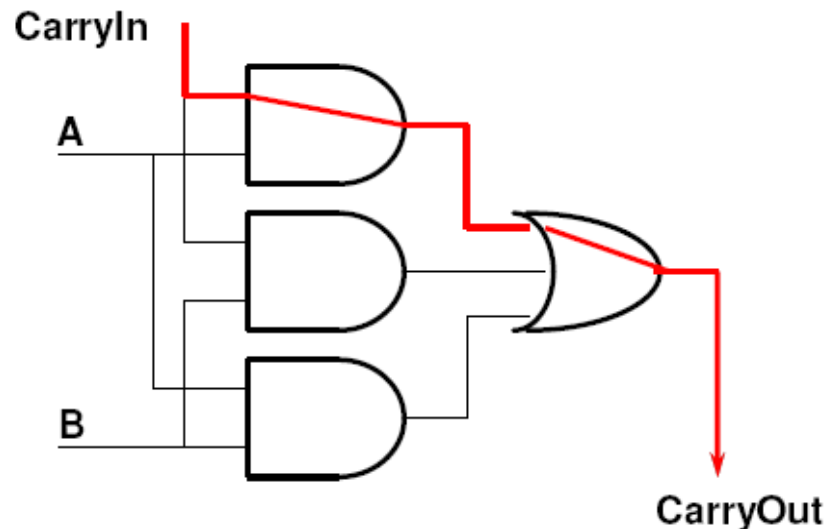
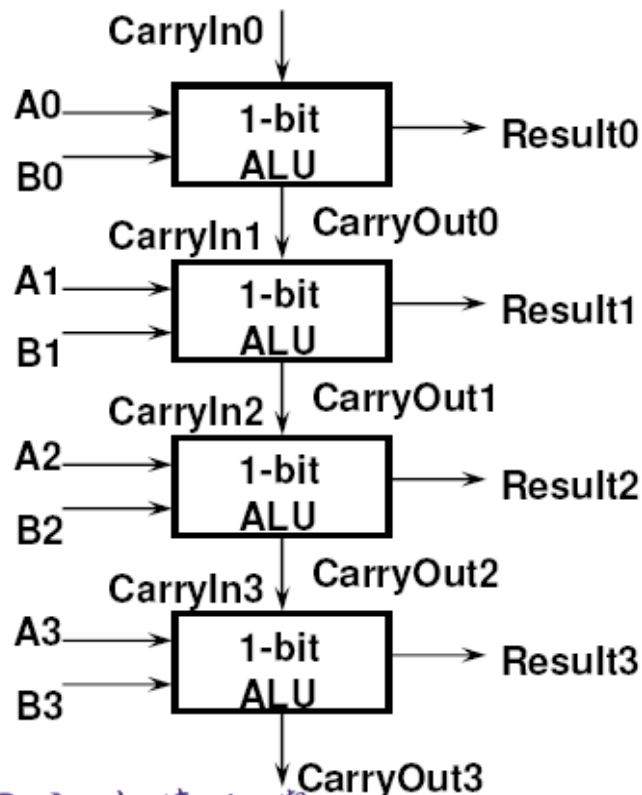


ALUop	Function
0000	and
0001	or
0010	add
0110	subtract
0111	set-less-than
1100	nor

Fig. B.5.12

Problems with Ripple Carry Adder

- ◆ Carry bit may have to propagate from LSB to MSB => worst case delay: N-stage delay



Design Trick: look for parallelism and throw hardware at it

Unsigned Multiply

- ♦ Paper and pencil example (unsigned):

Multiplicand
Multiplier

$$\begin{array}{r} 1000_{\text{ten}} \\ \times 1001_{\text{ten}} \\ \hline 1000 \end{array}$$

0000

0000

1000

Product

$$\begin{array}{r} 1000 \\ \hline 01001000_{\text{ten}} \end{array}$$

- ♦ m bits $\times$ n bits = $m+n$ bit product
- ♦ Binary makes it easy:
 - 0 $\Rightarrow$ place 0 (0 $\times$ multiplicand)
 - 1 $\Rightarrow$ place a copy (1 $\times$ multiplicand)
- ♦ 2 versions of multiply hardware and algorithm

Unsigned Multiplier (Ver. 1)

- ◆ 64-bit *multiplicand register* (with 32-bit multiplicand at right half), 64-bit ALU, 64-bit *product register*, 32-bit *multiplier register*

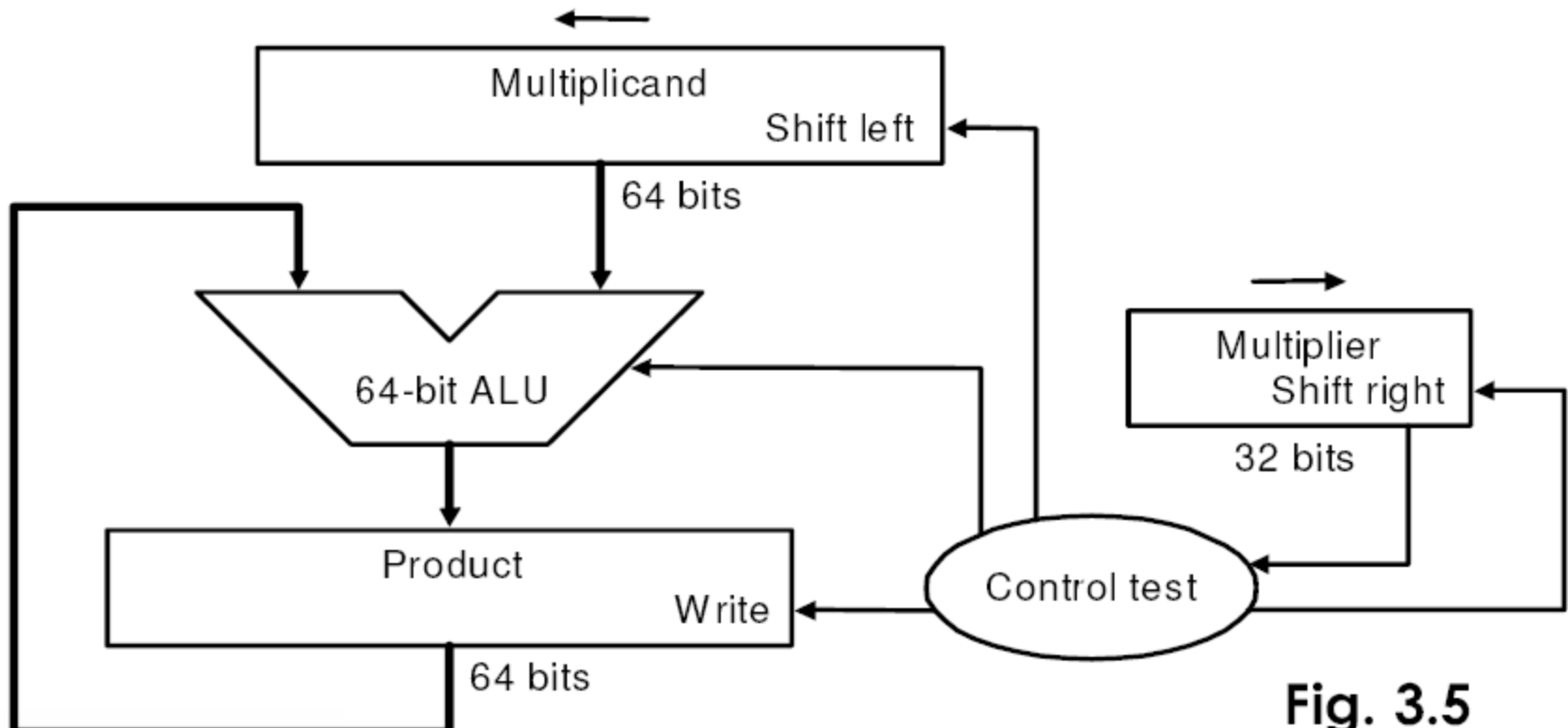
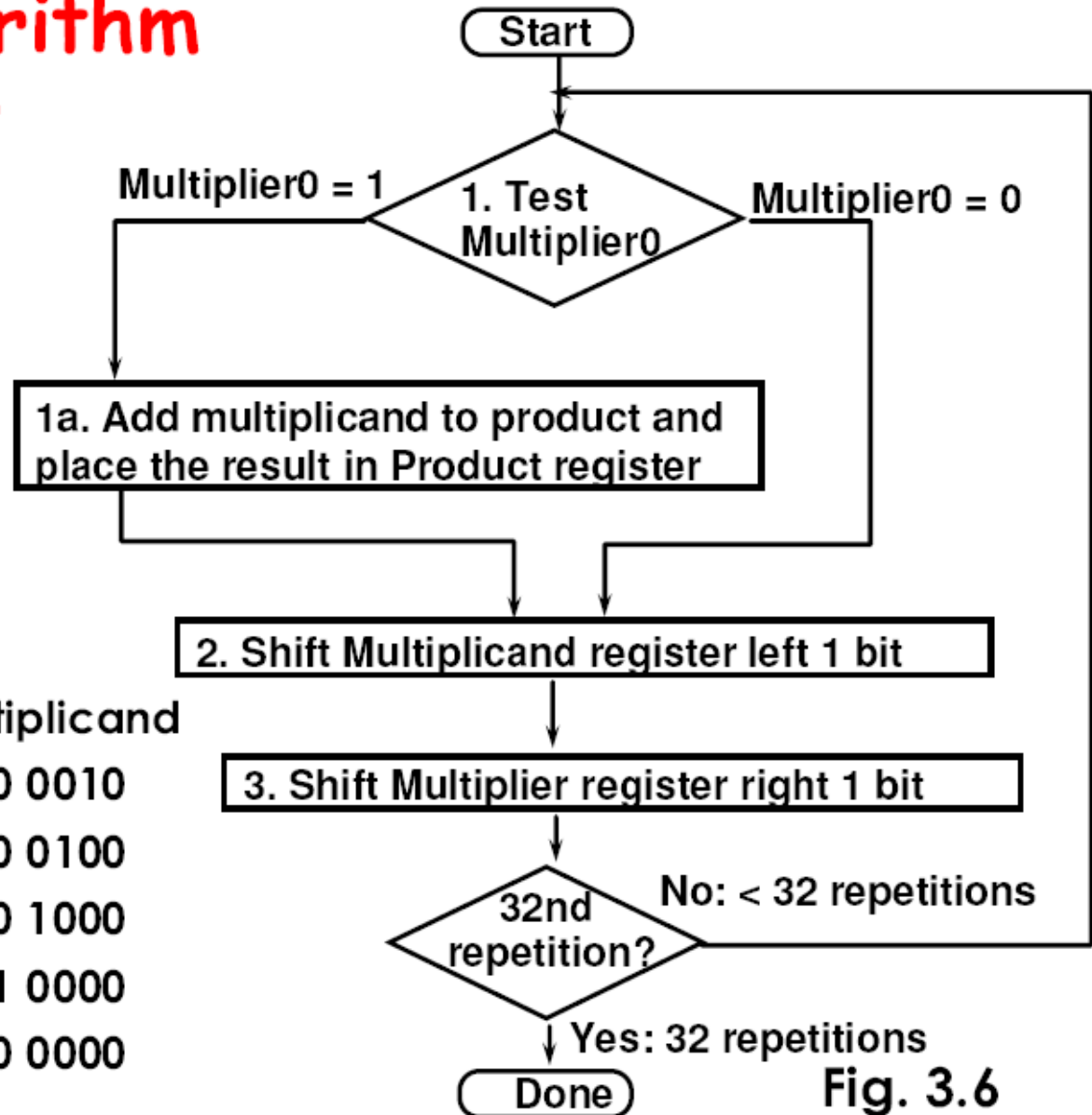


Fig. 3.5

Multiply Algorithm (Ver. 1)



0010 x 0011

Product	Multiplier	Multiplicand
0000 0000	0011	0000 0010
0000 0010	0001	0000 0100
0000 0110	0000	0000 1000
0000 0110	0000	0001 0000
0000 0110	0000	0010 0000

Fig. 3.6

Observations: Multiply Ver. 1

- ◆ 1 clock per cycle => ~100 clocks per multiply
 - Ratio of multiply to add 5:1 to 100:1
- ◆ Half of the bits in multiplicand always 0
=> 64-bit adder is wasted
- ◆ 0's inserted in left of multiplicand as shifted
=> least significant bits of product never changed once formed
- ◆ Instead of shifting multiplicand to left, shift product to right?
- ◆ Product register wastes space => combine Multiplier and Product register

Unsigned Multiplier (Ver. 2)

- ◆ 32-bit Multiplicand register, 32-bit ALU, 64-bit Product register (HI & LO in MIPS), (0-bit Multiplier register)

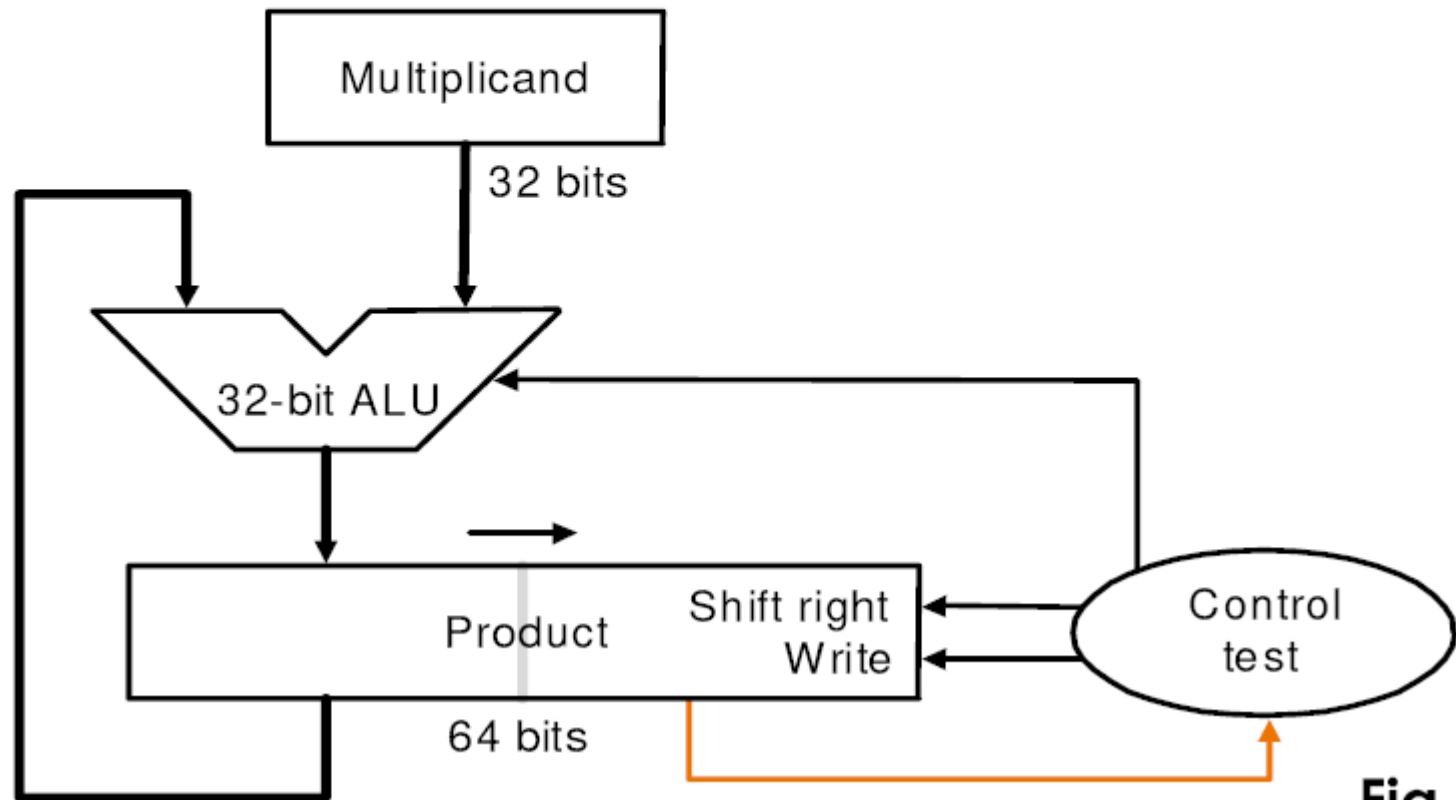
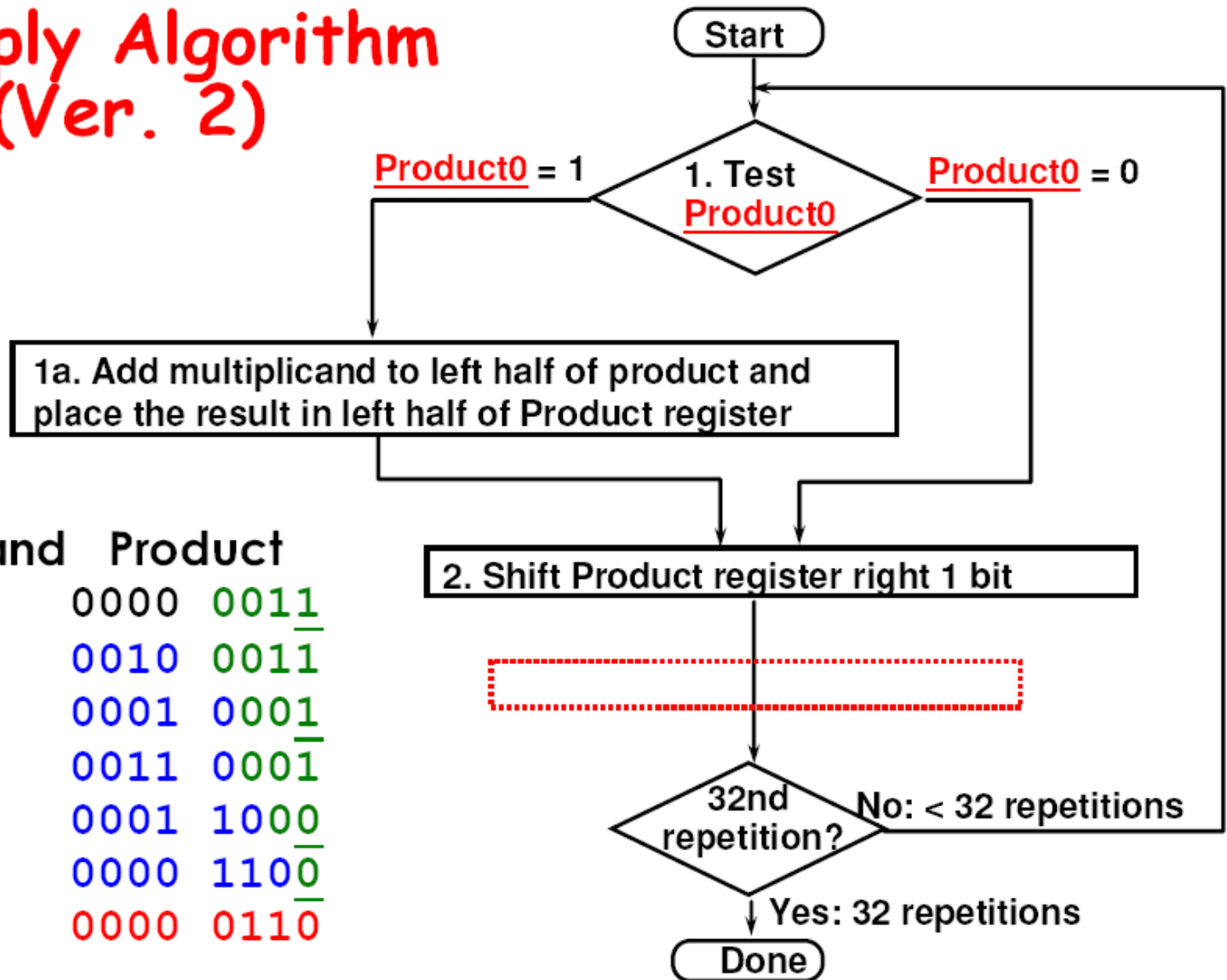


Fig. 3.7

Multiply Algorithm (Ver. 2)



Multiplicand	Product
0010	0000 0011
	0010 0011
0010	0001 0001
	0011 0001
0010	0001 1000
0010	0000 1100
0010	0000 0110

Observations: Multiply Ver. 2

- ◆ 2 steps per bit because multiplier and product registers combined
- ◆ MIPS registers Hi and Lo are left and right half of Product register
=> this gives the MIPS instruction MultU
- ◆ What about signed multiplication?
 - The easiest solution is to make both positive and remember whether to complement product when done (leave out sign bit, run for 31 steps)
 - Apply definition of 2's complement
 - sign-extend partial products and subtract at end
 - Booth's Algorithm is an elegant way to multiply signed numbers using same hardware as before and save cycles

Divide: Paper & Pencil

		1001 _{ten}	Quotient
Divisor	1000 _{ten}	$\overline{)1001010_{ten}}$	Dividend
		-1000	
		10	
		101	
		1010	
		-1000	
		10 _{ten}	Remainder

- ♦ See how big a number can be subtracted, creating quotient bit on each step
Binary $\Rightarrow$ 1 * divisor or 0 * divisor
- ♦ Two versions of divide, successive refinement
- ♦ *Both dividend and divisor are 32-bit positive integers*

Divide Hardware (Version 1)

- ◆ 64-bit *Divisor* register (initialized with 32-bit divisor in left half), 64-bit ALU, 64-bit *Remainder* register (initialized with 64-bit dividend), 32-bit *Quotient* register

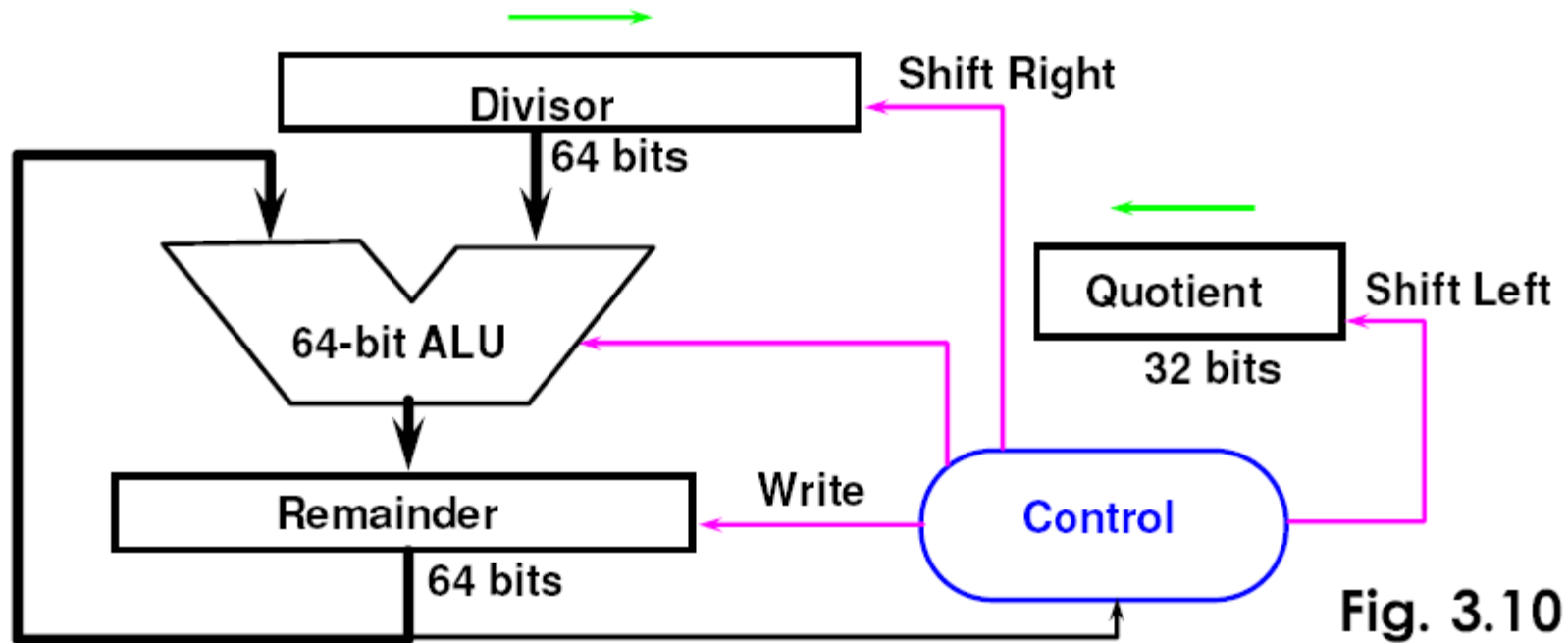


Fig. 3.10

Divide Algorithm (Version 1)

Quot.	Divisor	Rem.
0000	<u>00100000</u>	00000111
		11100111
		00000111
0000	<u>00010000</u>	00000111
		11110111
		00000111
0000	<u>00001000</u>	00000111
		11111111
		00000111
0000	<u>00000100</u>	00000111
		00000011
0001		00000011
0001	<u>00000010</u>	00000011
		00000001
0011		00000001
0011	<u>00000001</u>	00000001

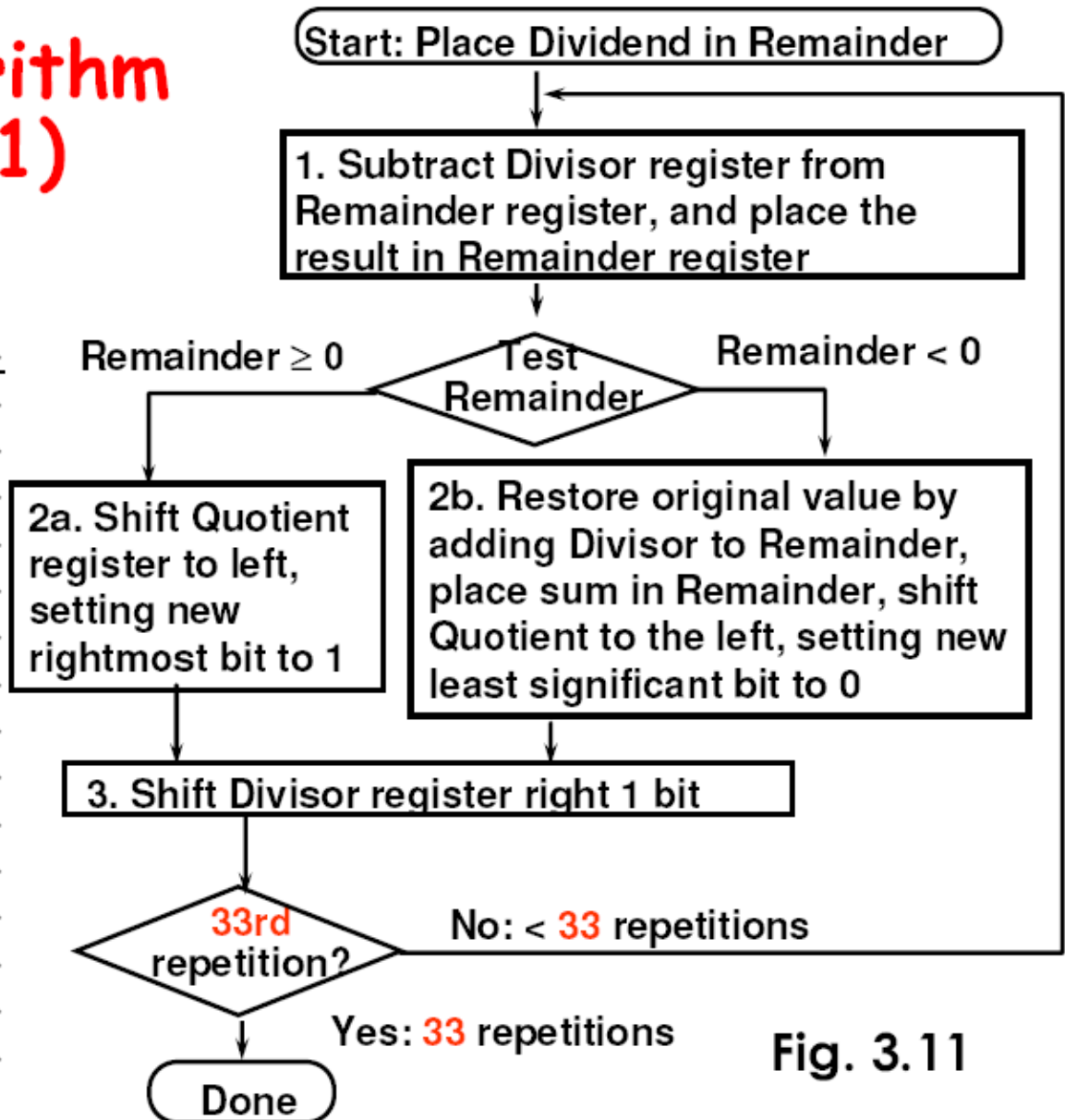


Fig. 3.11

Observations: Divide Version 1

- ◆ Half of the bits in divisor register always 0
=> 1/2 of 64-bit adder is wasted
=> 1/2 of divisor is wasted
- ◆ Instead of shifting divisor to right, shift remainder to left?
- ◆ 1st step cannot produce a 1 in quotient bit (otherwise quotient is too big for the register)
=> switch order to shift first and then subtract
=> save 1 iteration
- ◆ Eliminate Quotient register by combining with Remainder register as shifted left

Divide Hardware (Version 2)

- ◆ 32-bit Divisor register, 32-bit ALU, 64-bit Remainder register, (0-bit Quotient register)

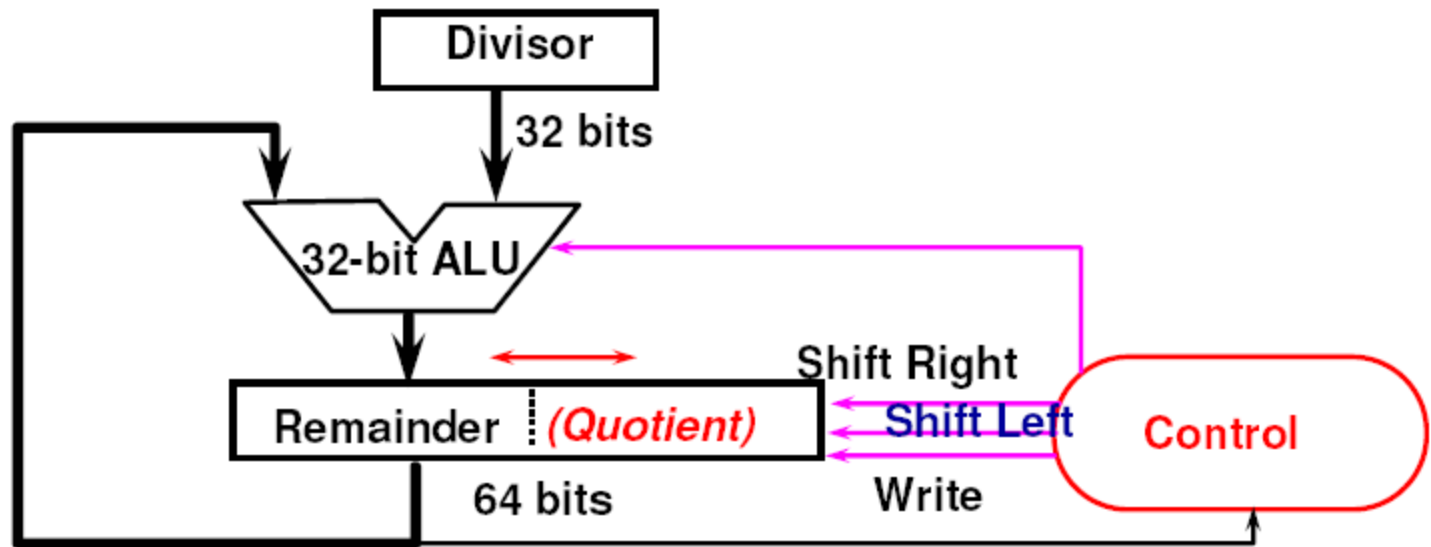
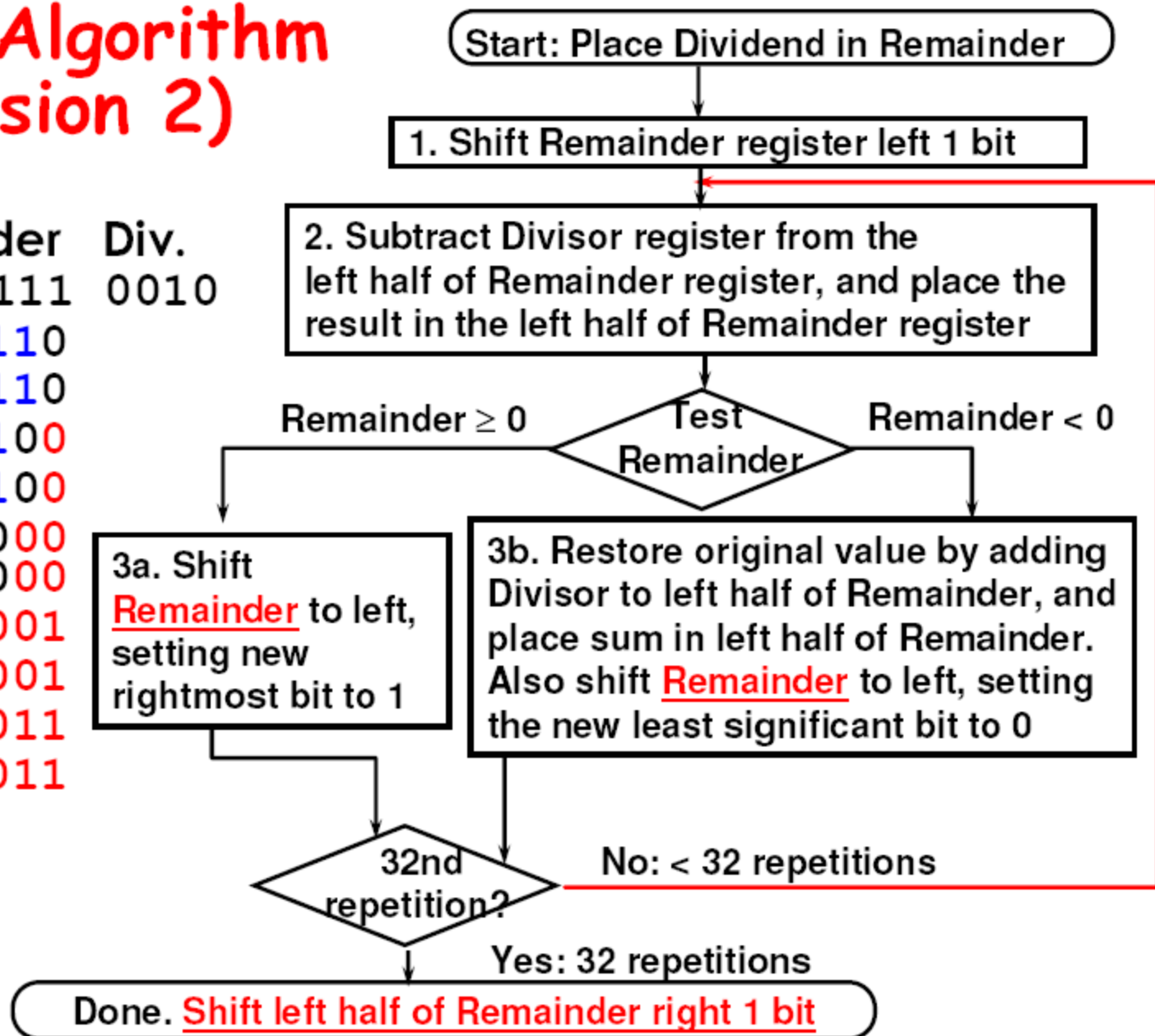


Fig. 3.13

Divide Algorithm (Version 2)

Step	Remainder	Div.
0	0000 0111	0010
1.1	0000 1110	
1.2	1110 1110	
1.3b	0001 1100	
2.2	1111 1100	
2.3b	0011 1000	
3.2	0001 1000	
3.3a	0011 0001	
4.2	0001 0001	
4.3a	0010 0011	
	0001 0011	

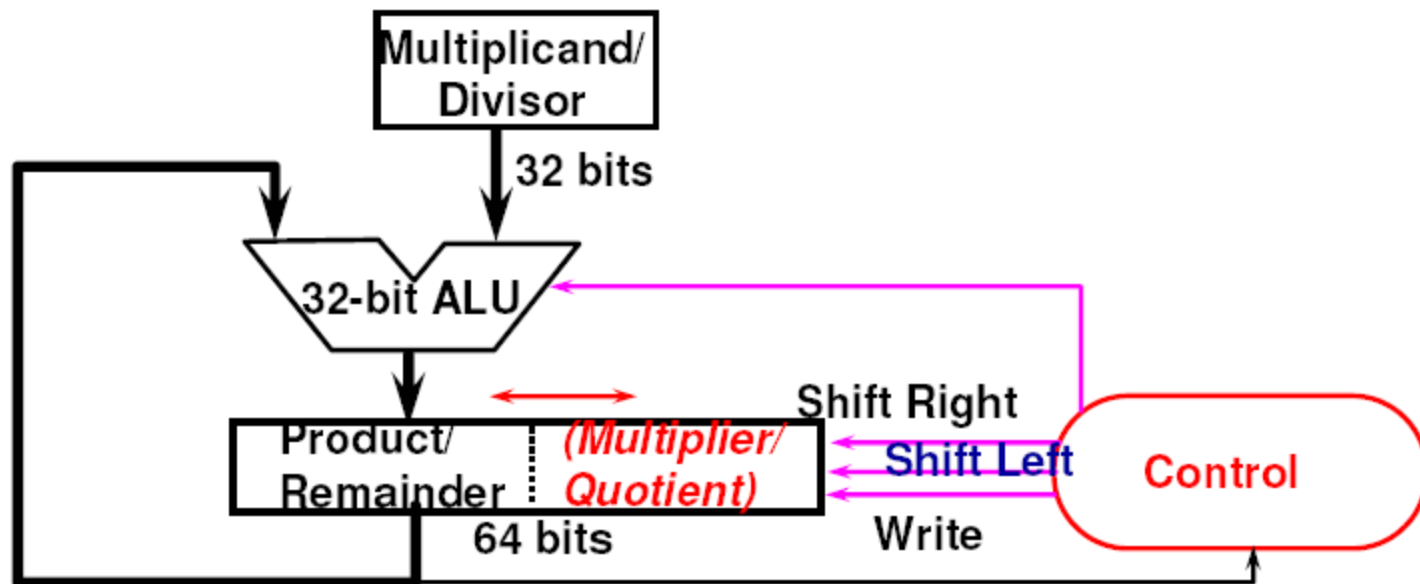


Observations: Multiply and Divide

- ◆ Same hardware as multiply: just need ALU to add or subtract, and 64-bit register to shift left or shift right
- ◆ Hi and Lo registers in MIPS combine to act as 64-bit register for multiply and divide

Multiply/Divide Hardware

- ◆ 32-bit Multiplicand/Divisor register, 32-bit ALU, 64-bit Product/Remainder register, (0-bit Multiplier/Quotient register)



Floating-Point: Motivation

◆ What can be represented in N bits?

Unsigned	0	to	$2^n - 1$
2's Complement	-2^{n-1}	to	$2^{n-1} - 1$
1's Complement	$-2^{n-1} + 1$	to	2^{n-1}
Excess M	$-M$	to	$2^n - M - 1$

◆ But, what about ...

- very large numbers?

9,349,398,989,787,762,244,859,087,678

- very small number?

0.0000000000000000000000000045691

- rationals $2/3$

- irrationals $\sqrt{2}$

- transcendentals e, π

Scientific Notation: Binary

Significand (Mantissa) *exponent*

1.0_{two} x 2⁻¹

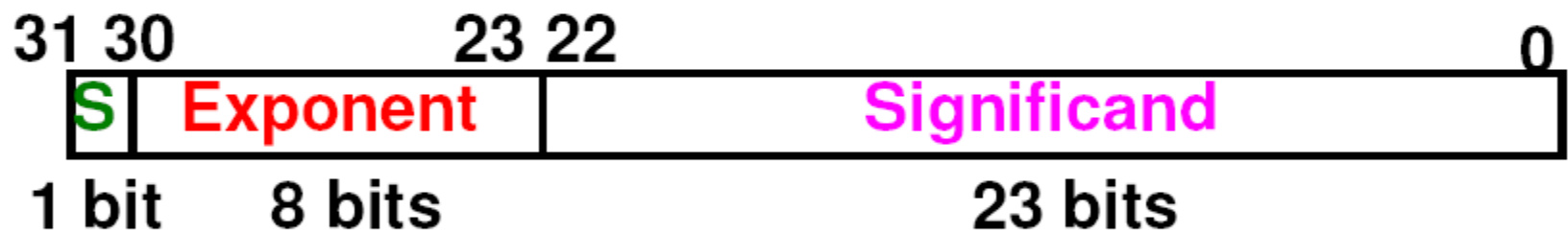
“**binary point**” *radix (base)*

The diagram shows the expression $1.0_{\text{two}} \times 2^{-1}$. An arrow points from the label "Significand (Mantissa)" to the "1.0" part. Another arrow points from "exponent" to the "⁻¹" part. A third arrow points from "radix (base)" to the "2" part. A fourth arrow points from "“binary point”" to the "." part.

- ◆ Computer arithmetic that supports it is called floating point, because the binary point is not fixed, as it is for integers
- ◆ Normalized form: no leading 0s
(exactly one digit to left of decimal point)
- ◆ Alternatives to represent 1/1,000,000,000
 - Normalized: 1.0×10^{-9}
 - Not normalized: 0.1×10^{-8} , 10.0×10^{-10}

FP Representation

- ◆ Normal format: $1.\text{xxxxxxxxxx}_{\text{two}} \times 2^{\text{yyyy}_{\text{two}}}$
- ◆ Want to put it into multiple words: 32 bits for *single-precision* and 64 bits for *double-precision*
- ◆ A simple single-precision representation:



S represents sign

Exponent represents y 's

Significand represents x 's

- ◆ Represent numbers as small as 2.0×10^{-38} to as large as 2.0×10^{38}

IEEE 754 Standard (2/4)

◆ Exponent:

- Need to represent positive and negative exponents
- Also want to compare FP numbers as if they were integers, to help in value comparisons
- If use 2's complement to represent?
e.g., 1.0×2^{-1} versus $1.0 \times 2^{+1}$ ($1/2$ versus 2)

1/2

0	1111 1111	000 0000 0000 0000 0000 0000
---	-----------	------------------------------

2

0	0000 0001	000 0000 0000 0000 0000 0000
---	-----------	------------------------------

*If we use integer comparison for these two words,
we will conclude that $1/2 > 2$!!!*

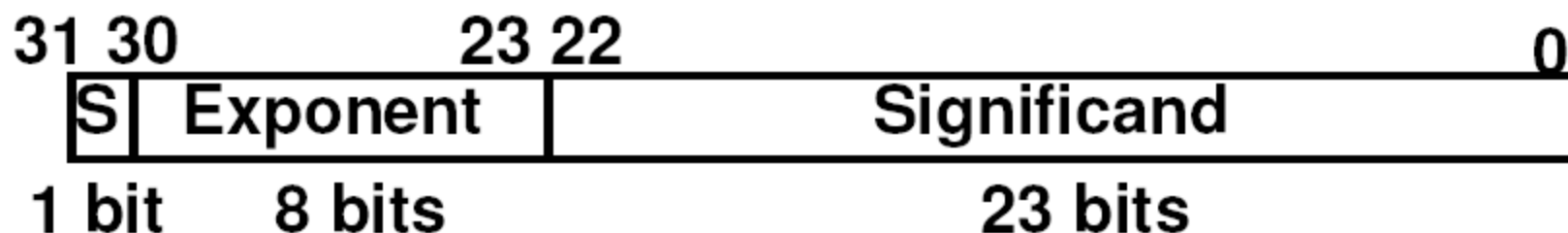
IEEE 754 Standard (3/4)

- ◆ Instead, let notation 0000 0000 be most negative, and 1111 1111 most positive
- ◆ Called biased notation, where bias is the number subtracted to get the real number
 - IEEE 754 uses bias of 127 for single precision:
Subtract 127 from Exponent field to get actual value for exponent
 - 1023 is bias for double precision

1/2	0	0111 1110	000 0000 0000 0000 0000 0000
2	0	1000 0000	000 0000 0000 0000 0000 0000

IEEE 754 Standard (4/4)

- ◆ Summary (single precision):



$$(-1)^S \times (1.\text{Significand}) \times 2^{(\text{Exponent}-127)}$$

- ◆ Double precision identical, except with exponent bias of 1023

Example: FP to Decimal

0	0110 1000	101 0101 0100 0011 0100 0010
---	-----------	------------------------------

- ◆ Sign: 0 => positive
- ◆ Exponent:
 - $0110\ 1000_{\text{two}} = 104_{\text{ten}}$
 - Bias adjustment: $104 - 127 = -23$
- ◆ Significand:
 - $1 + 2^{-1} + 2^{-3} + 2^{-5} + 2^{-7} + 2^{-9} + 2^{-14} + 2^{-15} + 2^{-17} + 2^{-22}$
 $= 1.0 + 0.666115$
- ◆ Represents: $1.666115_{\text{ten}} \times 2^{-23} \approx 1.986 \times 10^{-7}$

Example 1: Decimal to FP

- ◆ Number = - 0.75
 - = - $0.11_{\text{two}} \times 2^0$ (scientific notation)
 - = - $1.1_{\text{two}} \times 2^{-1}$ (normalized scientific notation)
- ◆ Sign: negative => 1
- ◆ Exponent:
 - Bias adjustment: $-1 + 127 = 126$
 - $126_{\text{ten}} = 0111\ 1110_{\text{two}}$

1	0111 1110	100 0000 0000 0000 0000 0000
---	-----------	------------------------------

Example 2: Decimal to FP

- ◆ A more difficult case: representing $1/3$?

$$= 0.33333..._{10} = 0.0101010101..._2 \times 2^0$$

$$= 1.0101010101..._2 \times 2^{-2}$$

- Sign: 0
- Exponent = $-2 + 127 = 125_{10} = 01111101_2$
- Significand = $0101010101...$

0	0111 1101	0101 0101 0101 0101 0101 010
---	-----------	------------------------------

Zero and Special Numbers

- ◆ What have we defined so far? (single precision)

<u>Exponent</u>	<u>Significand</u>	<u>Object</u>
0	0	<u>???</u>
0	nonzero	<u>???</u>
1-254	anything	+/- floating-point
255	0	<u>???</u>
255	nonzero	<u>???</u>

Representation for 0

◆ Represent 0?

- exponent all zeroes
- significand all zeroes too
- What about sign?
- +0: 0 00000000 000000000000000000000000
- -0: 1 00000000 000000000000000000000000

◆ Why two zeroes?

- Helps in some limit comparisons

Special Numbers

- ◆ What have we defined so far? (single precision)

<u>Exponent</u>	<u>Significand</u>	<u>Object</u>
0	0	0
0	nonzero	<u>???</u>
1-254	anything	+/- floating-point
255	0	<u>???</u>
255	nonzero	<u>???</u>

- ◆ Range:

$$1.0 \times 2^{-126} \approx 1.8 \times 10^{-38}$$

What if result too small? (>0 , $< 1.8 \times 10^{-38} \Rightarrow$ Underflow!)

$$(2 - 2^{-23}) \times 2^{127} \approx 3.4 \times 10^{38}$$

What if result too large? ($> 3.4 \times 10^{38} \Rightarrow$ Overflow!)

Representation for +/- Infinity

- ◆ In FP, divide by zero should produce +/- infinity, not overflow
- ◆ Why?
 - OK to do further computations with infinity, e.g., $X/0 > Y$ may be a valid comparison
- ◆ IEEE 754 represents +/- infinity
 - Most positive exponent reserved for infinity
 - Significands all zeroes

S	1111 1111	0000 0000 0000 0000 0000 000
---	-----------	------------------------------

Representation for Not a Number

- ◆ What do I get if I calculate $\text{sqrt}(-4.0)$ or $0/0$?
 - If infinity is not an error, these should not be either
 - They are called *Not a Number* (NaN)
 - Exponent = 255, Significand nonzero
- ◆ Why is this useful?
 - Hope NaNs help with debugging?
 - They contaminate: $\text{op}(\text{NaN}, X) = \text{NaN}$
 - OK if calculate but don't use it

Special Numbers (cont'd)

- ◆ What have we defined so far? (single-precision)

<u>Exponent</u>	<u>Significand</u>	<u>Object</u>
0	0	0
0	nonzero	denom (???)
1-254	anything	+/- fl. pt. #
255	0	+/- infinity
255	nonzero	NaN

Floating-Point Addition

Basic addition algorithm:

- (1) compute $Y_e - X_e$ (to align binary point)
- (2) right shift the smaller number, say X_m , that many positions to form $X_m \times 2^{X_e - Y_e}$
- (3) compute $X_m \times 2^{X_e - Y_e} + Y_m$

if demands normalization, then normalize:

- (4) left shift result, decrement result exponent
right shift result, increment result exponent
 - (4.1) check overflow or underflow during the shift
 - (4.2) round the mantissacontinue until MSB of data is 1
(NOTE: Hidden bit in IEEE Standard)
- (5) if result is 0 mantissa, set the exponent

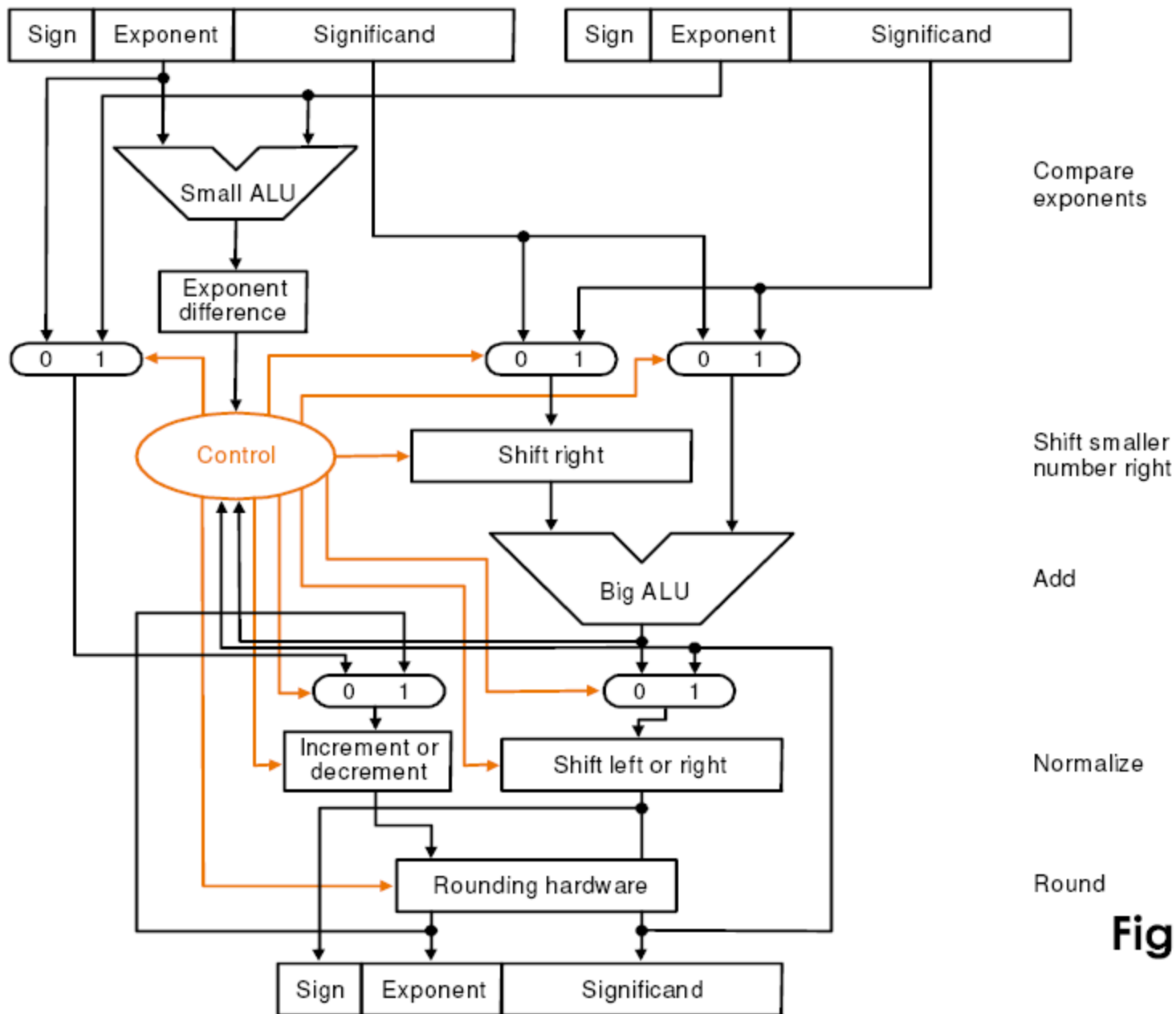


Fig. 3.17

Floating-Point Multiplication

Basic multiplication algorithm

(1) add exponents of operands to get exponent of product

doubly biased exponent must be corrected:

$$X_e = 7 \quad X_e = 1111 \quad = 15 \quad = 7 + 8$$

$$Y_e = -3 \quad Y_e = 0101 \quad = 5 \quad = -3 + 8$$

$$\text{Excess } 8 \quad 10100 \quad 20 \quad 4 + 8 + 8$$

need extra subtraction step of the bias amount

(2) multiplication of operand mantissa

(3) normalize the product

(3.1) check overflow or underflow during the shift

(3.2) round the mantissa

continue until MSB of data is 1

(4) set the sign of product

MIPS Floating Point

- ◆ Separate floating point instructions:
 - Single precision: `add.s`, `sub.s`, `mul.s`, `div.s`
 - Double precision: `add.d`, `sub.d`, `mul.d`, `div.d`
- ◆ FP part of the processor:
 - contains 32 32-bit registers: `$f0`, `$f1`, ...
 - most registers specified in `.s` and `.d` instruction refer to this set
 - separate load and store: `lwc1` and `swc1`
 - Double Precision: by convention, even/odd pair contain one DP FP number: `$f0/$f1`, `$f2/$f3`
 - Instructions to move data between main processor and coprocessors:
 - `mfc0`, `mtc0`, `mfc1`, `mtc1`, etc.
 - See CD A-73 to A-80

Floating Point Fallacy

◆ FP add, subtract associative?

FALSE!

- $x = -1.5 \times 10^{38}, y = 1.5 \times 10^{38}, z = 1.0$
- $x + (y + z) = -1.5 \times 10^{38} + (1.5 \times 10^{38} + 1.0)$
 $= -1.5 \times 10^{38} + (1.5 \times 10^{38}) = 0.0$
- $(x + y) + z = (-1.5 \times 10^{38} + 1.5 \times 10^{38}) + 1.0$
 $= (0.0) + 1.0 = 1.0$

◆ Therefore, Floating Point add, subtract are not associative!

- Why? FP result approximates real result!
- This example: 1.5×10^{38} is so much larger than 1.0 that $1.5 \times 10^{38} + 1.0$ in floating point representation is still 1.5×10^{38}

第 3 章

計算機的算術

3.1 介紹

3.2 加法與減法

3.3 乘法

3.4 除法

3.5 浮點(加分、乘法)

3.6 平行性與計算機算術：部分字的平行性

3.7 實例：x86 中串流式SIMD 延伸與先進的向量(處理)延伸

3.8 走得更快：部分字平行性與矩陣乘法

3.9 謬誤與陷阱

3.10 總結評論

3.11 歷史觀點與進一步閱讀

3.1 介紹

▣ 本章目標

- 實數表示法
- 算術演算法
- 演算法的硬體
- 所有這些對指令集的意義

3.2 加法與減法

▣ 計算機中的加法

- 各位數由右向左逐位元相加，進位則向左方一個位數傳遞

▣ 減法利用到加法：

- 減數運算元會先被變號再加至被減數

▣ 對2的補數而言，當運算的結果無法以其所具有的硬體，來表示時，即發生溢位(overflow)

3.2 加法與減法

- ▣ 加法中溢位何時可能發生？
 - 當加的兩個運算元符號不同時，溢位不會發生
- ▣ 減法中溢位的發生有類似的限制
 - 當兩個運算元的符號相同時，溢位不可能發生
- ▣ 當其發生時，我們該如何偵測得知？
 - 溢位發生的情況是
 - 加兩個正數其和為負，或是反之
 - 從正數減去一個負數而得負值，或是從一個負數減去一個正數而得正值

Overflow Detection

- ◆ **Overflow: result too big/small to represent**
 - $-8 \leq 4\text{-bit binary number} \leq 7$
 - When adding operands with different signs, overflow cannot occur!
 - Overflow occurs when adding:
 - 2 positive numbers and the sum is negative
 - 2 negative numbers and the sum is positive $\Rightarrow$ sign bit is set with the value of the result
 - Overflow if: Carry into MSB $\neq$ Carry out of MSB

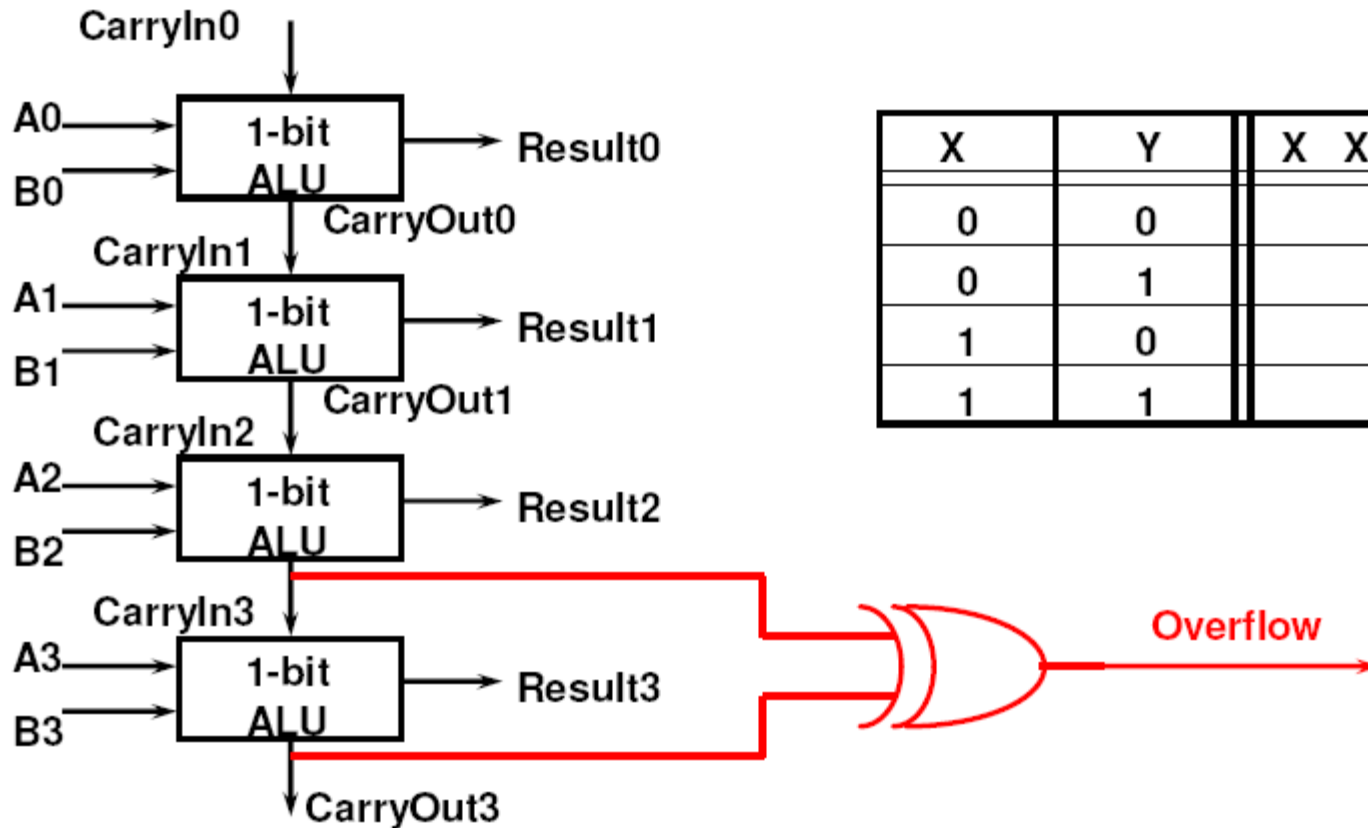
The diagram shows a 4-bit ripple-carry adder. The inputs are 0111 (7) and 0011 (3). The carry-in is 1. The carry-out is 1. The sum is 1010 (-6). The carry-out is highlighted in red.

Carry	7	3	Sum
1	0	0	1
0	1	0	0
0	1	1	1
0	1	1	0

$$\begin{array}{r}
 1000 \\
 + 1101 \\
 \hline
 0111
 \end{array}$$

Overflow Detection Logic

- ◆ $\text{Overflow} = \text{CarryIn}[N-1] \text{ XOR } \text{CarryOut}[N-1]$



3.2 加法與減法

▣ MIPS 中的做法

- 加(add)、加立即值(addi)及減(sub)在溢位時呼叫例外處理
- 無號加(addu)、無號加立即值(addiu)、及無號減(subu)在溢位時不呼叫例外處理

硬體/軟體介面

- MIPS 以例外(exception)的方式來偵測溢位
 - MIPS 中含有一個稱為例外程式計數器(exception program counter, EPC)的暫存器來存放造成例外的指令的位址

- 飽和運算(saturating operations)：
當一運算溢位時，則結果將被設為(反映運算結果的)最大的正值或最負的值
- 加法的速度可經由及早決定較高位元處的進位位元來提昇
 - 最常用者稱為進位前瞻(carry lookahead)

3.3 乘法

▣ 暫時假設我們作正數的相乘

Unsigned Multiply

- ◆ Paper and pencil example (unsigned):

Multiplicand 被乘數	1000 _{ten}
Multiplier 乘數	X 1001 _{ten}
	1000
	0000
	0000
	1000
Product 乘積	01001000 _{ten}

- ◆ m bits x n bits = m+n bit product
- ◆ Binary makes it easy:
 - 0 => place 0 (0 x multiplicand)
 - 1 => place a copy (1 x multiplicand)
- ◆ 2 versions of multiply hardware and algorithm

Unsigned Multiplier (Ver. 1)

- ◆ 64-bit *multiplicand register* (with 32-bit multiplicand at right half), 64-bit ALU, 64-bit *product register*, 32-bit *multiplier register*

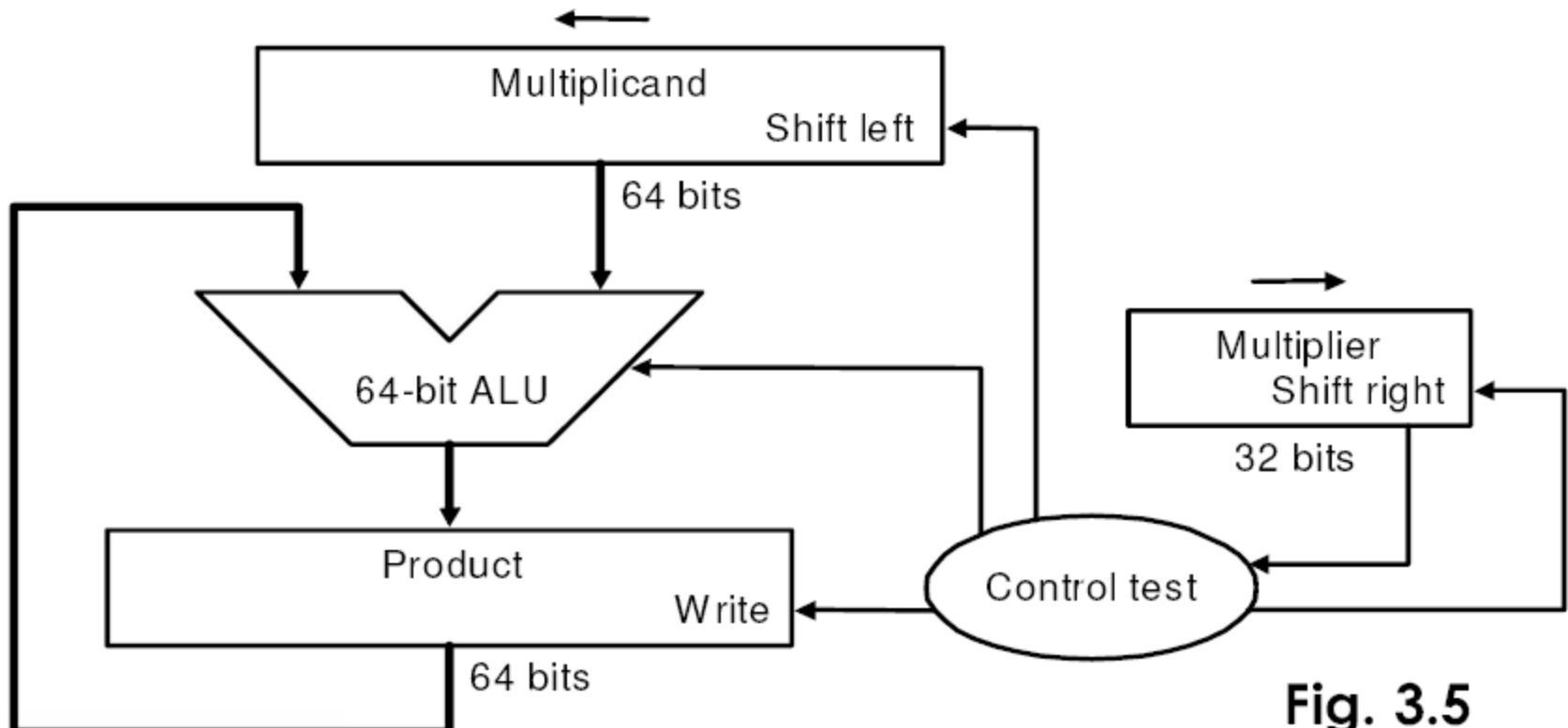


Fig. 3.5

循序版本的乘法演算法及硬體

圖3.3 乘法硬體的第一個版本

被乘數暫存器、算術邏輯單元及乘積暫存器都是64位元寬，只有乘數暫存器是32位元寬。(附錄C中有算術邏輯單元的說明。) 32位元的被乘數先被放置在被乘數暫存器的右半部，並在每一步驟中左移1位元。乘數則在每一步驟中往相反方向移位。本演算法由乘積被初始化為0開始。控制單元決定何時要將被乘數與乘數暫存器移位以及何時將新值寫入乘積暫存器



循序版本的乘法演算法及硬體

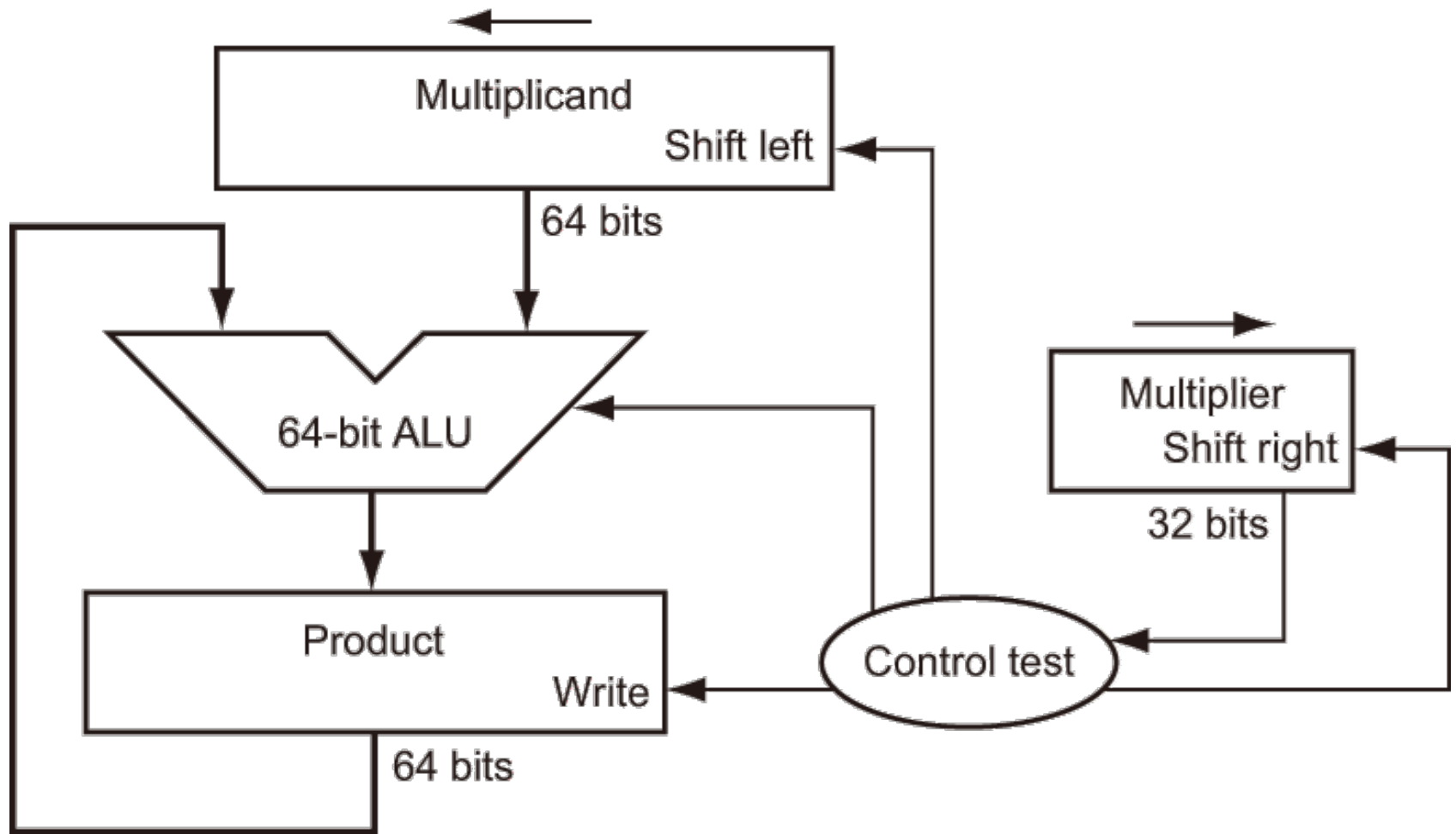
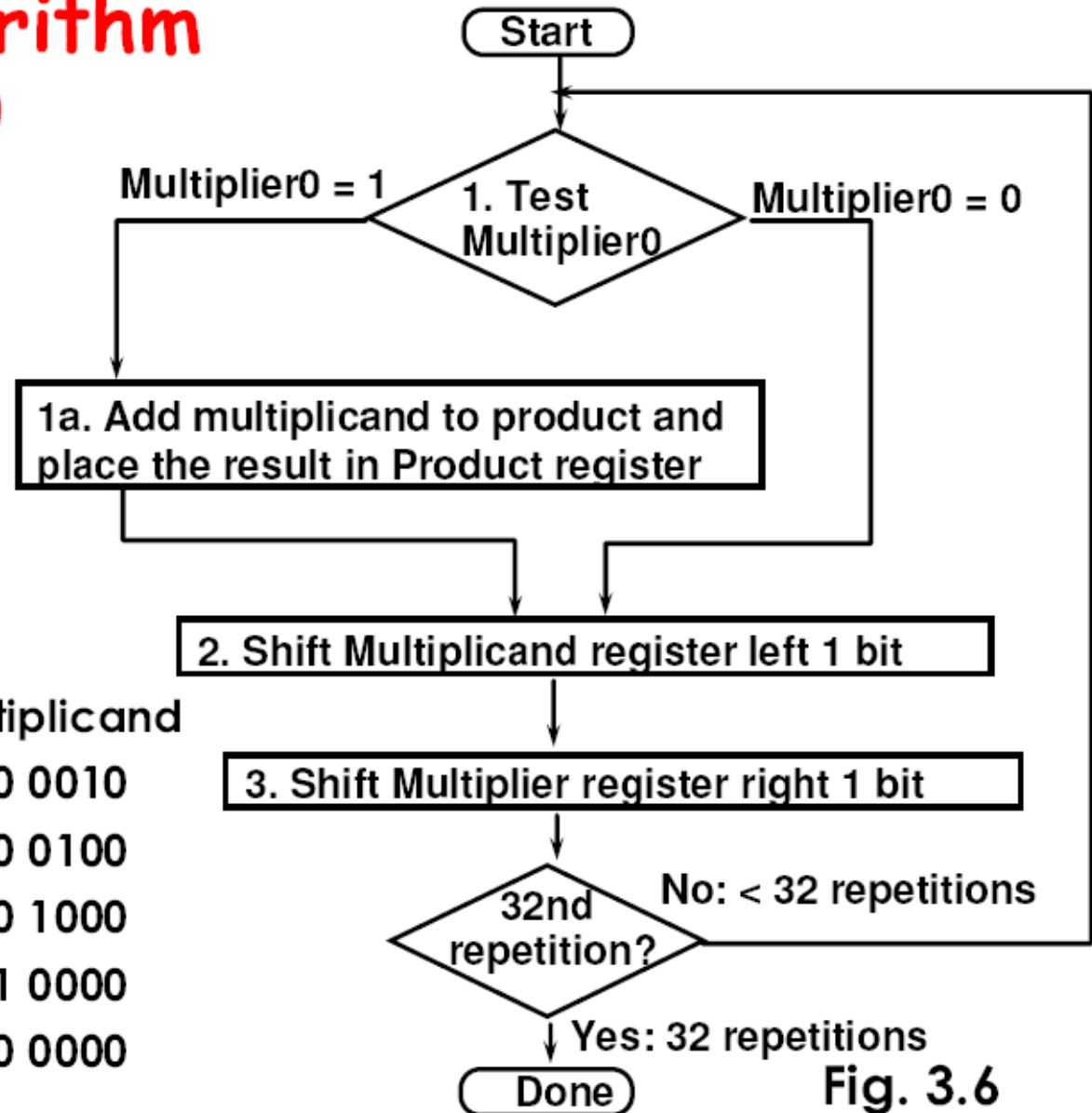


圖3.3 乘法硬體的第一個版本

Multiply Algorithm (Ver. 1)



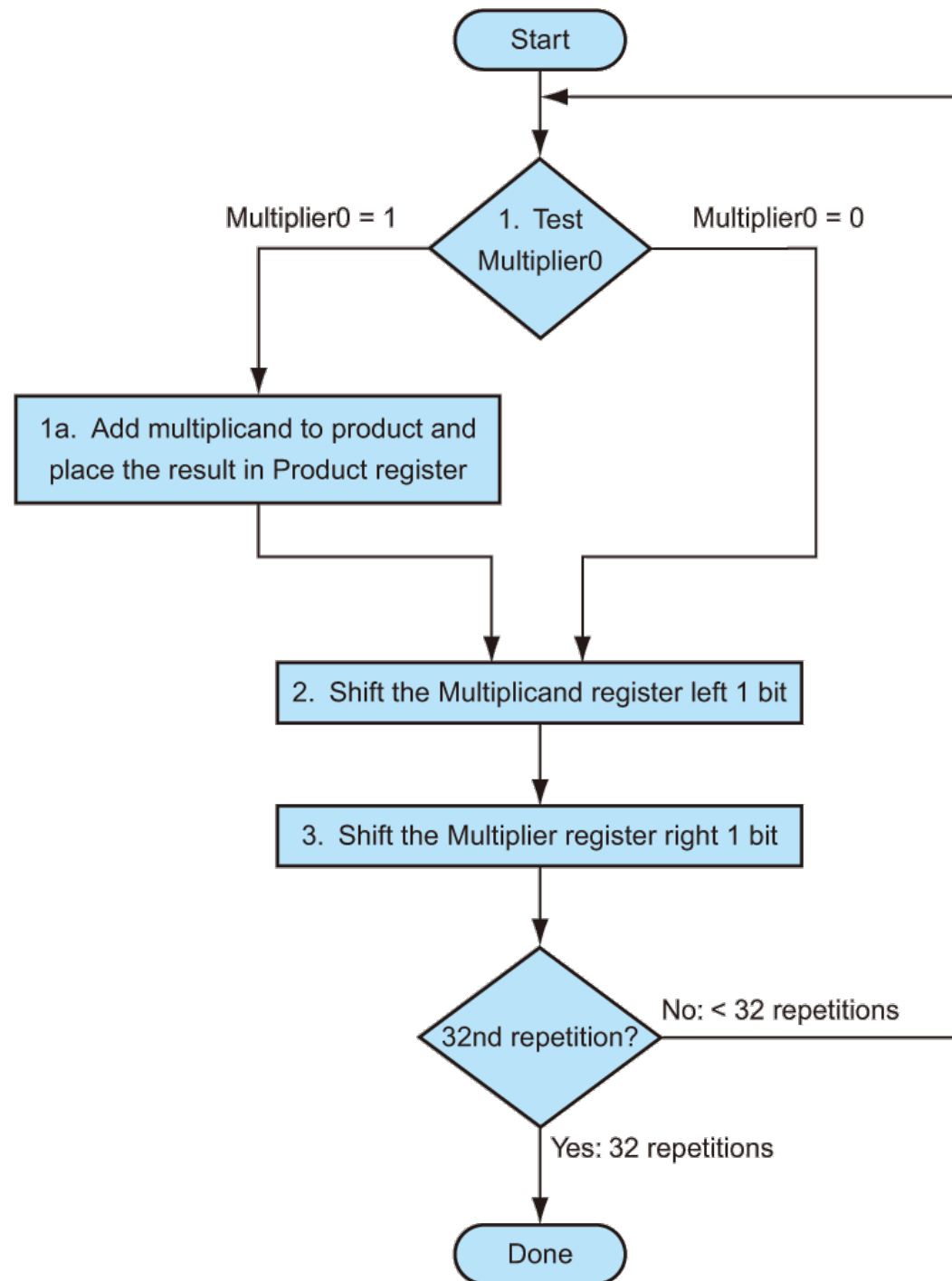
0010 x 0011

Product	Multiplier	Multiplicand
0000 0000	0011	0000 0010
0000 0010	0001	0000 0100
0000 0110	0000	0000 1000
0000 0110	0000	0001 0000
0000 0110	0000	0010 0000

Fig. 3.6

圖3.4 使用圖3.3 中硬體的第一個乘法演算法

如果乘數的最低位元為1，將被乘數加到乘積。若否，則前往下一步驟。接下來兩個步驟將被乘數左移與乘數右移。這三個步驟會重複32次



Observations: Multiply Ver. 1

- ◆ 1 clock per cycle => ~100 clocks per multiply
 - Ratio of multiply to add 5:1 to 100:1
- ◆ Half of the bits in multiplicand always 0
=> 64-bit adder is wasted
- ◆ 0's inserted in left of multiplicand as shifted
=> least significant bits of product never changed once formed
- ◆ Instead of shifting multiplicand to left, shift product to right?
- ◆ Product register wastes space => combine Multiplier and Product register

循序版本的乘法演算法及硬體

- 假設乘數置於32 位元的乘數暫存器中以及64 位元的乘積暫存器初值為零
- 需將被乘數逐步左移一個位元，以便其可能被加至過渡乘積中
- 32 步之後，32 位元的被乘數將被左移32 位元位置。
 - 需要一個64 位元的被乘數暫存器，其最初值是32 位元的被乘數位於其右半而左半為零

循序版本的乘法演算法及硬體

- 硬體可簡單地改良成每處理一個位元僅需時一時脈週期
 - 平行地進行運作：若乘數最低位元為1 則將被乘數加至積中，同時也將乘數及被乘數移位
 - 經由注意到各暫存器及加法器中有些沒用到的部分
 - 將加法器以及一些暫存器縮減為一半寬
 - 將乘數與乘積共用暫存器

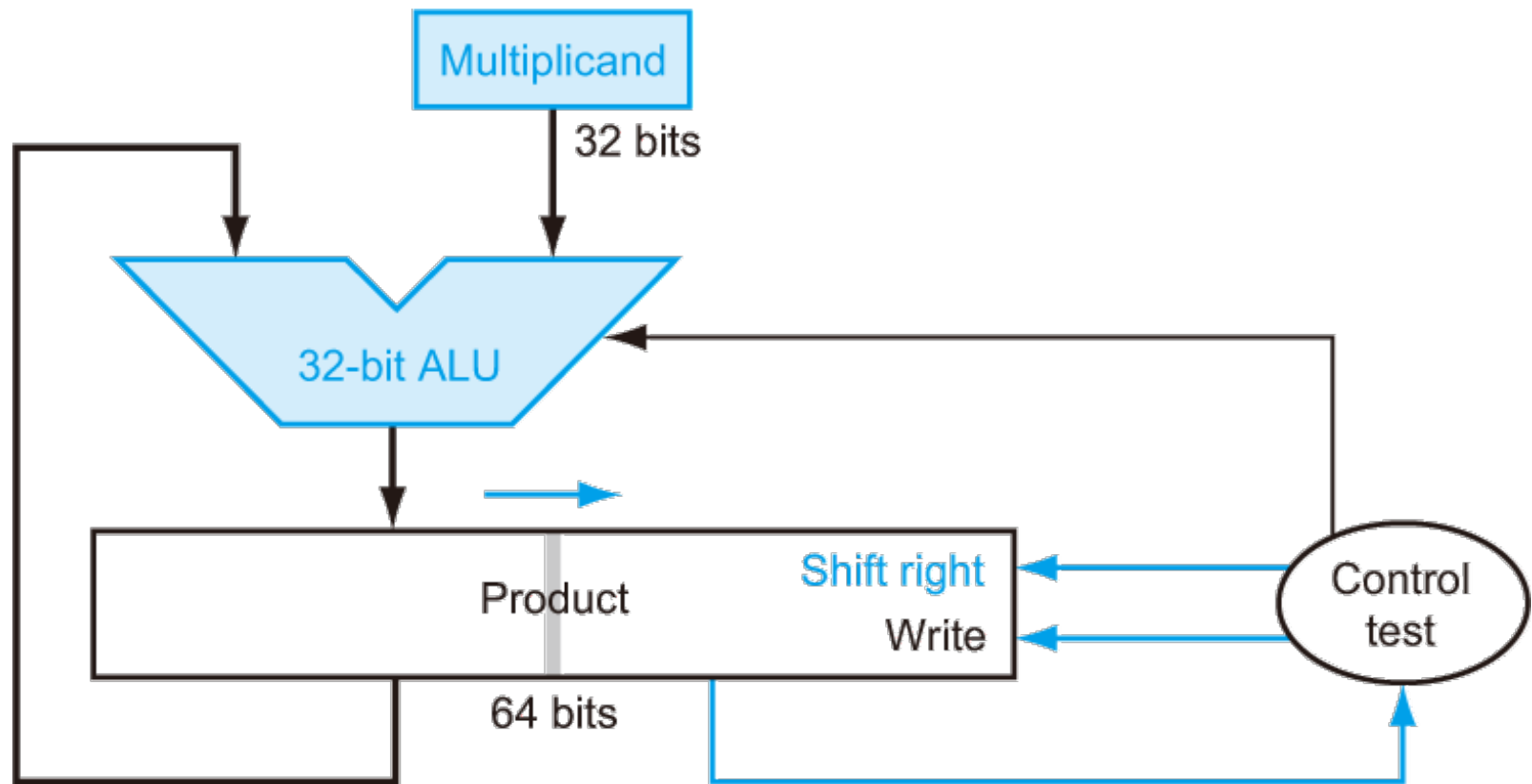


圖3.5 乘法硬體的改良版本

跟圖3.3 的第一版作個比較。被乘數暫存器、算術邏輯單元以及乘數暫存器都是32 位元寬，只有乘積暫存器是64 位元寬。現在乘積會被右移。原本獨立的乘數暫存器也消失了。乘數現在被放置在乘積暫存器的右半部。這些改變以灰色凸顯出來。(乘積暫存器其實應該是65 個位元以存放加法器的進位輸出，不過這裡還是以64 位元表之以強調其由圖3.4 所作的演進。)

Unsigned Multiplier (Ver. 2)

- ◆ 32-bit Multiplicand register, 32-bit ALU, 64-bit Product register (HI & LO in MIPS), (0-bit Multiplier register)

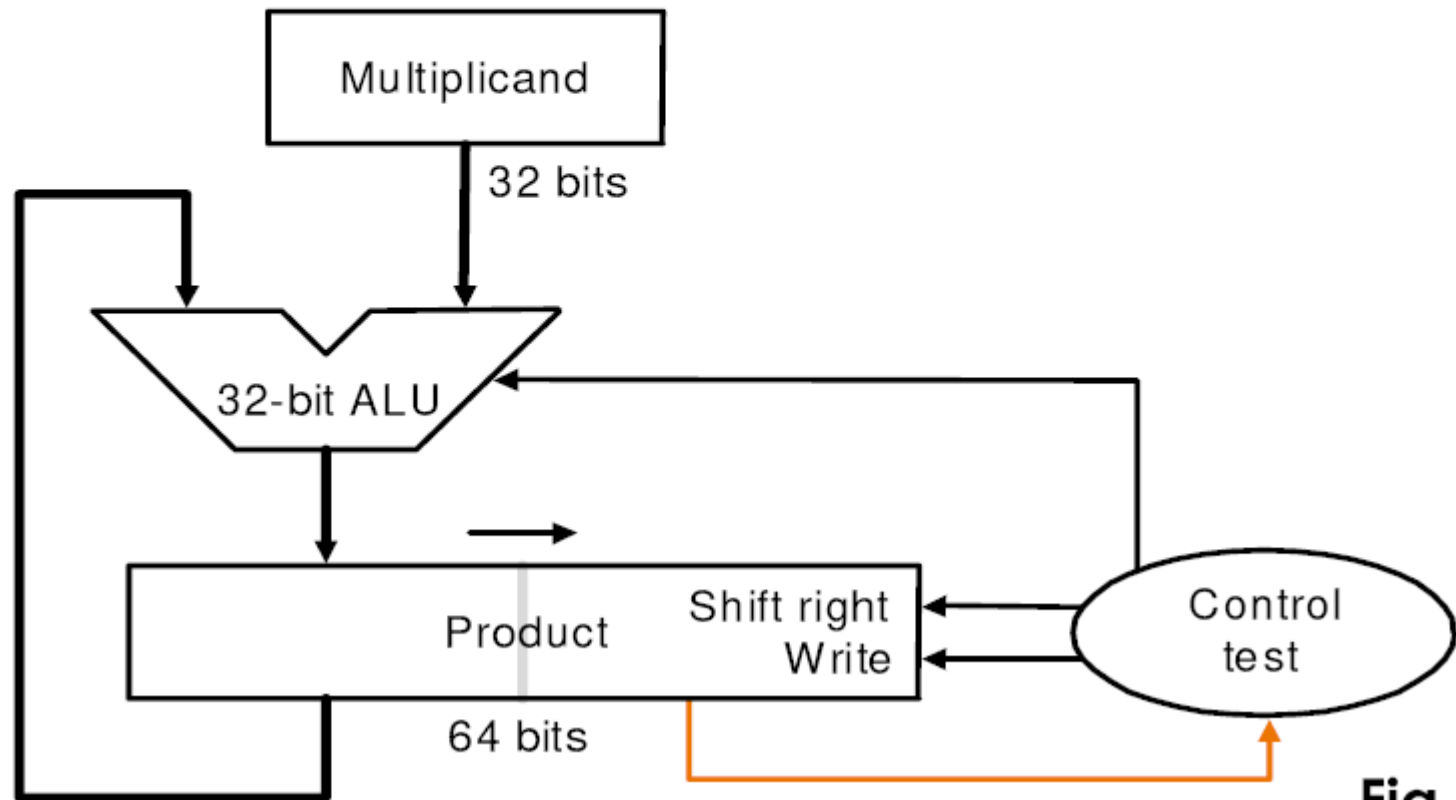
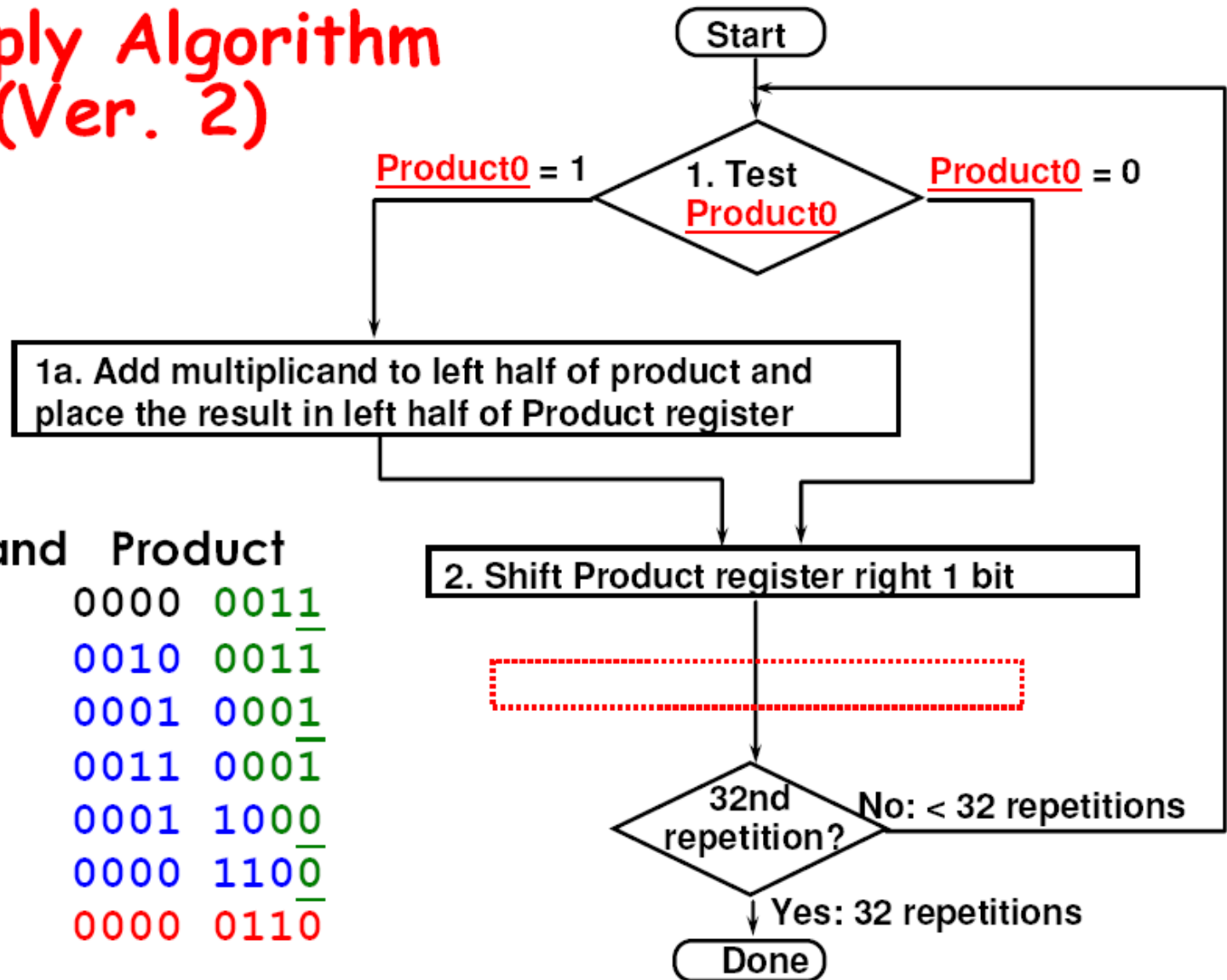


Fig. 3.7

Multiply Algorithm (Ver. 2)



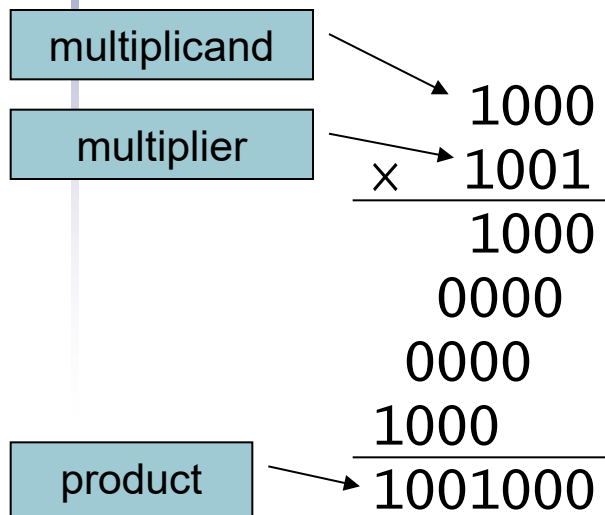
Multiplicand	Product
0010	0000 0011
	0010 0011
0010	0001 0001
	0011 0001
0010	0001 1000
0010	0000 1100
0010	0000 0110

Observations: Multiply Ver. 2

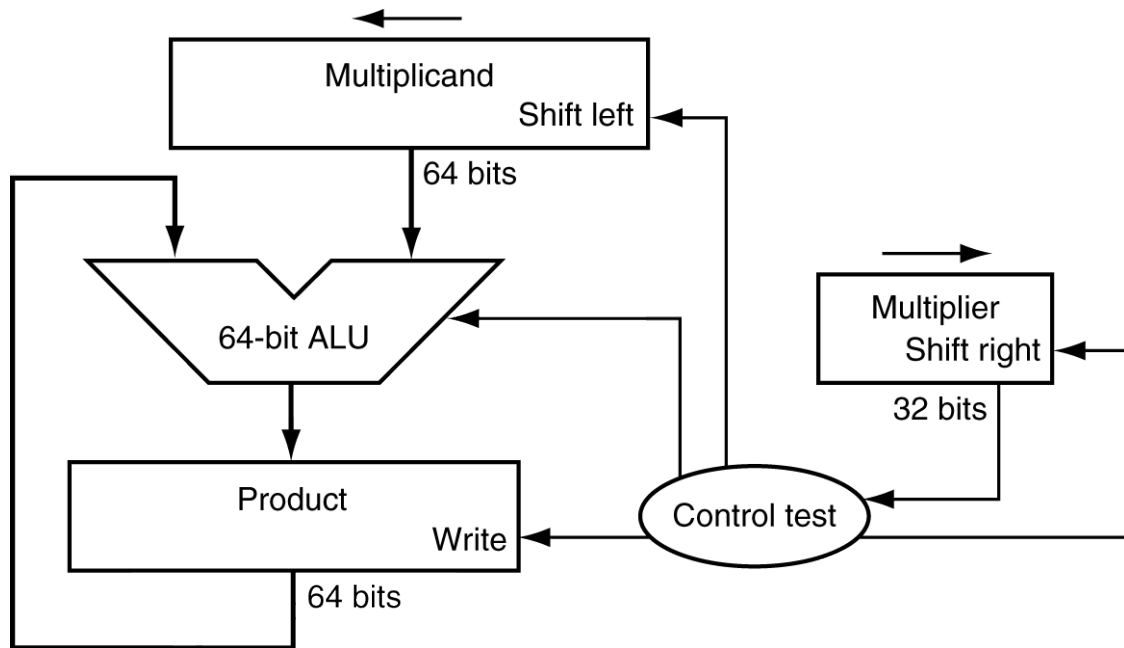
- ◆ 2 steps per bit because multiplier and product registers combined
- ◆ MIPS registers Hi and Lo are left and right half of Product register
=> this gives the MIPS instruction MultU
- ◆ What about signed multiplication?
 - The easiest solution is to make both positive and remember whether to complement product when done (leave out sign bit, run for 31 steps)
 - Apply definition of 2's complement
 - sign-extend partial products and subtract at end
 - Booth's Algorithm is an elegant way to multiply signed numbers using same hardware as before and save cycles

Multiplication

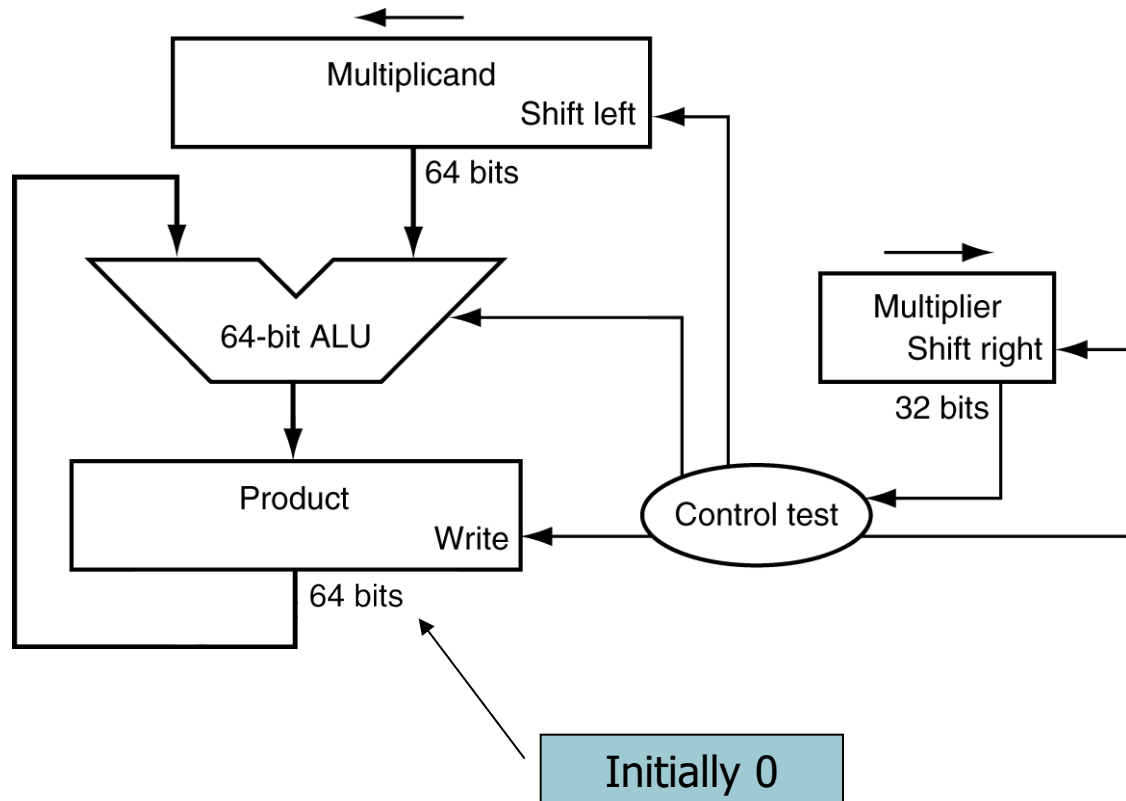
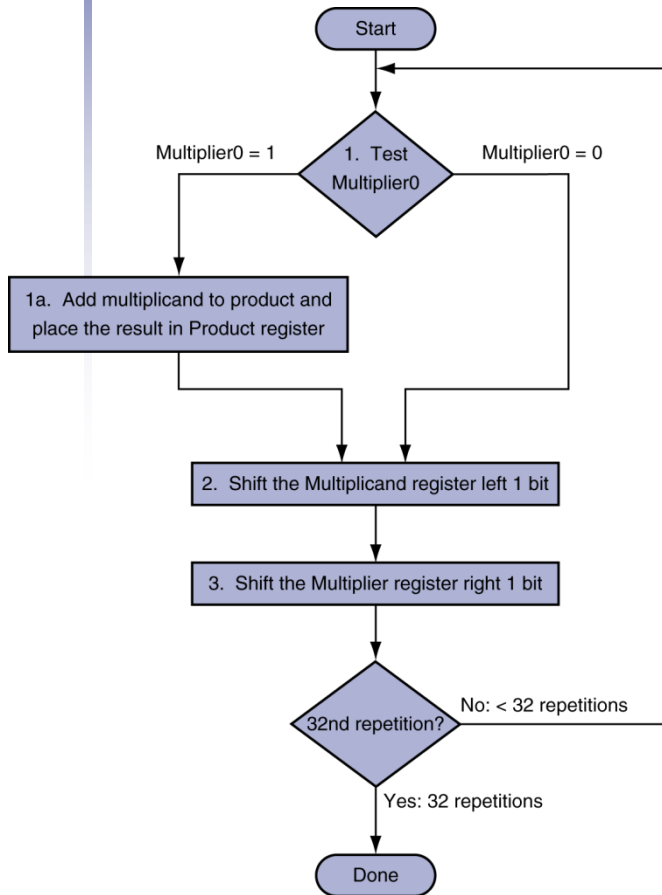
- Start with long-multiplication approach



Length of product is the sum of operand lengths



Multiplication Hardware

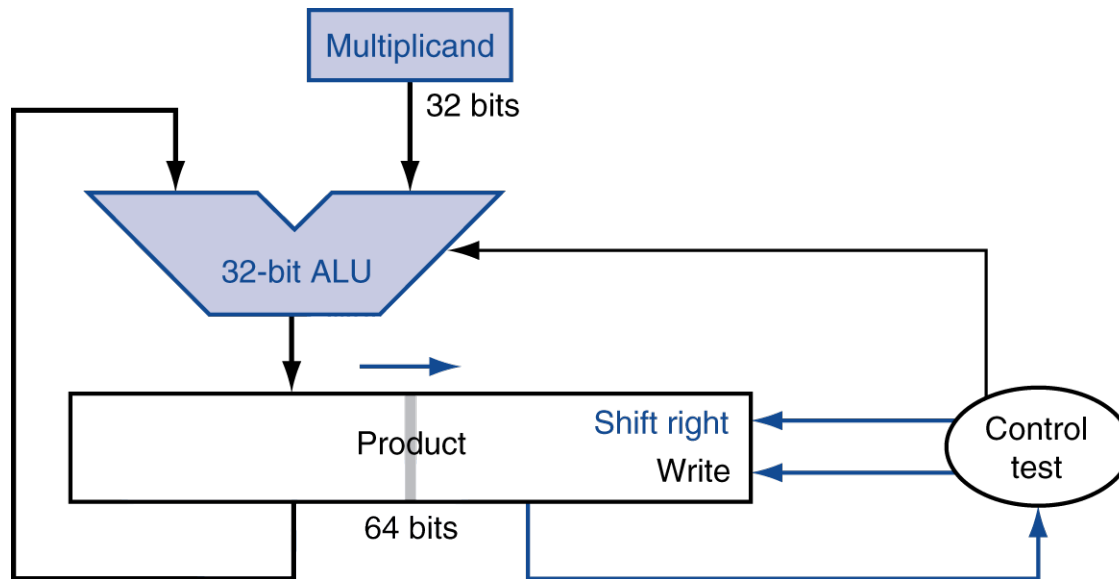


Iteration	Step	Multiplier	Multiplicand	Product
0	Initial values	001 ^①	0000 0010	0000 0000
1	1a: $1 \Rightarrow \text{Prod} = \text{Prod} + \text{Mcand}$	0011	0000 0010	0000 0010
	2: Shift left Multiplicand	0011	0000 0100	0000 0010
	3: Shift right Multiplier	000 ^①	0000 0100	0000 0010
2	1a: $1 \Rightarrow \text{Prod} = \text{Prod} + \text{Mcand}$	0001	0000 0100	0000 0110
	2: Shift left Multiplicand	0001	0000 1000	0000 0110
	3: Shift right Multiplier	000 ^②	0000 1000	0000 0110
3	1: $0 \Rightarrow$ No operation	0000	0000 1000	0000 0110
	2: Shift left Multiplicand	0000	0001 0000	0000 0110
	3: Shift right Multiplier	000 ^②	0001 0000	0000 0110
4	1: $0 \Rightarrow$ No operation	0000	0001 0000	0000 0110
	2: Shift left Multiplicand	0000	0010 0000	0000 0110
	3: Shift right Multiplier	0000	0010 0000	0000 0110

FIGURE 3.6 Multiply example using algorithm in Figure 3.4. The bit examined to determine the next step is circled in color.

Optimized Multiplier

- Perform steps in parallel: add/shift



- One cycle per partial-product addition
 - That's ok, if frequency of multiplications is low

硬體/軟體介面

- ▣ 有些編譯器以數個短的常數配合一連串的移位與加來取代乘法
 - 幾乎所有編譯器都會作以左移來取代乘以 2 的次方的「強度減弱」最佳化

有號乘法

- 最簡單的方法是首先將乘數與被乘數都轉換為正數並記住其原有符號
 - 執行31 個反覆
 - 若原有二符號不同則我們需將乘積改為負值

較快的乘法

- 檢視 32 位元乘數中的每一個位元，在乘法之初即可知道被乘數在每一個位元位置是否應該拿來加
 - 給乘數的每一個位元配置一個 32 位元的加法器即可做出快速的乘法器：
 - 加法器的一個輸入是被乘數與相對乘數位元 AND 的結果，而另一輸入則為其之前加法器的輸出
 - 將這 32 個加法器組織在一棵平行的樹中
 - 祇需等待 $\log_2(32)$ 或五個 32 位元加法的時間

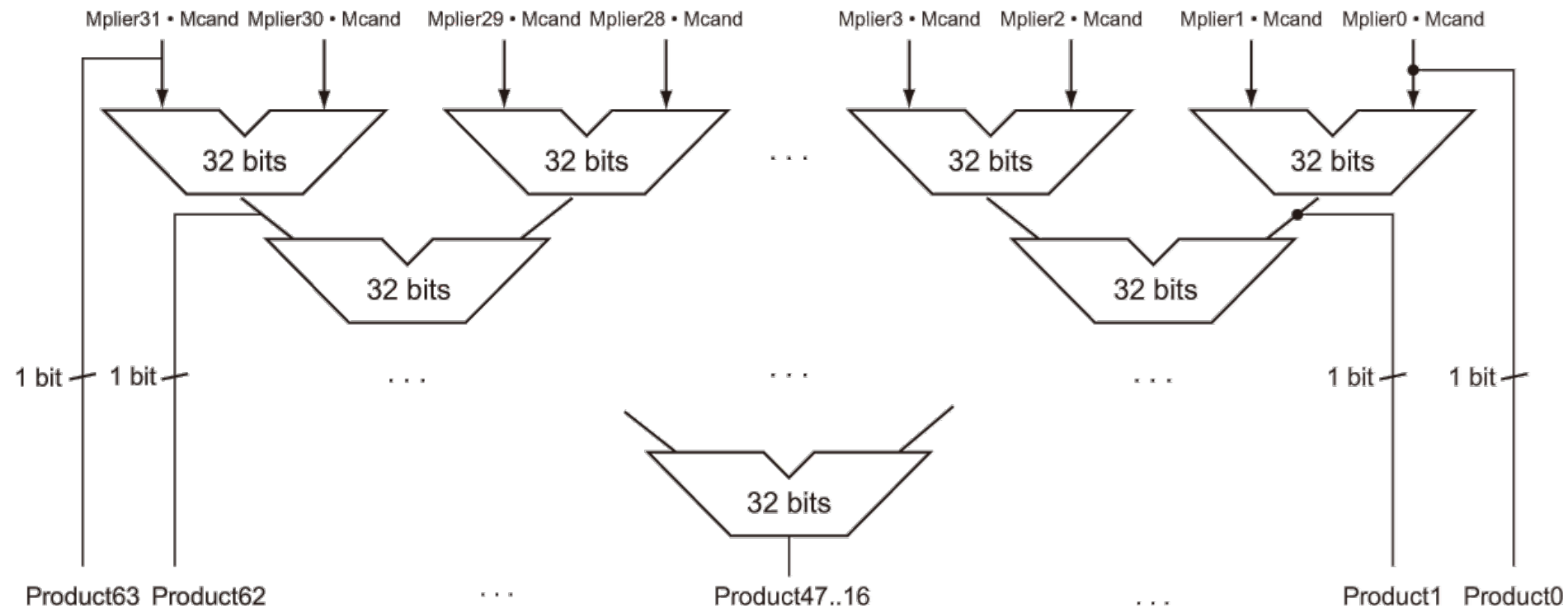


圖3.7 快速乘法硬體

與其使用單一個 32 位元加法器 31 次，這個硬體「將迴圈展開」來使用 31 個加法器並將它們安排成使延遲降至最低

較快的乘法

- 乘法可以做得比五個加法時間還快
 - 可以使用進位保留加法器
 - 管道化

MIPS 中的乘法

- MIPS 另外提供了一組32 位元的暫存器，稱為 H_i 及 L_o ，來放置64 位元的乘積
- MIPS 有兩道不同的指令：乘(mult)及無號乘(multu)

MIPS assembly language				
Category	Instruction	Example	Meaning	Comments
Arithmetic	add	add \$s1,\$s2,\$s3	\$s1 = \$s2 + \$s3	Three operands; overflow detected
	subtract	sub \$s1,\$s2,\$s3	\$s1 = \$s2 - \$s3	Three operands; overflow detected
	add immediate	addi \$s1,\$s2,100	\$s1 = \$s2 + 100	+ constant; overflow detected
	add unsigned	addu \$s1,\$s2,\$s3	\$s1 = \$s2 + \$s3	Three operands; overflow undetected
	subtract unsigned	subu \$s1,\$s2,\$s3	\$s1 = \$s2 - \$s3	Three operands; overflow undetected
	add immediate unsigned	addiu \$s1,\$s2,100	\$s1 = \$s2 + 100	+ constant; overflow undetected
	move from coprocessor register	mfc0 \$s1,\$epc	\$s1 = \$epc	Copy Exception PC + special regs
	multiply	mult \$s2,\$s3	Hi, Lo = \$s2 × \$s3	64-bit signed product in Hi, Lo
	multiply unsigned	multu \$s2,\$s3	Hi, Lo = \$s2 × \$s3	64-bit unsigned product in Hi, Lo
	divide	div \$s2,\$s3	Lo = \$s2 / \$s3, Hi = \$s2 mod \$s3	Lo = quotient, Hi = remainder
	divide unsigned	divu \$s2,\$s3	Lo = \$s2 / \$s3, Hi = \$s2 mod \$s3	Unsigned quotient and remainder
	move from Hi	mghi \$s1	\$s1 = Hi	Used to get copy of Hi
	move from Lo	mlo \$s1	\$s1 = Lo	Used to get copy of Lo
Data transfer	load word	lw \$s1,20(\$s2)	\$s1 = Memory[\$s2 + 20]	Word from memory to register
	store word	sw \$s1,20(\$s2)	Memory[\$s2 + 20] = \$s1	Word from register to memory
	load half unsigned	lhu \$s1,20(\$s2)	\$s1 = Memory[\$s2 + 20]	Halfword memory to register
	store half	sh \$s1,20(\$s2)	Memory[\$s2 + 20] = \$s1	Halfword register to memory
	load byte unsigned	lbu \$s1,20(\$s2)	\$s1 = Memory[\$s2 + 20]	Byte from memory to register
	store byte	sb \$s1,20(\$s2)	Memory[\$s2 + 20] = \$s1	Byte from register to memory
	load linked word	ll \$s1,20(\$s2)	\$s1 = Memory[\$s2 + 20]	Load word as 1st half of atomic swap
	store conditional word	sc \$s1,20(\$s2)	Memory[\$s2+20]=\$s1; \$s1=0 or 1	Store word as 2nd half atomic swap
Logical	load upper immediate	lui \$s1,100	\$s1 = 100 × 2 ¹⁶	Loads constant in upper 16 bits
	AND	AND \$s1,\$s2,\$s3	\$s1 = \$s2 & \$s3	Three reg. operands; bit-by-bit AND
	OR	OR \$s1,\$s2,\$s3	\$s1 = \$s2 \$s3	Three reg. operands; bit-by-bit OR
	NOR	NOR \$s1,\$s2,\$s3	\$s1 = ~ (\$s2 \$s3)	Three reg. operands; bit-by-bit NOR
	AND immediate	ANDI \$s1,\$s2,100	\$s1 = \$s2 & 100	Bit-by-bit AND with constant
	OR immediate	ORI \$s1,\$s2,100	\$s1 = \$s2 100	Bit-by-bit OR with constant
	shift left logical	sll \$s1,\$s2,10	\$s1 = \$s2 << 10	Shift left by constant
Conditional branch	shift right logical	srl \$s1,\$s2,10	\$s1 = \$s2 >> 10	Shift right by constant
	branch on equal	beq \$s1,\$s2,25	if (\$s1 == \$s2) go to PC + 4 + 100	Equal test; PC-relative branch
	branch on not equal	bne \$s1,\$s2,25	if (\$s1 != \$s2) go to PC + 4 + 100	Not equal test; PC-relative
	set on less than	slt \$s1,\$s2,\$s3	if (\$s2 < \$s3) \$s1 = 1; else \$s1 = 0	Compare less than; two's complement
	set less than immediate	slti \$s1,\$s2,100	if (\$s2 < 100) \$s1 = 1; else \$s1 = 0	Compare < constant; two's complement
	set less than unsigned	sltu \$s1,\$s2,\$s3	if (\$s2 < \$s3) \$s1 = 1; else \$s1 = 0	Compare less than; natural numbers
	set less than immediate unsigned	sltiu \$s1,\$s2,100	if (\$s2 < 100) \$s1 = 1; else \$s1 = 0	Compare < constant; natural numbers
Unconditional jump	jump	j 2500	go to 10000	Jump to target address
	jump register	jr \$ra	go to \$ra	For switch, procedure return
	jump and link	jal 2500	\$ra = PC + 4; go to 10000	For procedure call

FIGURE 3.12 MIPS core architecture. The memory and registers of the MIPS architecture are not included for space reasons, but this section added the Hi and Lo registers to support multiply and divide. MIPS machine language is listed in the MIPS Reference Data Card at the front of this book.

MIPS Multiplication

- Two 32-bit registers for product
 - HI: most-significant 32 bits
 - LO: least-significant 32-bits
- Instructions
 - `mult rs, rt` / `multu rs, rt`
 - 64-bit product in HI/LO
 - `mfhi rd` / `mflo rd`
 - Move from HI/LO to rd
 - Can test HI value to see if product overflows 32 bits
 - `mul rd, rs, rt`
 - Least-significant 32 bits of product → rd

Unsigned Multiplier (Ver. 2)

- ◆ 32-bit Multiplicand register, 32-bit ALU, 64-bit Product register (HI & LO in MIPS), (0-bit Multiplier register)

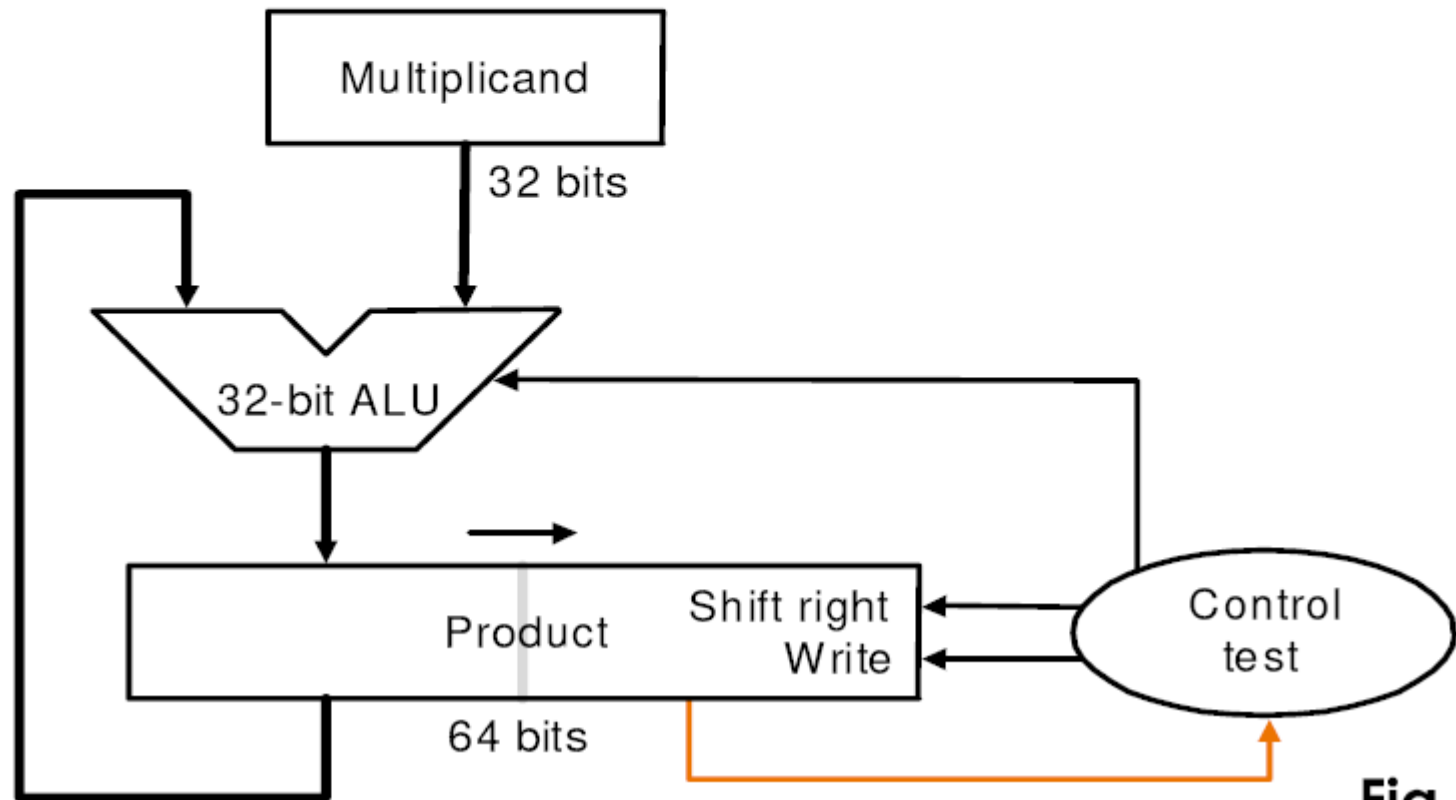


Fig. 3.7

Divide: Paper & Pencil

		1001 _{ten}	Quotient
Divisor	1000 _{ten}	$\overline{)1001010_{ten}}$	Dividend
		-1000	
		10	
		101	
		1010	
		-1000	
		10 _{ten}	Remainder

- ♦ See how big a number can be subtracted, creating quotient bit on each step
Binary $\Rightarrow$ 1 * divisor or 0 * divisor
- ♦ Two versions of divide, successive refinement
- ♦ *Both dividend and divisor are 32-bit positive integers*

Divide Hardware (Version 1)

- ◆ 64-bit *Divisor* register (initialized with 32-bit divisor in left half), 64-bit ALU, 64-bit *Remainder* register (initialized with 64-bit dividend), 32-bit *Quotient* register

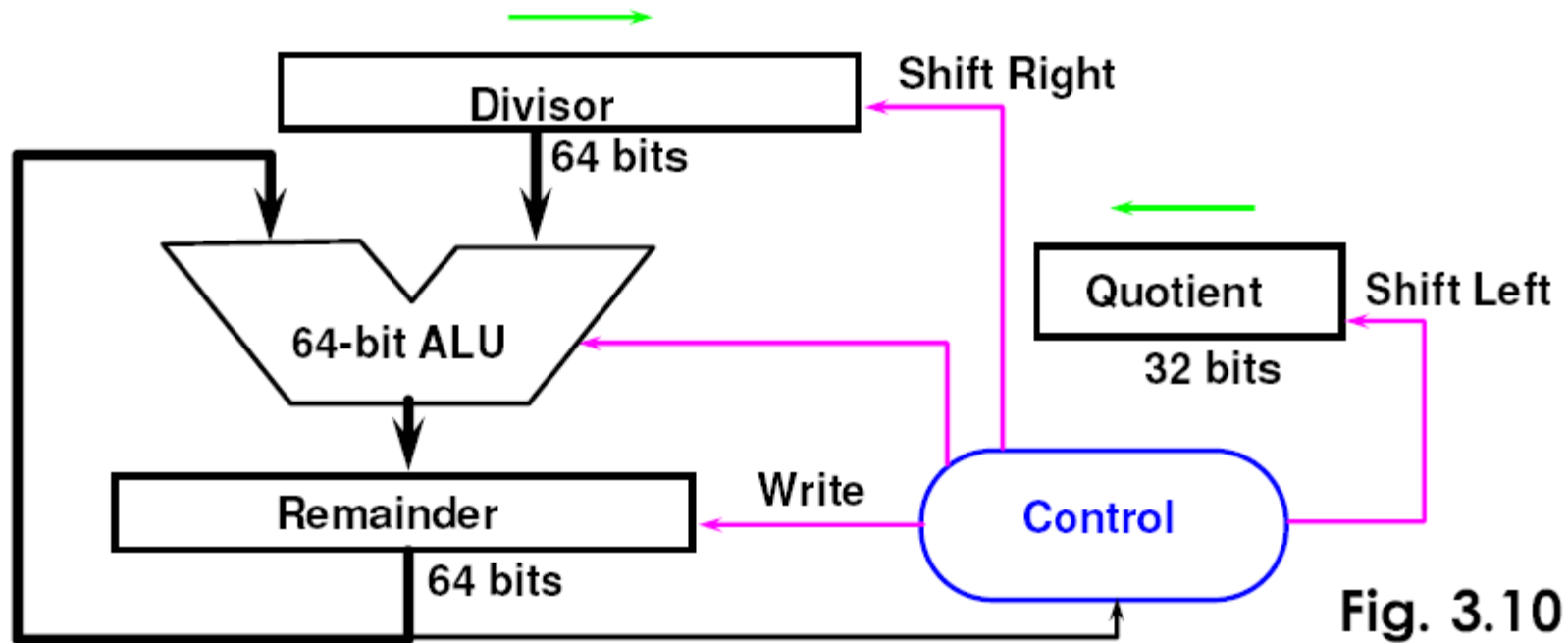


Fig. 3.10

除法演算法及硬體

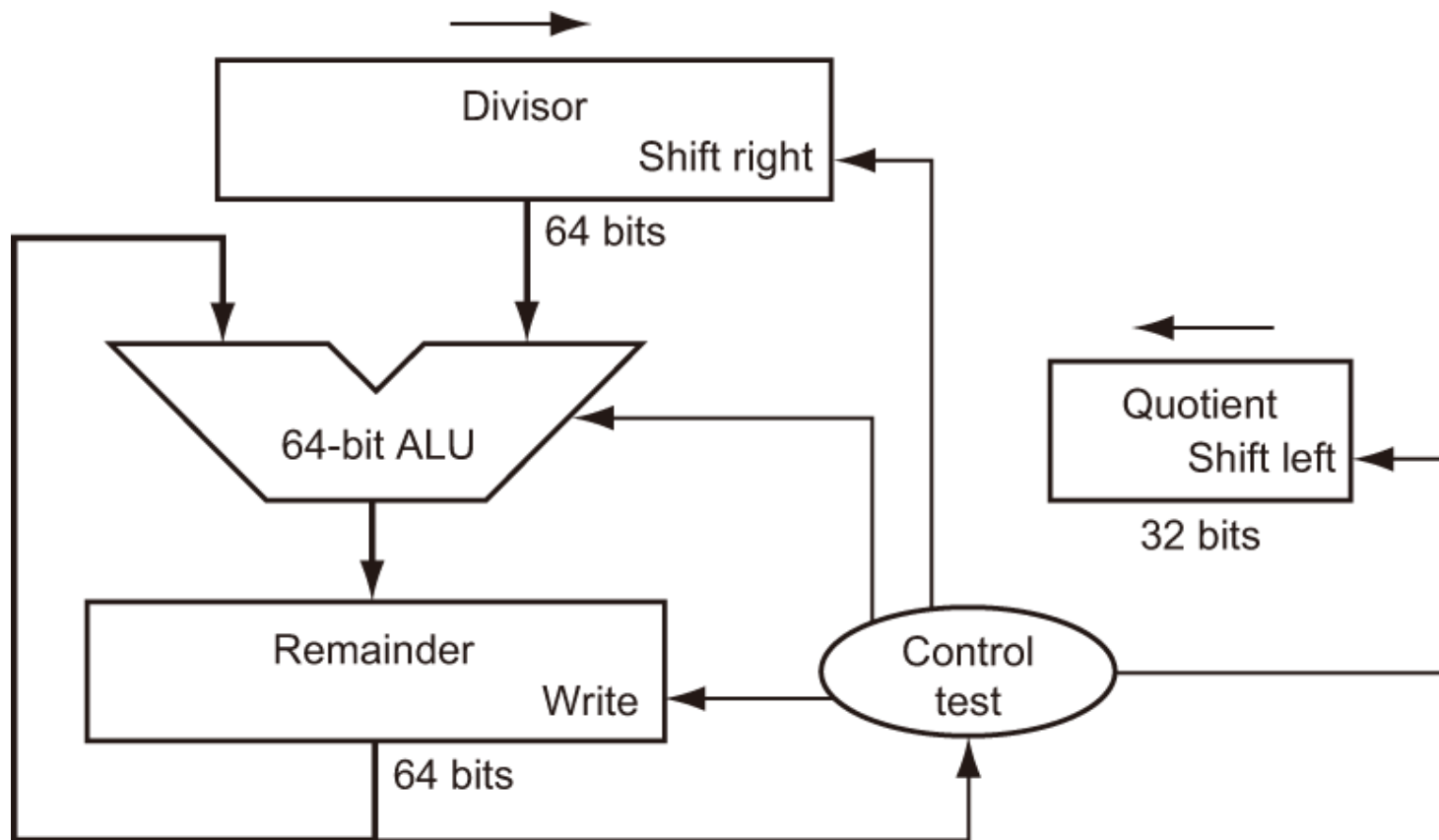
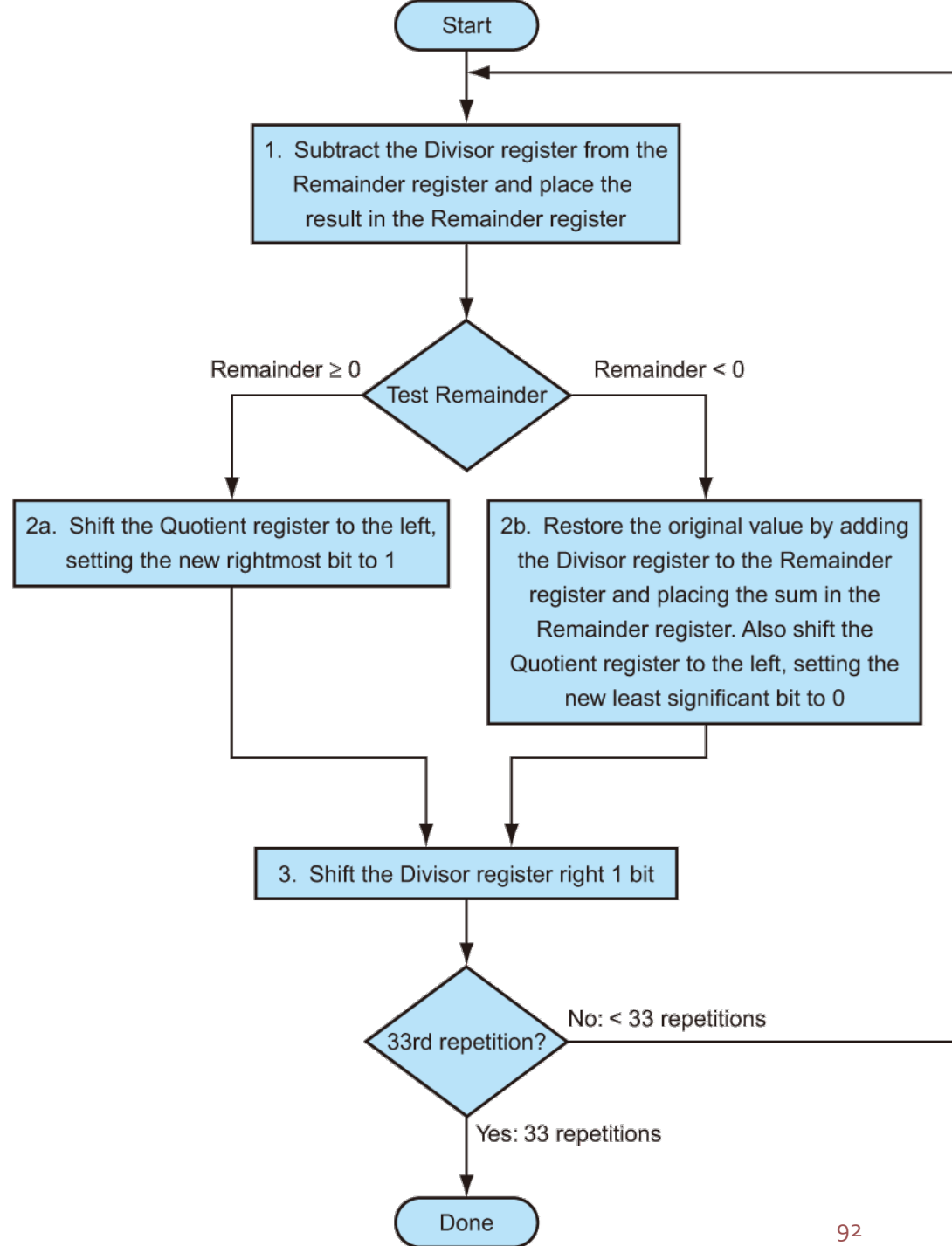


圖3.8 除法硬體的
第一個版本

除數暫存器、
算術邏輯單元
以及餘數暫存
器都是 64 位元
寬，只有商數
暫存器是 32 位
元寬。32 位元
的除數先被放
置在除數暫存
器的左半部，
並在每次反覆
中右移 1 位元。
餘數被初始
化為被除數的
值。控制單元
決定何時要將
除數與商數暫
存器移位以及
何時要將新值
寫入餘數暫存
器

圖3.9 使用圖3.8 中硬體的除法演算法

如果餘數暫存器的值為正，表示被除數的確可以讓除數來除，所以步驟 2a 在商數中產生一個 1。在步驟 1 之後若餘數暫存器的值為負，表示被除數不夠讓除數來除，所以步驟 2b 在商數中產生一個 0，並將除數加回餘數，因此抵消了步驟 1 的減法。步驟 3 最後的移位將除數恰當地與被除數作下一次反覆中的對齊。這些步驟會重複 33 次



Divide Algorithm (Version 1)

Ver 1. Restoring Division (先減再說，負再加回去)

Quot.	Divisor	Rem.
0000	<u>00100000</u>	00000111
		11100111
		00000111
0000	<u>00010000</u>	00000111
		11110111
		00000111
0000	<u>00001000</u>	00000111
		11111111
		00000111
0000	<u>00000100</u>	00000111
		00000011
0001		00000011
0001	<u>00000010</u>	00000011
		00000001
0011		00000001
0011	<u>00000001</u>	00000001

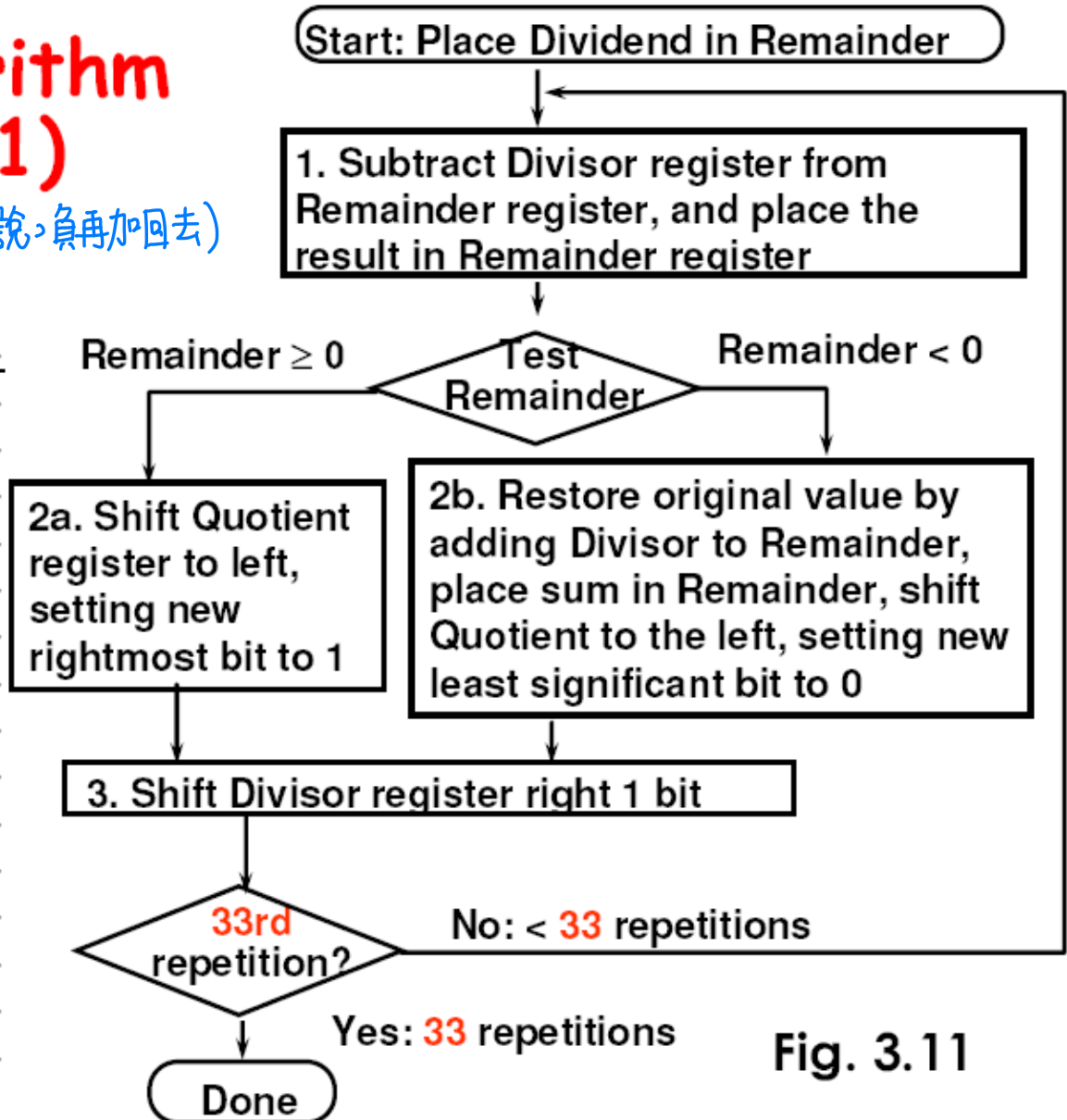


Fig. 3.11

Observations: Divide Version 1

- ◆ Half of the bits in divisor register always 0
 - => 1/2 of 64-bit adder is wasted
 - => 1/2 of divisor is wasted
- ◆ Instead of shifting divisor to right, shift remainder to left?
- ◆ 1st step cannot produce a 1 in quotient bit (otherwise quotient is too big for the register)
 - => switch order to shift first and then subtract
 - => save 1 iteration
- ◆ Eliminate Quotient register by combining with Remainder register as shifted left

Divide Hardware (Version 2)

- ◆ 32-bit Divisor register, 32-bit ALU, 64-bit Remainder register, (0-bit Quotient register)

② Non-Restoring Division (先比較, 再減)

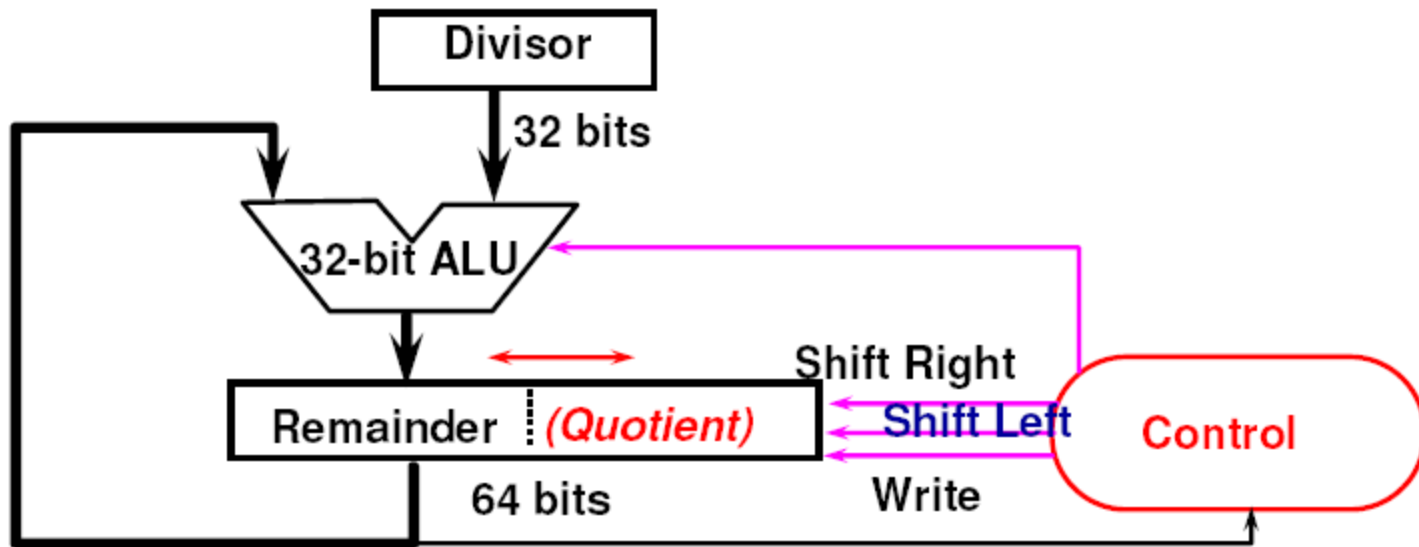
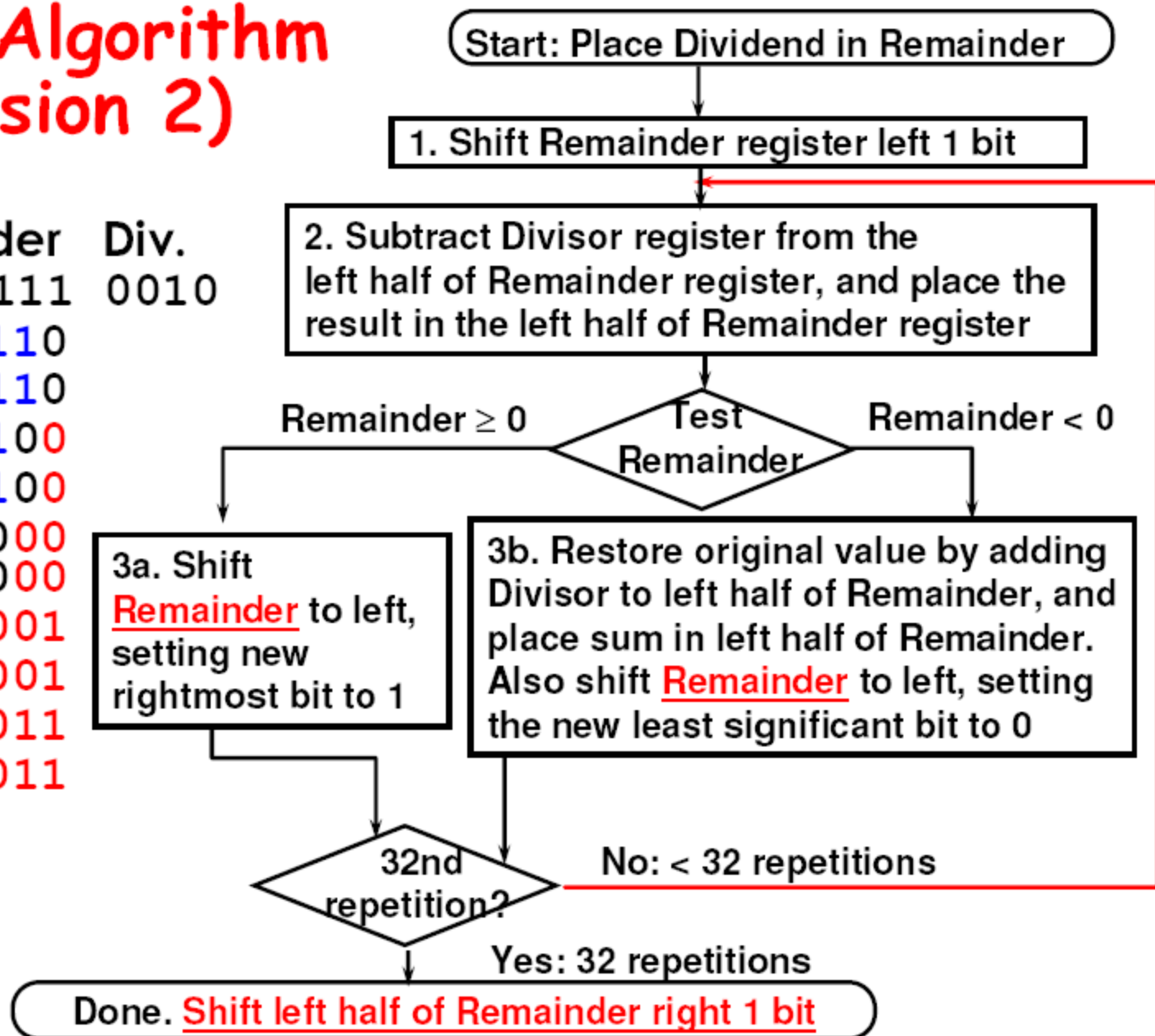


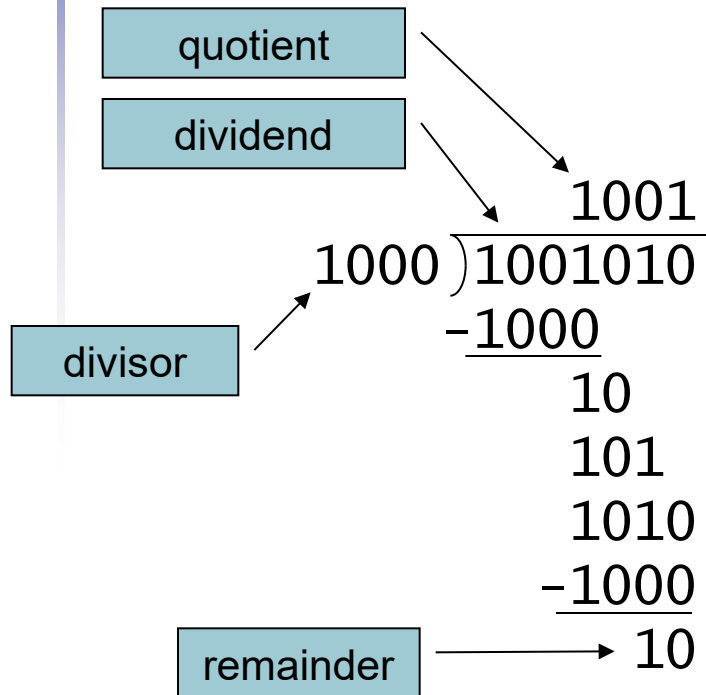
Fig. 3.13

Divide Algorithm (Version 2)

Step	Remainder	Div.
0	0000 0111	0010
1.1	0000 1110	
1.2	1110 1110	
1.3b	0001 1100	
2.2	1111 1100	
2.3b	0011 1000	
3.2	0001 1000	
3.3a	0011 0001	
4.2	0001 0001	
4.3a	0010 0011	
	0001 0011	



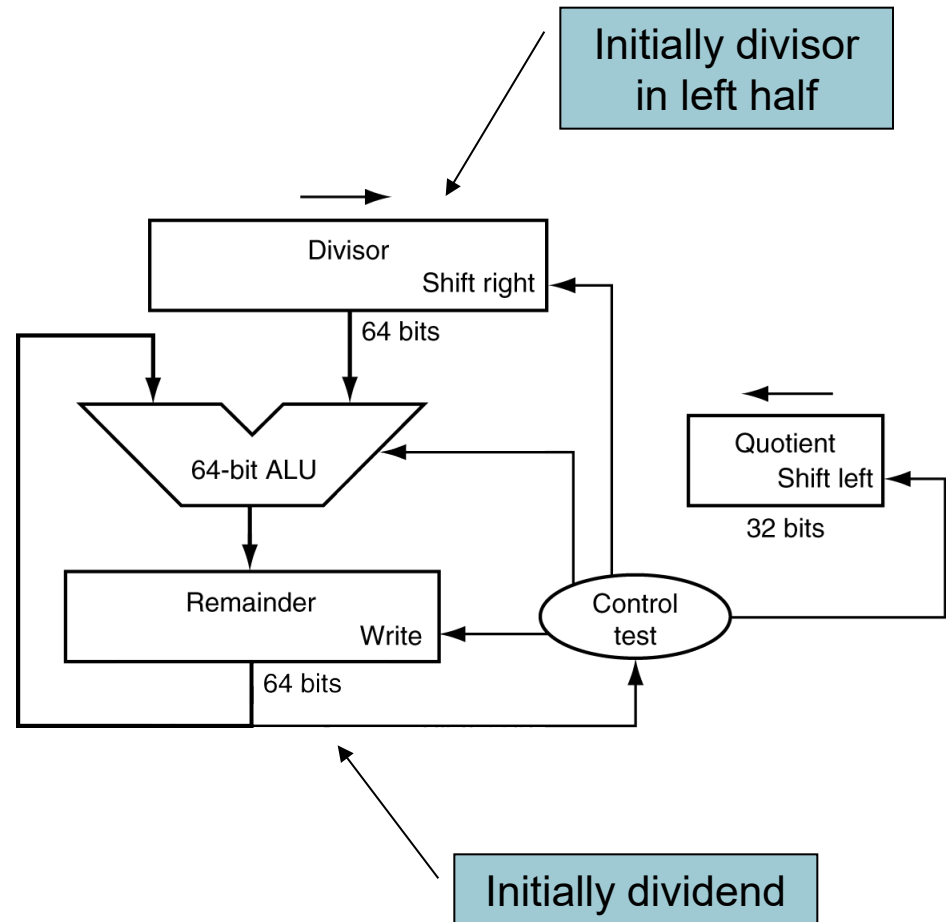
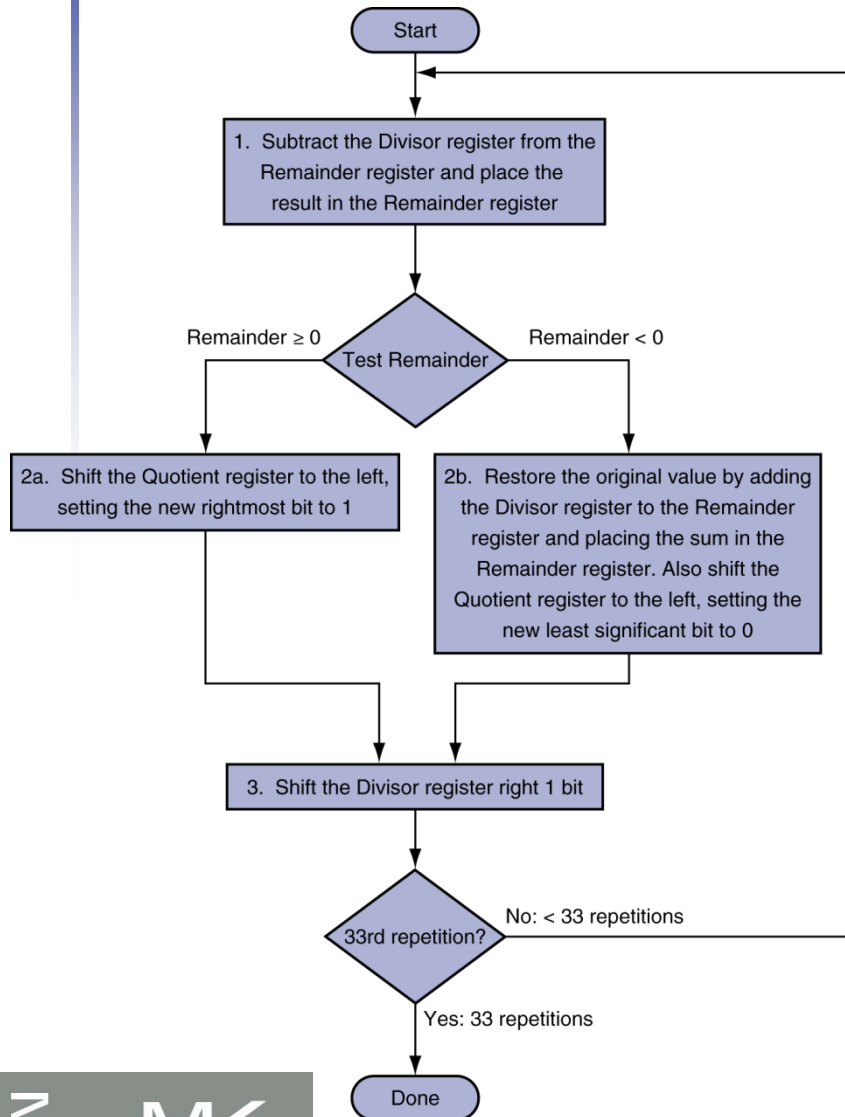
Division



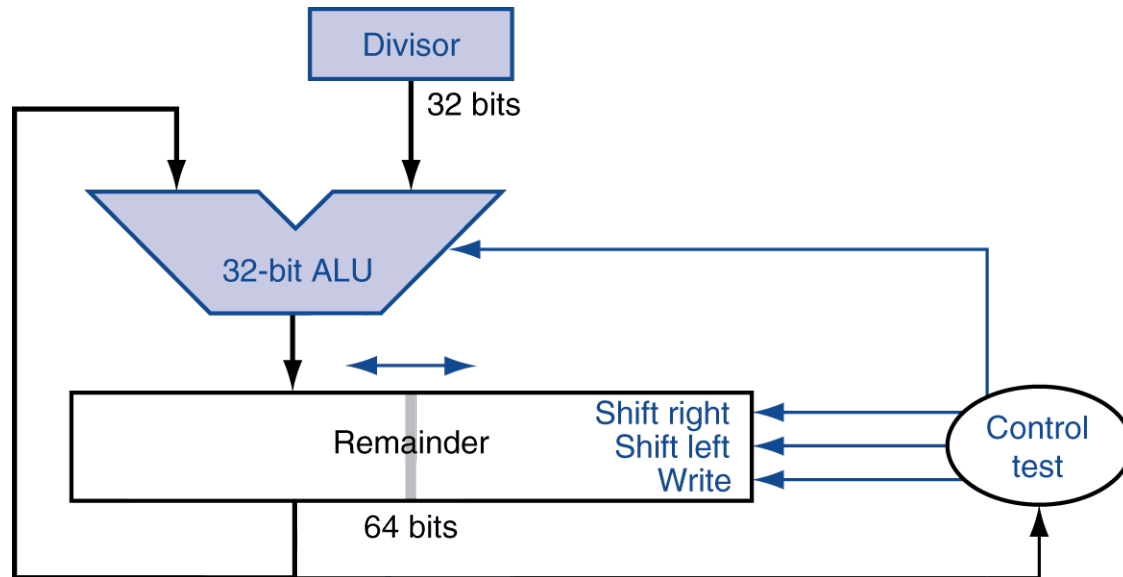
n-bit operands yield *n*-bit quotient and remainder

- Check for 0 divisor
- Long division approach
 - If divisor $\leq$ dividend bits
 - 1 bit in quotient, subtract
 - Otherwise
 - 0 bit in quotient, bring down next dividend bit
- Restoring division
 - Do the subtract, and if remainder goes < 0 , add divisor back
- Signed division
 - Divide using absolute values
 - Adjust sign of quotient and remainder as required

Division Hardware



Optimized Divider



- One cycle per partial-remainder subtraction
- Looks a lot like a multiplier!
 - Same hardware can be used for both

3.4 除法

- ▣ 被除數=商×除數+餘數，其中餘數需小於除數
- ▣ 假設被除數與除數均為正數，因此商與餘數均為非負。除法的運算元以及兩個結果都是32 位元的值，並暫時忽略其符號

3.4 除法

- ▣ 該演算法及硬體可改良成更快及更省
 - 將運算元及商的移位與減法同時進行
 - 暫存器及加法器都有沒用到的部分，因此其寬度均予以減半

3.4 除法

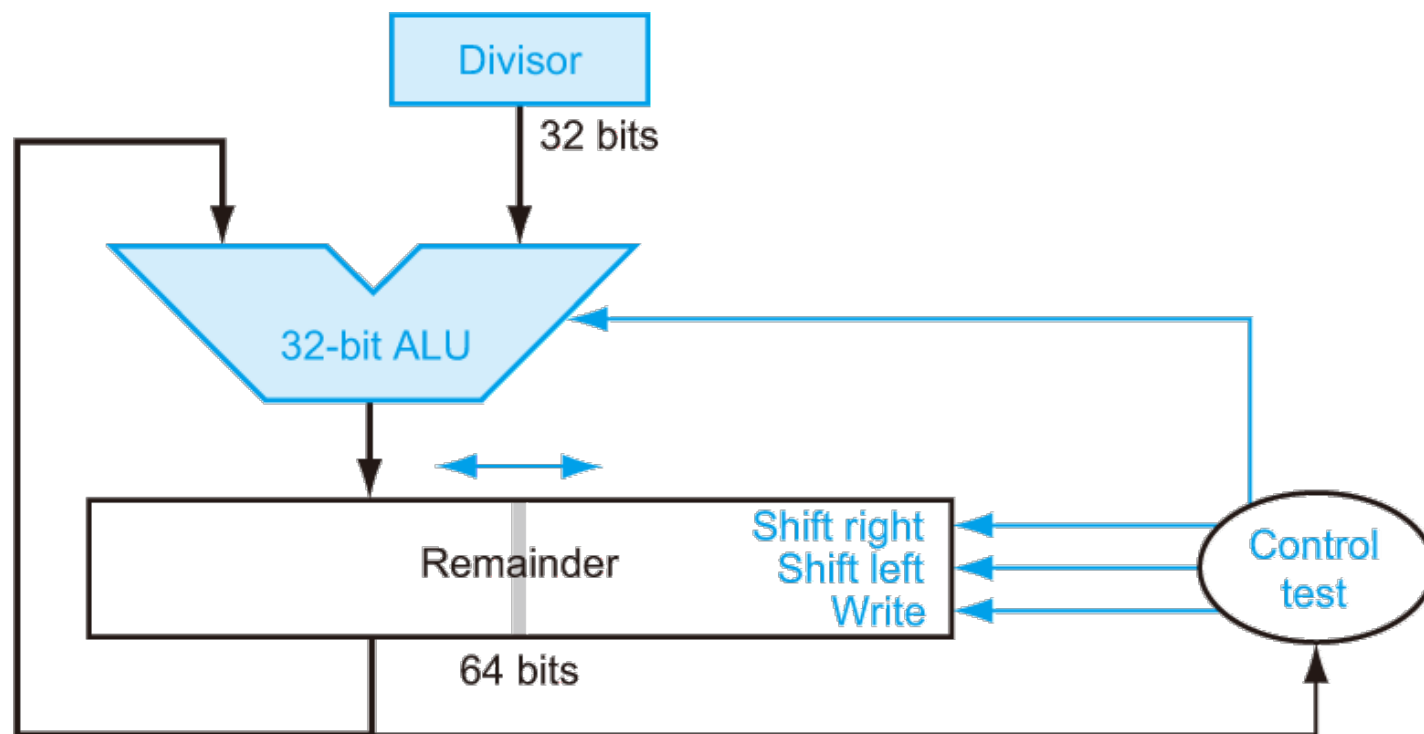


圖3.11 除法硬體改良版本

除數暫存器、算術邏輯單元以及商數暫存器都是 32 位元寬，只有餘數暫存器是 64 位元寬，只有餘數暫存器是 64 位元寬，跟圖 3.8 比較，算術邏輯單元與除數暫存器的寬度減半而餘數會被左移。這個版本也將商數暫存器與餘數暫存器的右半部合併起來。(如同在圖 3.5 中一般，餘數暫存器其實應該是 65 個位元以確保加法器的進位輸出不會遺失。)

有號除法

- 最簡單的方法就是記住除數與被除數的符號然後若符號不一致則取商的負值
- 有號除法有一個複雜處即是我們也需要設定餘數的符號
 - 正確的有號除法演算法在運算元符號相異時取商的負值並令非零餘數的符號與被除數相同

較快的除法

- 我們曾使用許多加法器來加速乘法，但我們無法將相同手法用於除法
 - 原因在於我們在進行演算法的下一個步驟前需要先知道差值的符號
- 也有在一步驟中產生多於一個商位元的技法
 - *SRT* 除法技術

MIPS 中的除法

- MIPS 的乘法與除法運算都使用 32 位元的 Hi 暫存器與 32 位元的 Lo 暫存器
 - 在除法指令執行完後，Hi 暫存器內含有餘數，而 Lo 暫存器內則含有商數
- MIPS 提供兩種指令：除法(div)與無號除法(divu)

MIPS Division

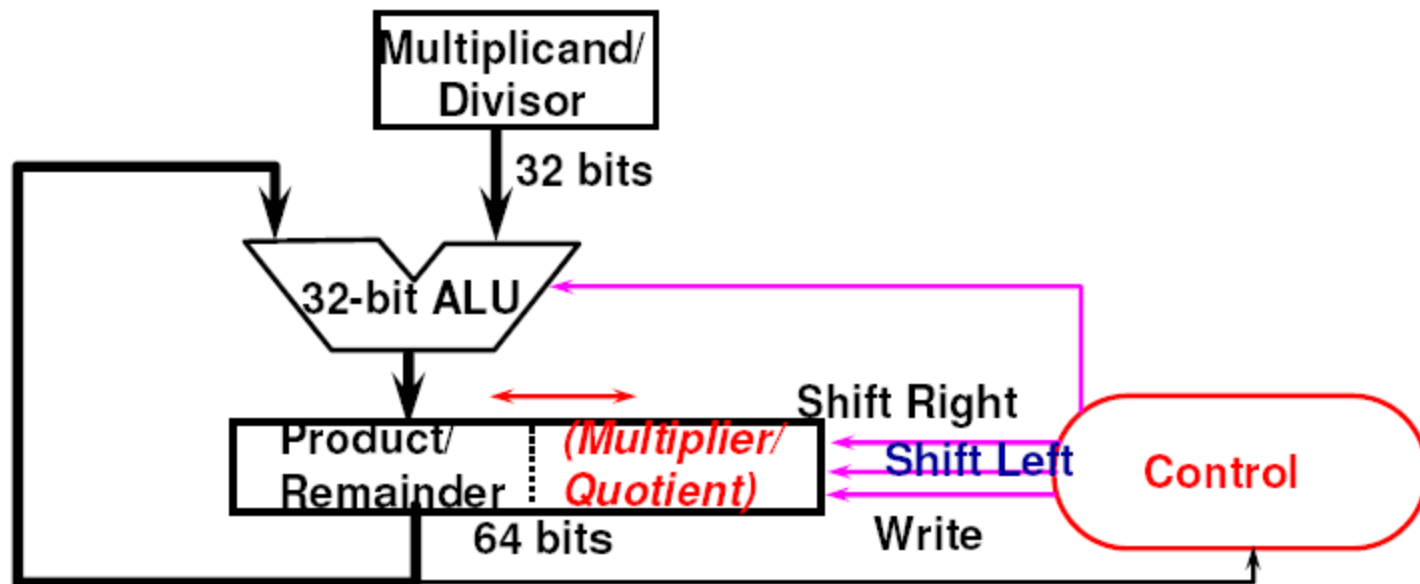
- Use HI/LO registers for result
 - HI: 32-bit remainder
 - LO: 32-bit quotient
- Instructions
 - `div rs, rt` / `divu rs, rt`
 - No overflow or divide-by-0 checking
 - Software must perform checks if required
 - Use `mfhi`, `mflo` to access result

Observations: Multiply and Divide

- ◆ Same hardware as multiply: just need ALU to add or subtract, and 64-bit register to shift left or shift right
- ◆ Hi and Lo registers in MIPS combine to act as 64-bit register for multiply and divide

Multiply/Divide Hardware

- ◆ 32-bit Multiplicand/Divisor register, 32-bit ALU, 64-bit Product/Remainder register, (0-bit Multiplier/Quotient register)



MIPS Multiply/Divide Exercises

- 3.12, 3.13
 - $62 * 12$
- 3.18, 3.19
 - $74/21$

硬體/軟體介面

- ▣ MIPS 的除法指令不處理溢位的偵測
- ▣ MIPS 軟體不但要偵測溢位，還必須檢查除數以便發現除以 0 的狀況

- 非回復型除法演算法 (nonrestoring division algorithm)每個步驟只耗費 1 個時脈週期
 - 將在習題中有更多的探討
- 本節所介紹的演算法則稱為回復型除法(restoring division)
- 第三種稱為非施行型除法(nonperforming division)演算法
 - 其平均可減少三分之一的算術運算

3.5 浮點

- ▣ 科學記號法(scientific notation)在小數點(decimal point)的左方具有一個位數
 - 表示的數字如果不是以 0 開頭，則稱為正規化(normalized)的數
- ▣ 支援這種數字的計算機算術稱為浮點
 - 不同於表示整數時小數點的位置固定不變，它可以表示小數點位置不固定的數
- ▣ 浮點數的小數點左邊會有一個非零的位元

$$1.xxxxxxxxxx_2 \times 2^{yyyy}$$

3.5 浮點

- 好處：

- 由於數字一定會以該形式表示而可以簡化浮點數的算術演算法
- 因為不需要的 0 都被小數點右方的實際數字所取代而增加了字組中存放的數字的準確度

浮點數的表示法

- 固定的字組大小代表浮點表示法的設計者必須在分數(fraction)大小與指數(exponent)大小之間尋求折衷
 - 增加分數的大小可提高它的精確度
 - 增加指數的大小則擴增了可以表示的數字的範圍
- MIPS 裡的浮點數表示法
 - 符號為浮點數的正負號(1 表示為負)
 - 指數為 8 位元的指數欄位的值(包含指數的正負號)
 - 分數則為 23 位元的數字

浮點數的表示法

$$(-1)^S \times F \times 2^E$$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
符號	指 數								分 數																						

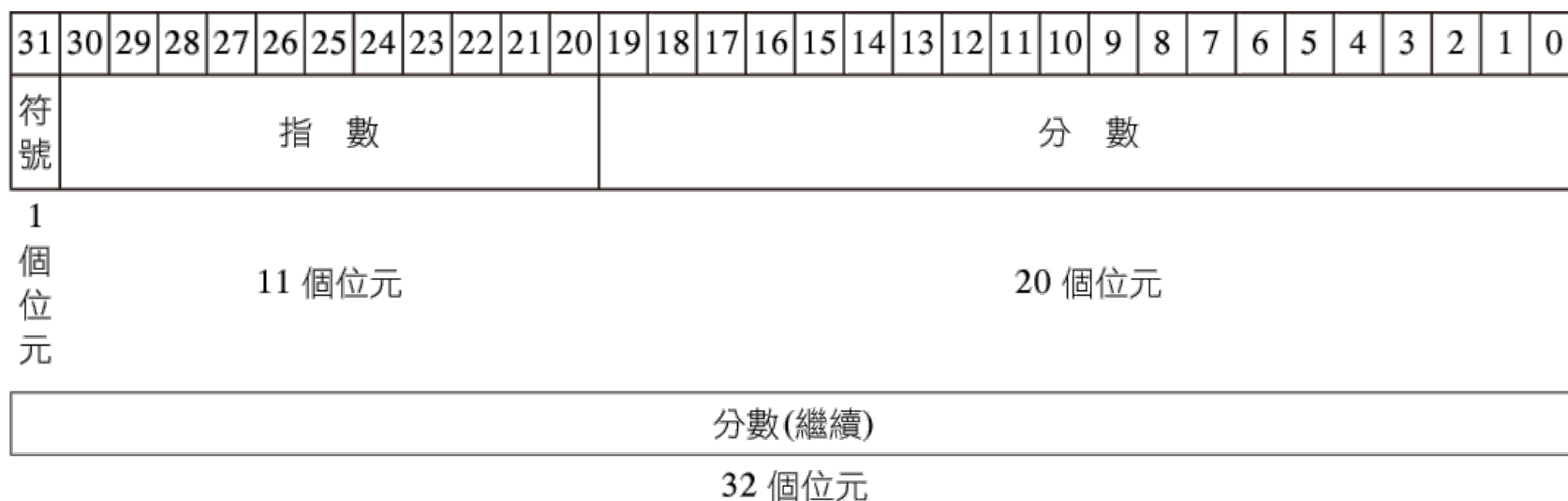
1
個
位
元

8 個位元

23 個位元

浮點數的表示法

- 溢位(overflow)指的是指數太大而超出指數欄位所能表示的範圍
- 短值(underflow)：當負的指數大到指數欄位無法容納時
- 雙精確度(double precision)與單精確度(single precision)



浮點數的表示法

- 有效數字(significand) 為分數fraction 的別名；
另一別名為尾數mantissa
- 是IEEE 754 浮點數標準的一部分
 - IEEE 754 將正規化的二進數中第一個1 位元設為隱喻的(implicit)
 - 為了明確起見，我們使用有效數字(significand) 這個詞來表示24 或是53 位元的1 加上分數部分的數，以及使用小數(fraction)這個詞來表示23 或52 位元的數
 - $00...00_2$ 代表0

Floating Point Standard

- Defined by IEEE Std 754-1985
- Developed in response to divergence of representations
 - Portability issues for scientific code
- Now almost universally adopted
- Two representations
 - Single precision (32-bit), FP32
 - Double precision (64-bit), FP64

IEEE Floating-Point Format

single: 8 bits
double: 11 bits

single: 23 bits
double: 52 bits

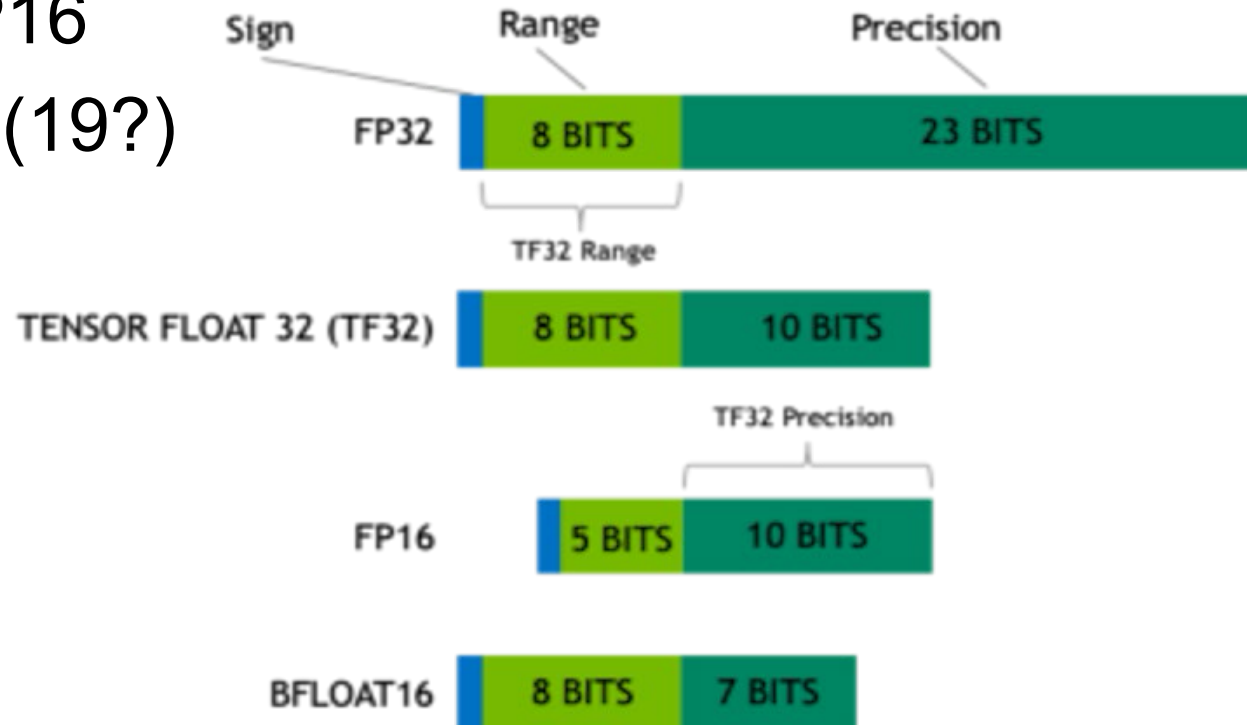
S	Exponent	Fraction
---	----------	----------

$$x = (-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$$

- S: sign bit (0 $\Rightarrow$ non-negative, 1 $\Rightarrow$ negative)
- Normalize significand: $1.0 \leq |\text{significand}| < 2.0$
 - Always has a leading pre-binary-point 1 bit, so no need to represent it explicitly (hidden bit)
 - Significand is Fraction with the “1.” restored
- Exponent: excess representation: actual exponent + Bias
 - Ensures exponent is unsigned
 - Single: Bias = 127; Double: Bias = 1023

More Recent Floating Point Formats

- More representations
 - For the arising DL/ML applications of AI
 - Half precision (16-bit), FP16
 - Google's BFP16
 - nVidia's TF32(19?)



Single-Precision Range (FP32,BFP16)

- Exponents 00000000 and 11111111 reserved
- Smallest value
 - Exponent: 00000001
 $\Rightarrow$ actual exponent = $1 - 127 = -126$
 - Fraction: 000...00 $\Rightarrow$ significand = 1.0
 - $\pm 1.0 \times 2^{-126} \approx \pm 1.2 \times 10^{-38}$
- Largest value
 - exponent: 11111110
 $\Rightarrow$ actual exponent = $254 - 127 = +127$
 - Fraction: 111...11 $\Rightarrow$ significand ≈ 2.0
 - $\pm 2.0 \times 2^{+127} \approx \pm 3.4 \times 10^{+38}$

Double-Precision Range

- Exponents 0000...00 and 1111...11 reserved
- Smallest value
 - Exponent: 000000000001
 $\Rightarrow$ actual exponent = $1 - 1023 = -1022$
 - Fraction: 000...00 $\Rightarrow$ significand = 1.0
 - $\pm 1.0 \times 2^{-1022} \approx \pm 2.2 \times 10^{-308}$
- Largest value
 - Exponent: 111111111110
 $\Rightarrow$ actual exponent = $2046 - 1023 = +1023$
 - Fraction: 111...11 $\Rightarrow$ significand ≈ 2.0
 - $\pm 2.0 \times 2^{+1023} \approx \pm 1.8 \times 10^{+308}$

Floating-Point Precision

- Relative precision
 - all fraction bits are significant
 - Single (FP32): approx 2^{-23}
 - Equivalent to $23 \times \log_{10} 2 \approx 23 \times 0.3 \approx 6(7)$ decimal digits of precision
 - Double (FP64): approx 2^{-52}
 - Equivalent to $52 \times \log_{10} 2 \approx 52 \times 0.3 \approx 16$ decimal digits of precision
 - BFP16: approx 2^{-7}
 - Equivalent to $7 \times \log_{10} 2 \approx 7 \times 0.3 \approx 2$ decimal digits of precision

Floating-Point Example

- Represent -0.75
 - $-0.75 = (-1)^1 \times 1.1_2 \times 2^{-1}$
 - $S = 1$
 - Fraction = $1000\dots00_2$
 - Exponent = $-1 + \text{Bias}$
 - Single: $-1 + 127 = 126 = 01111110_2$
 - Double: $-1 + 1023 = 1022 = 011111111110_2$
- Single: $1011111101000\dots00$
- Double: $10111111111101000\dots00$

Floating-Point Example

- Represent 46.25
 - $46.25 = (-1)^0 \times \text{????}_2 \times 2^{???}$
 - $S = 0$
 - Fraction = $???..._2$
 - Exponent = $-1 + \text{Bias}$
 - Single:
 - Double:
- Single:
- Double:
- BFP:

Floating-Point Example

- Represent 3.1416
 - $3.1416 = (-1)^0 \times \text{????}_2 \times 2^{???}$
 - $S = 0$
 - Fraction = $???..._2$
 - Exponent = $-1 + \text{Bias}$
 - Single:
 - Double:
- Single:
- Double:
- BFP:

Floating-Point Example

- Represent 0.3
 - $0.3 = (-1)^0 \times \text{????}_2 \times 2^{???}$
 - $S = 0$
 - Fraction = $???..._2$
 - Exponent = $-1 + \text{Bias}$
 - Single:
 - Double:
- Single:
- Double:
- BFP:

Floating-Point Example

- What number is represented by the single-precision float

11000000101000...00

- $S = 1$
 - Fraction = $01000...00_2$
 - Exponent = $10000001_2 = 129$
- $x = (-1)^1 \times (1 + 01_2) \times 2^{(129 - 127)}$
 $= (-1) \times 1.25 \times 2^2$
 $= -5.0$

Floating-Point Example

- What number is represented by the single-precision float

01000011101101...00

- $S = 0$

- Fraction = $???_2$

- Exponent = $????_2 =$

- $$\begin{aligned} x &= (-1)^0 \times (1 + ?????_2) \times 2^{(??? - 127)} \\ &= (+1) \times 1.???? \times 2^2 \\ &= ????? \end{aligned}$$

浮點數的表示法

■ 其餘數字

$$(-1)^S \times (1 + \text{分數}) \times 2^E$$

■ 圖3.13 顯示IEEE 754 浮點數的編碼

單精確度		雙精確度		表示的數字
指數	分數	指數	分數	
0	0	0	0	0
0	非零	0	非零	±非標準數
1-254	任何數	1-2046	任何數	±浮點數
255	0	2047	0	±無限
255	非零	2047	非零	NaN(不是個數字)

圖3.13 浮點數的IEEE 754 編碼。

獨立的符號位元決定了正負號。非正規化的數字在第275 頁的仔細深思中說明。這些資訊也可見於本書開頭處的MIPS 參考數據卡第四個直排中

浮點數的表示法

- 將最負的指數表示為 $00\dots00_2$ ，而將最正的指數表示為 $11\dots11_2$
 - 偏移表示法(biased notation)
 - IEEE 754 在單精確度中使用127 做為偏移值

- IEEE754-2008 包含幾乎所有的IEEE754-1985 內容，並加入一個16 位元的格式(「半精確度，half precision」)和一個128 位元的格式(「四倍精確度，quadruple precision」)
- 修正的標準也加入了十進浮點算術

浮點數加法

圖3.14 浮點數加法

正常的路徑是執行步驟三與步驟四各一次，但是如果進位處理造成總和非正規化，我們就必須重複步驟三

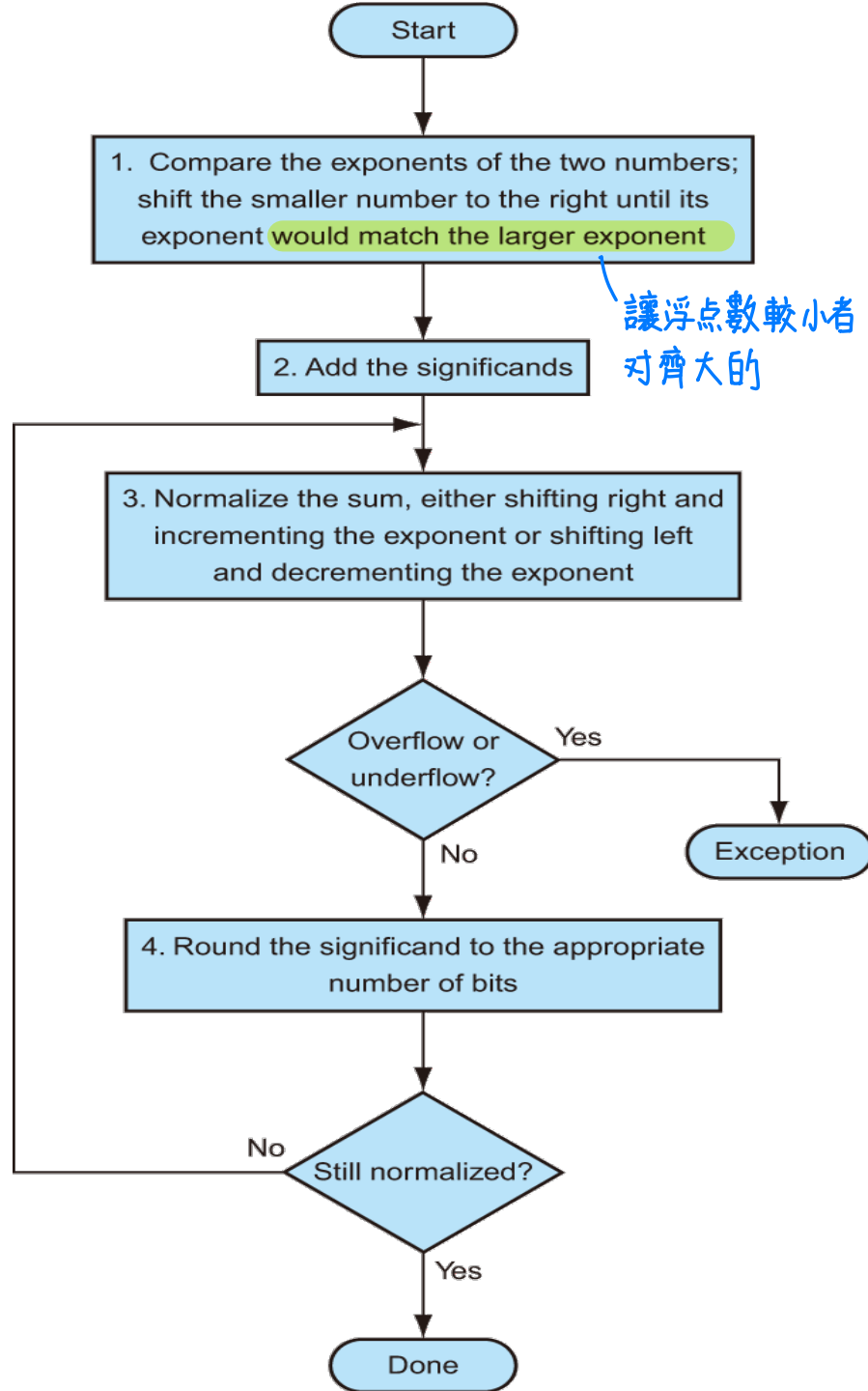


圖3.15
專為浮
點數加
法之算
術運算
單元方
塊圖

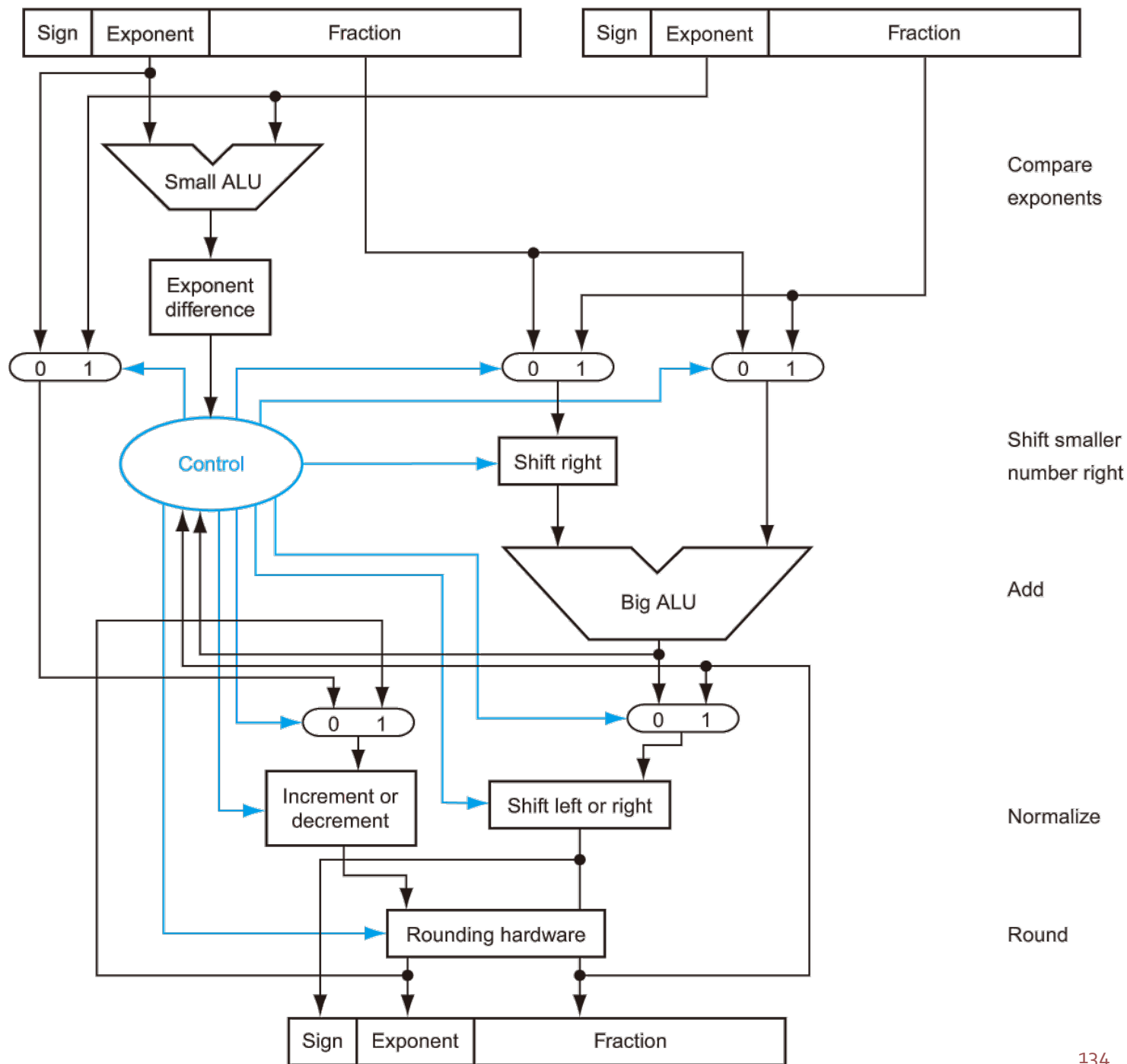


圖3.15 專為浮點數加法之算術運算單元方塊圖

圖3.14 中的步驟由上至下對應到每個方塊。首先，使用小算術邏輯單元將兩運算元的指數部分相減，以決定何者比較大與大多少。這個差值控制了三個多工器；由左至右，它們分別選擇較大的指數、較小數的有效數字、較大數的有效數字，將較小數的有效數字右移，然後使用大算術邏輯單元將有效數字相加。接下來正規化步驟將總和左移或右移，並將指數遞增或遞減。進位處理後得到最後的結果，而這可能需要再經過一次的正規化

浮點數乘法

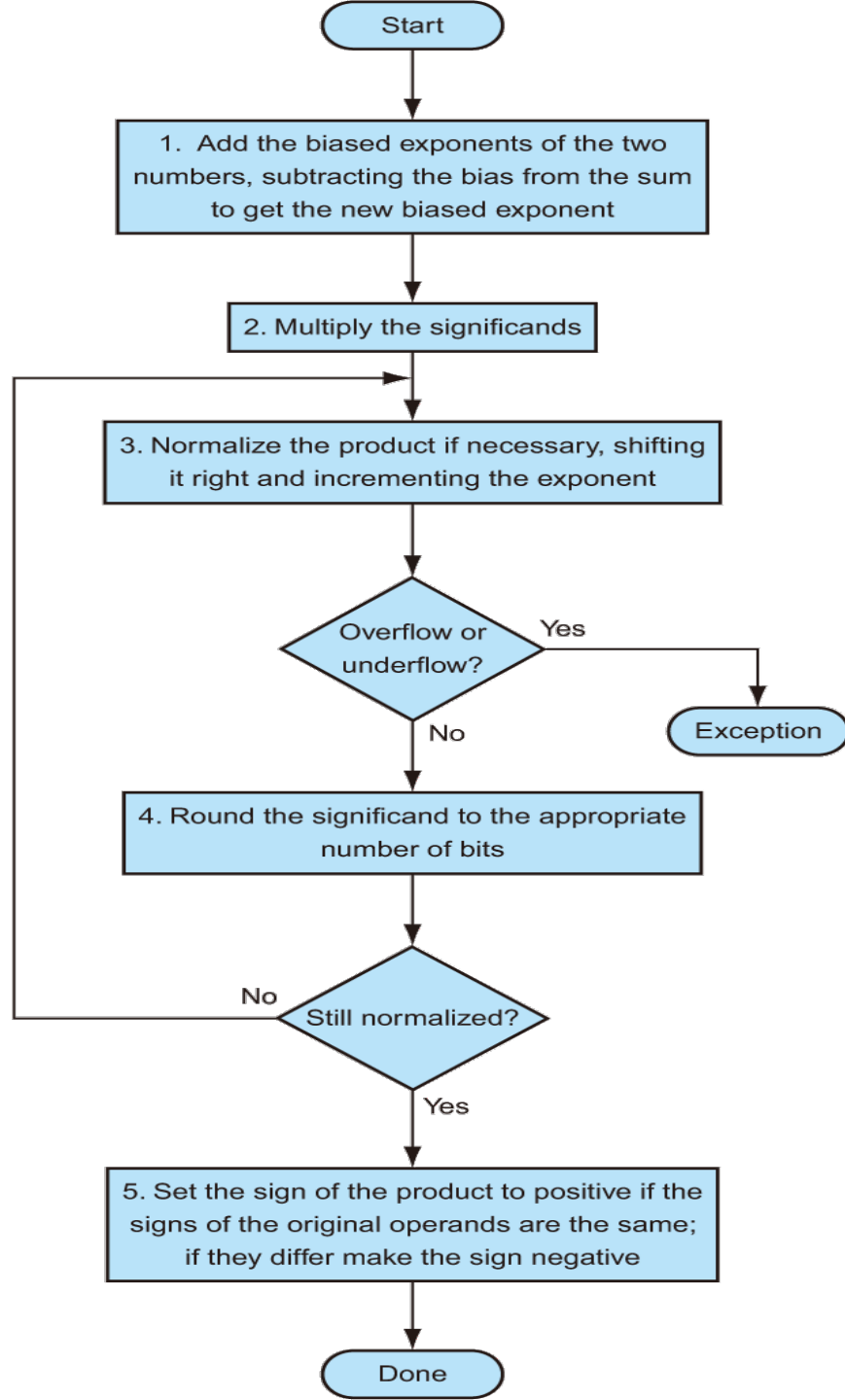
圖3.16 浮點數乘法

正常的路徑是執行步驟三與步驟四各一次，但是如果進位處理造成總和非正規化，我們就必須重複步驟三

課本P. 227 範例: 0.5×-0.4375

Handwritten calculation showing the multiplication of 0.5 and -0.4375 in binary:

$$\begin{aligned} 0.5 &\times -0.4375_{10} = -0.21875_{10} \\ (1.000 \times 2^{-1}) &\times (-1.110 \times 2^{-2}) \\ &= (1.000 \times (-1.110)) \times 2^{-1-2} \\ &= (-1.110) \times 2^{-3} \\ &= -1.110 \times 2^{-3} \end{aligned}$$



MIPS 中的浮點指令

■ MIPS 以這些指令來支援IEEE 754 單精確度與雙精確度的格式：

- 浮點單精確度加法(add.s)與雙精確度加法(add.d)
- 浮點單精確度減法(sub.s)與雙精確度減法(sub.d)
- 浮點單精確度乘法(mul.s)與雙精確度乘法(mul.d)
- 浮點單精確度除法(div.s)與雙精確度除法(div.d)
- 浮點單精確度比較(c.x.s)與雙精確度比較(c.x.d) , 其中x 可能是相等(eq) 、 不相等(neq) 、 小於(lt) 、 小於等於(le) 、 大於(gt) 、 大於等於(ge)
- 浮點為真分支(bc1t)與為偽分支(bc1f)

MIPS 中的浮點指令

- 加入另外的浮點暫存器稱為 \$f0 、 \$f1 、 \$f2
、
- 也加入了浮點暫存器的載入與儲存指令： lwc1 與 swc1

MIPS floating-point operands		
Name	Example	Comments
32 floating-point registers	\$f0, \$f1, \$f2, . . . , \$f31	MIPS floating-point registers are used in pairs for double precision numbers.
2 ³⁰ memory words	Memory[0], Memory[4], . . . , Memory[4294967292]	Accessed only by data transfer instructions. MIPS uses byte addresses, so sequential word addresses differ by 4. Memory holds data structures, such as arrays, and spilled registers, such as those saved on procedure calls.

MIPS floating-point assembly language				
Category	Instruction	Example	Meaning	Comments
Arithmetic	FP add single	add.s \$f2,\$f4,\$f6	\$f2 = \$f4 + \$f6	FP add (single precision)
	FP subtract single	sub.s \$f2,\$f4,\$f6	\$f2 = \$f4 - \$f6	FP sub (single precision)
	FP multiply single	mul.s \$f2,\$f4,\$f6	\$f2 = \$f4 × \$f6	FP multiply (single precision)
	FP divide single	div.s \$f2,\$f4,\$f6	\$f2 = \$f4 / \$f6	FP divide (single precision)
	FP add double	add.d \$f2,\$f4,\$f6	\$f2 = \$f4 + \$f6	FP add (double precision)
	FP subtract double	sub.d \$f2,\$f4,\$f6	\$f2 = \$f4 - \$f6	FP sub (double precision)
	FP multiply double	mul.d \$f2,\$f4,\$f6	\$f2 = \$f4 × \$f6	FP multiply (double precision)
	FP divide double	div.d \$f2,\$f4,\$f6	\$f2 = \$f4 / \$f6	FP divide (double precision)
Data transfer	load word copr. 1	lwc1 \$f1,100(\$s2)	\$f1 = Memory[\$s2 + 100]	32-bit data to FP register
	store word copr. 1	swc1 \$f1,100(\$s2)	Memory[\$s2 + 100] = \$f1	32-bit data to memory
Conditional branch	branch on FP true	bclt 25	if (cond == 1) go to PC + 4 + 100	PC-relative branch if FP cond.
	branch on FP false	bclf 25	if (cond == 0) go to PC + 4 + 100	PC-relative branch if not cond.
	FP compare single (eq,ne,lt,le,gt,ge)	c.lt.s \$f2,\$f4	if (\$f2 < \$f4) cond = 1; else cond = 0	FP compare less than single precision
	FP compare double (eq,ne,lt,le,gt,ge)	c.lt.d \$f2,\$f4	if (\$f2 < \$f4) cond = 1; else cond = 0	FP compare less than double precision

MIPS floating-point machine language								
Name	Format	Example						Comments
add.s	R	17	16	6	4	2	0	add.s \$f2,\$f4,\$f6
sub.s	R	17	16	6	4	2	1	sub.s \$f2,\$f4,\$f6
mul.s	R	17	16	6	4	2	2	mul.s \$f2,\$f4,\$f6
div.s	R	17	16	6	4	2	3	div.s \$f2,\$f4,\$f6
add.d	R	17	17	6	4	2	0	add.d \$f2,\$f4,\$f6
sub.d	R	17	17	6	4	2	1	sub.d \$f2,\$f4,\$f6
mul.d	R	17	17	6	4	2	2	mul.d \$f2,\$f4,\$f6
div.d	R	17	17	6	4	2	3	div.d \$f2,\$f4,\$f6
lwc1	I	49	20	2	100			lwc1 \$f2,100(\$s4)
swc1	I	57	20	2	100			swc1 \$f2,100(\$s4)
bclt	I	17	8	1	25			bclt 25
bclf	I	17	8	0	25			bclf 25
c.lt.s	R	17	16	4	2	0	60	c.lt.s \$f2,\$f4
c.lt.d	R	17	17	4	2	0	60	c.lt.d \$f2,\$f4
Field size		6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	All MIPS instructions 32 bits

FIGURE 3.17 MIPS floating-point architecture revealed thus far. See Appendix A, Section A.10, for more detail. This information is also found in column 2 of the MIPS Reference Data Card at the front of this book.

精確的算術

- ▣ 浮點數通常是一個其無法確實表示之數的近似值
- ▣ IEEE 754 提供數種進位處理(rounding)模式以便讓程式師選用所欲的近似法
 - 精準地進位需要硬體在計算中使用額外的位元
 - 在過程中的加法時，總是在右邊帶著兩個額外的位元，分別稱為保護位元(guard)與進位位元(round)
 - 浮點數的準確度通常是以有效數字中有幾個最低位元具有誤差來表示

精確的算術

- 其量測單位稱為最後位置的單元數(units in the last place, ulp)
- 在沒有溢位、短值、或是無效運算例外事件的情形下，IEEE 754 保證了計算機所使用的數字其誤差在半個ulp 之內

■ IEEE 754 有四種捨入模式：

① 無條件向正向進位(朝向 $+\infty$)

② 無條件向負向進位(朝向 $-\infty$)

③ 截短

④ 進位至最接近的偶數

■ 為了支援進位至最接近偶數的目標，標準中除了保護位元與進位位元之外，還有第三個位元黏的位元(sticky bit)：

■ 每當在進位位元右邊有非零位元時，它就會被設為1

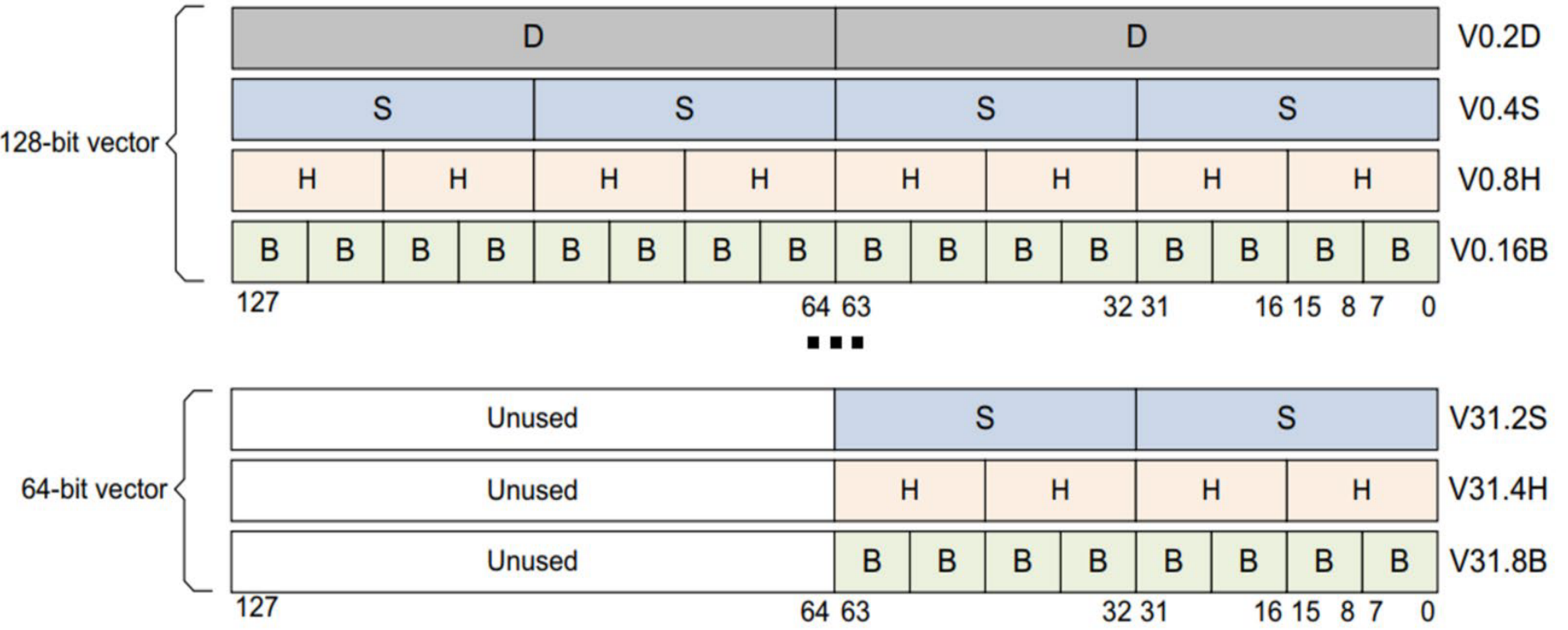
□ $a = a + (b \times c)$

- 是與其做兩次進位處理一個在乘之後然後一個在加之後，該乘加指令可以只在加之後做一次進位處理
 - 增加了乘加的精確度
 - 稱為熔合的乘加(fused multiply add)
- 已被加入修訂後的IEEE 754-2008 標準

3.6 平行性與計算機算術：部分字的平行性

- 許多圖形及音訊的應用中將會對以向量形式呈現的數據執行相同的運算
 - 藉由切斷128 位元加法器中的進位鏈，處理器即可利用平行性(parallelism)同時對 16 個 8 位元、8 個 16 位元、4 個 32 位元或 2 個 64 位元的運算元這種短小的向量執行運算
 - 所需的額外成本不多
 - 被歸類為部分字的平行性(subword parallelism)
 - 數據階層平行性(data level parallelism) 向量或 SIMD——代表單一指令(流)多筆數據(流)(single instruction (stream), multiple data (stream))

SIMD(單令多量,向量)指令運作使用向量暫存器



Scalar (純量)指令與SIMD(單令多量,向量)指令 add vs. vadd

Scalar mode

(one instruction produces one result)

$a[i]$

+

$b[i]$

$a[i]+b[i]$

a

+

b

$a+b$

SIMD processing

(one instruction can produce multiple results)

$a[i+7]$ $a[i+6]$ $a[i+5]$ $a[i+4]$ $a[i+3]$ $a[i+2]$ $a[i+1]$ $a[i]$

+

$b[i+7]$ $b[i+6]$ $b[i+5]$ $b[i+4]$ $b[i+3]$ $b[i+2]$ $b[i+1]$ $b[i]$

$c[i+7]$ $c[i+6]$ $c[i+5]$ $c[i+4]$ $c[i+3]$ $c[i+2]$ $c[i+1]$ $c[i]$



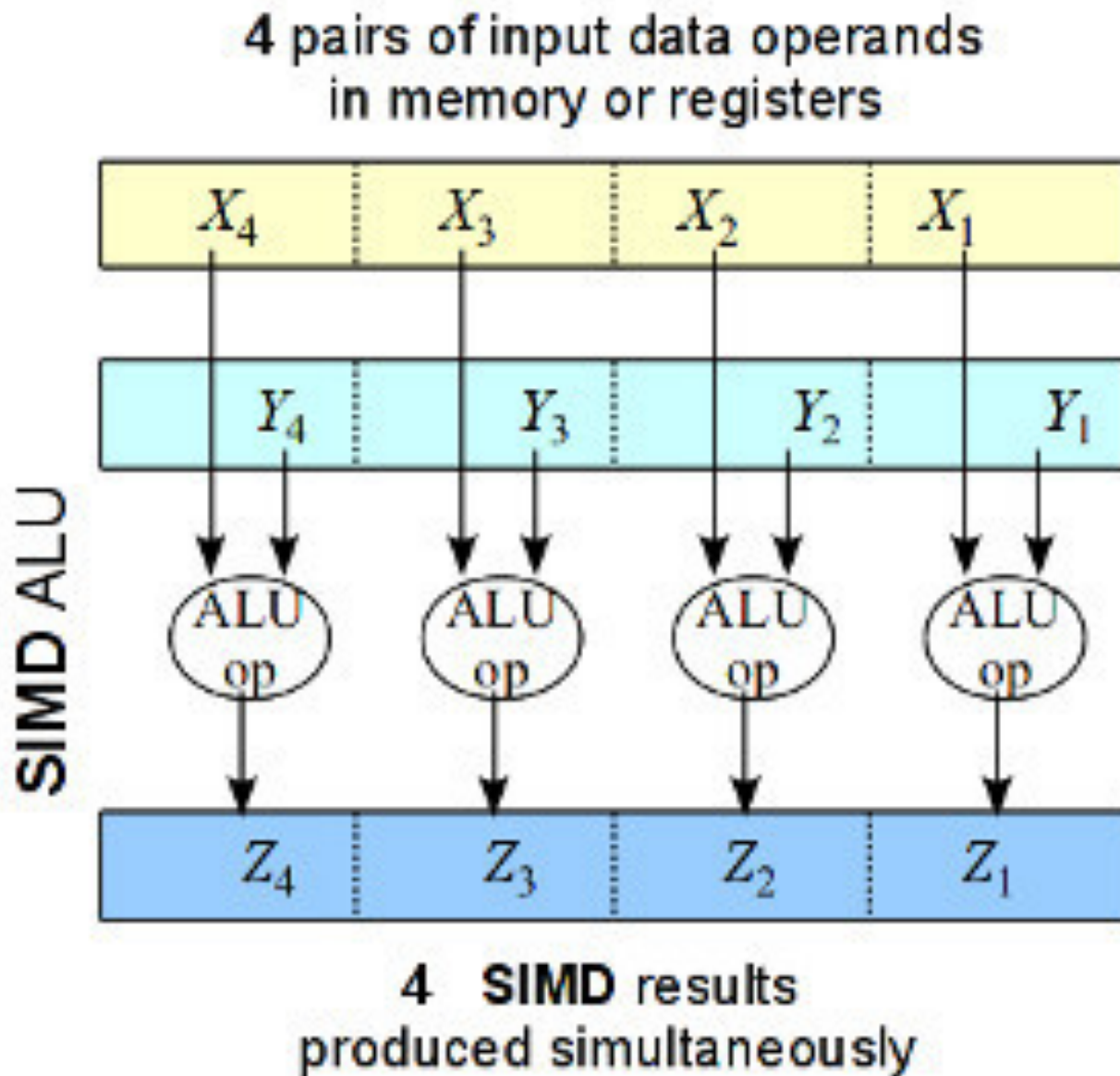
3.6 平行性與計算機算術：部分字的平行性

- ARM 在 NEON 多媒體指令延伸中加入 了 100 道以上支援部分字平行性的指令

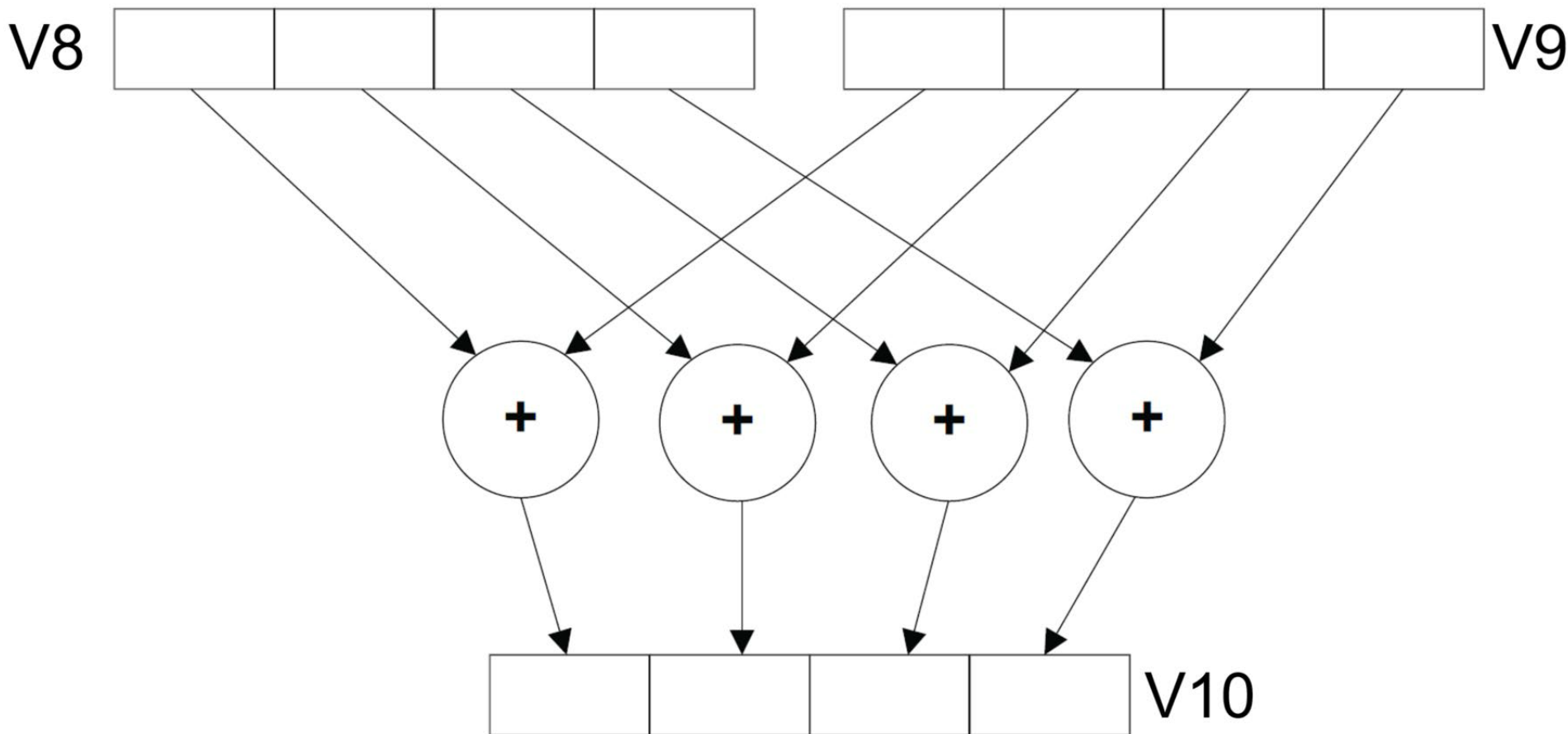
Data transfer	Arithmetic	Logical/Compare
VLDR.F32	VADD.F32, VADD{L,W}{S8,U8,S16,U16,S32,U32}	VAND.64, VAND.128
VSTR.F32	VSUB.F32, VSUB{L,W}{S8,U8,S16,U16,S32,U32}	VORR.64, VORR.128
VLD{1,2,3,4}.{I8,I16,I32}	VMUL.F32, VMULL{S8,U8,S16,U16,S32,U32}	VEOR.64, VEO.128
VST{1,2,3,4}.{I8,I16,I32}	VMLA.F32, VMLAL{S8,U8,S16,U16,S32,U32}	VBIC.64, VBIC.128
VMOV.{I8,I16,I32,F32}, #imm	VMLS.F32, VMLSL{S8,U8,S16,U16,S32,U32}	VORN.64, VORN.128
VMVN.{I8,I16,I32,F32}, #imm	VMAX.{S8,U8,S16,U16,S32,U32,F32}	VCEQ.{I8,I16,I32,F32}
VMOV.{I64,I128}	VMIN.{S8,U8,S16,U16,S32,U32,F32}	VCGE.{S8,U8,S16,U16,S32,U32,F32}
VMVN.{I64,I128}	VABS.{S8,S16,S32,F32}	VCGT.{S8,U8,S16,U16,S32,U32,F32}
	VNEG.{S8,S16,S32,F32}	VCLE.{S8,U8,S16,U16,S32,U32,F32}
	VSHL.{S8,U8,S16,U16,S32,S64,U64}	VCLT.{S8,U8,S16,U16,S32,U32,F32}
	VSHR.{S8,U8,S16,U16,S32,S64,U64}	VTST.{I8,I16,I32}

FIGURE 3.19 Summary of ARM NEON instructions for subword parallelism. We use the curly brackets {} to show optional variations of the basic operations: {S8,U8,8} stand for signed and unsigned 8-bit integers or 8-bit data where type doesn't matter, of which 16 fit in a 128-bit register; {S16,U16,16} stand for signed and unsigned 16-bit integers or 16-bit type-less data, of which 8 fit in a 128-bit register; {S32,U32,32} stand for signed and unsigned 32-bit integers or 32-bit type-less data, of which 4 fit in a 128-bit register; {S64,U64,64} stand for signed and unsigned 64-bit integers or type-less 64-bit data, of which 2 fit in a 128-bit register; {F32} stand for signed and unsigned 32-bit floating point numbers, of which 4 fit in a 128-bit register. Vector Load reads one n-element structure from memory into 1, 2, 3, or 4 NEON registers. It loads a single n-element structure to one lane (See Section 6.6), and elements of the register that are not loaded are unchanged. Vector Store writes one n-element structure into memory from 1, 2, 3, or 4 NEON registers.

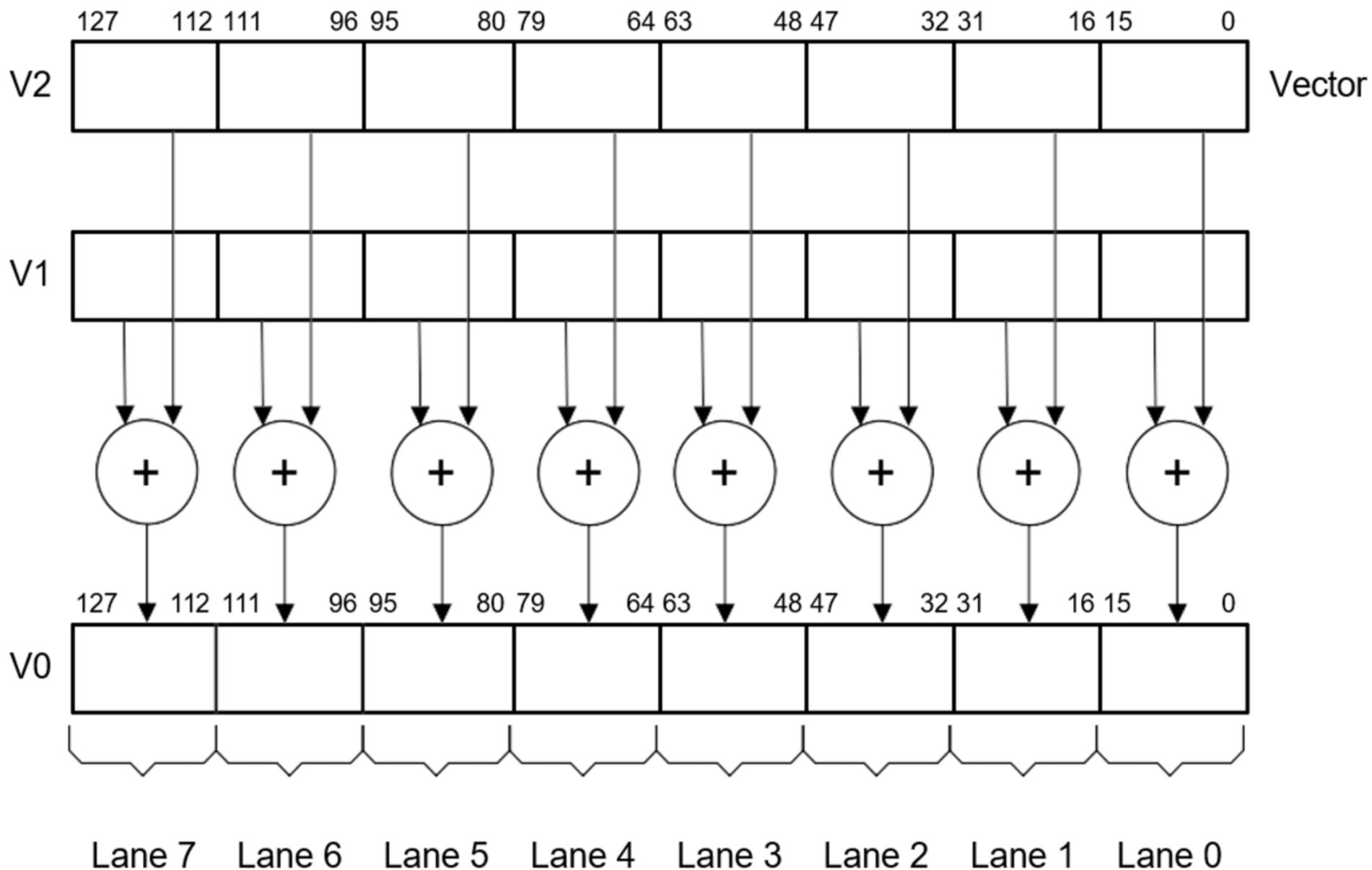
4 Lane(四通道)SIMD(單令多量,向量)ALU運算



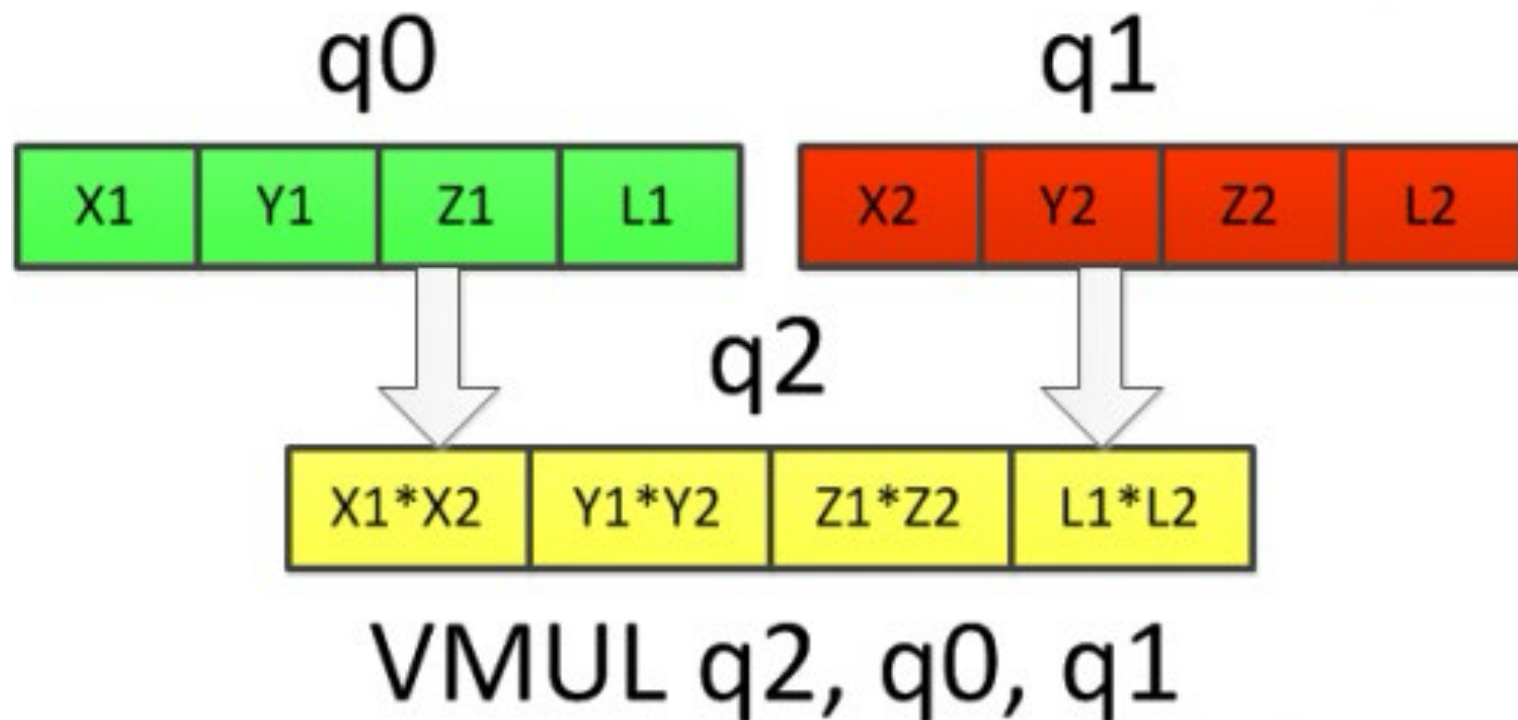
4 Lane(四通道)SIMD(單令多量,向量)加法運算



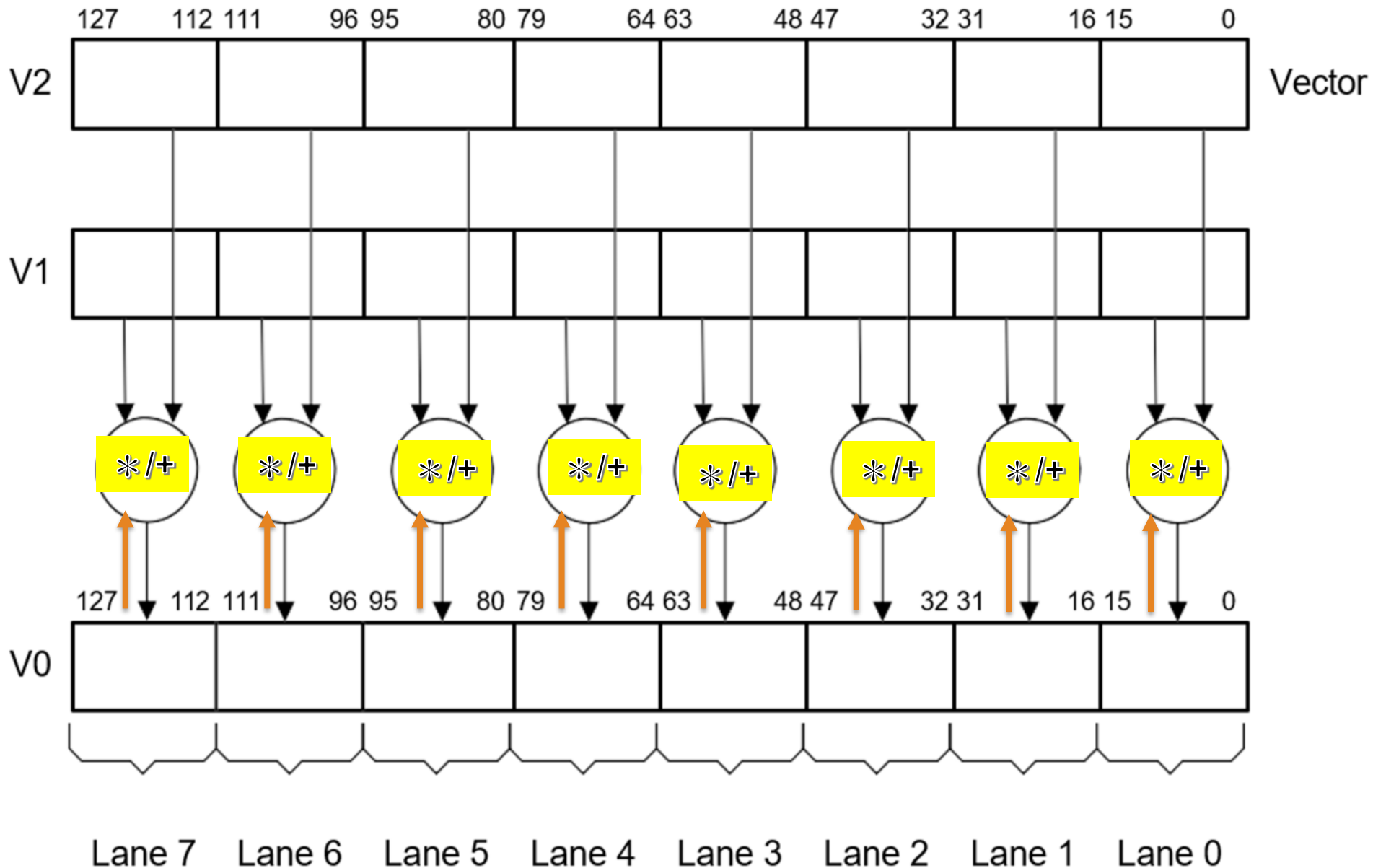
8 Lane(八通道)SIMD(單令多量,向量)加法運算



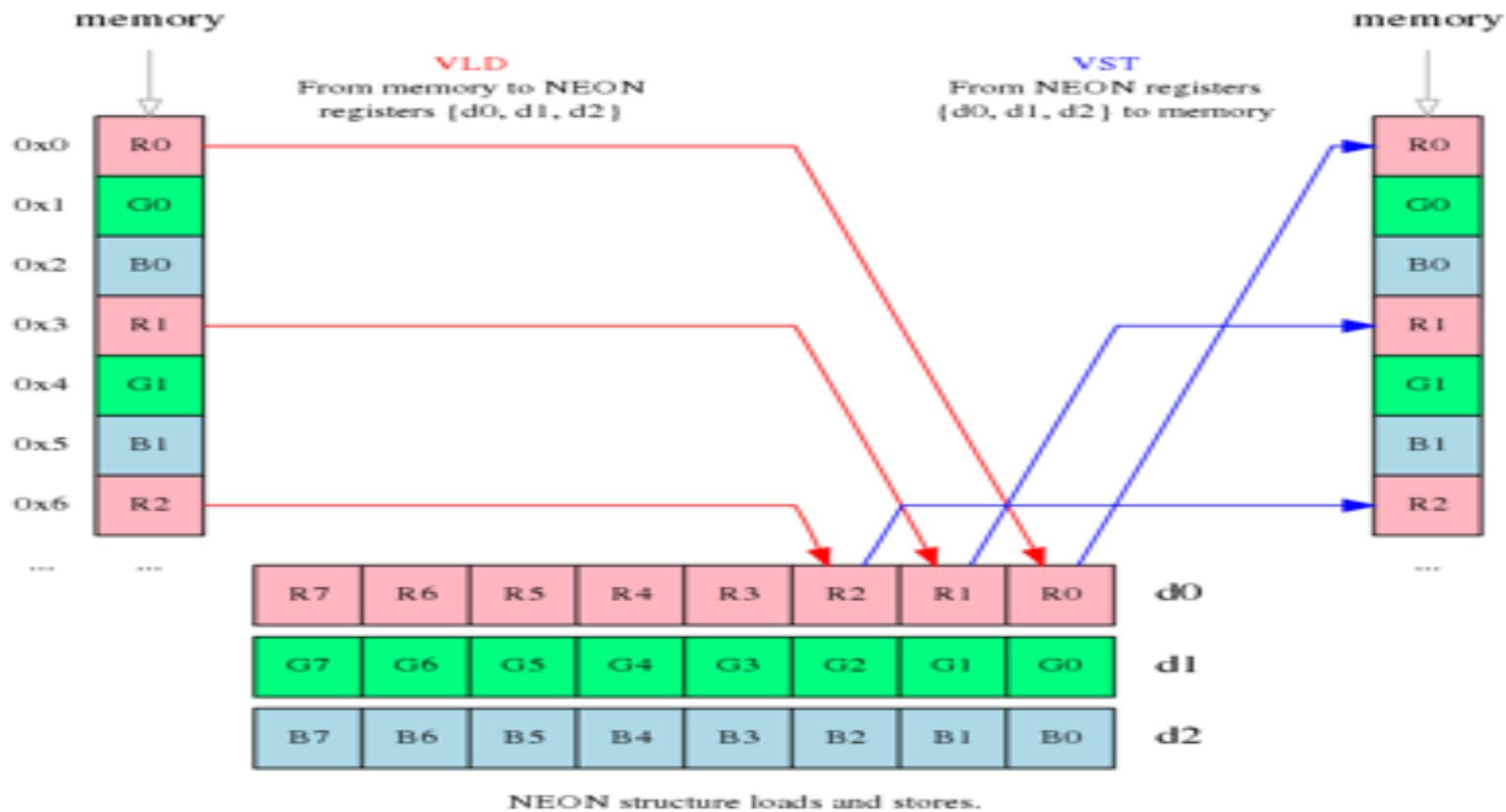
4 Lane(四通道)SIMD(單令多量,向量)乘法運算



8 Lane(八通道)SIMD(單令多量,向量)乘加運算



SIMD(單令多量,向量)載入、儲存指令



3.7 實例：x86 中串流式SIMD 延伸與先進的向量(處理)延伸

- ▣ x86 原始的MMX(MultiMedia eXtension)及 SSE(Streaming SIMD Extension)指令包括類似於 ARM NEON 中的各種運作

數據轉移	算術	比較
MOV{A/U}{SS/PS/SD/PD} xmm, mem/xmm	ADD{SS/PS/SD/PD} xmm, mem/xmm	CMP{SS/PS/SD/PD}
	SUB{SS/PS/SD/PD} xmm, mem/xmm	
MOV {H/L} {PS/PD} xmm, mem/xmm	MUL{SS/PS/SD/PD} xmm, mem/xmm	
	DIV{SS/PS/SD/PD} xmm, mem/xmm	
	SQRT{SS/PS/SD/PD} mem/xmm	
	MAX {SS/PS/SD/PD} mem/xmm	
	MIN{SS/PS/SD/PD} mem/xmm	

圖3.20 x86 的SSE/SSE2 浮點指令群

3.7 實例：x86 中串流式SIMD 延伸與先進的向量(處理)延伸

圖3.20 x86 的SSE/SSE2 浮點指令群

xmm 代表有一個運算元是一個128 位元的SSE2 暫存器，而 mem/xmm 代表另一個運算元或是在記憶體中或是一個SSE2 暫存器。我們用大括號{}來代表各基本運作的可以選用的變形：{SS}代表在128 位元的暫存器中的純量(scalar)單精確度浮點數，或是一個32 位元的運算元；{PS}代表在128 位元的暫存器中的打包了(packed)的單精確度浮點數，或是四個32 位元的運算元；{SD}代表在128 位元的暫存器中的純量(scalar)雙精確度浮點數，或是一個64 位元的運算元；{PD}代表在128 位元的暫存器中的打包了(packed)的雙精確度浮點數，或是兩個64位元的運算元；{A}代表該128 位元的運算元在記憶體中是對齊的(aligned)；{U}代表該128位元的運算元在記憶體中是沒有對齊的(unaligned)；{H}代表移動該128 位元暫存器高位的一半位元；而{L}代表移動該128 位元暫存器低位的一半位元

3.7 實例：x86 中串流式SIMD 延伸與先進的向量(處理)延伸

- 16 個SSE2 的XMM浮點暫存器是128 個位元寬
- 打包的浮點格式可用於能同時運作於四個單精確度(PS)或兩個雙精確度(PD)的算術運算中
- Intel 先進向量延伸 (Advanced Vector Extensions , AVX)的YMM 暫存器又擴充為256位元寬
 - 若要從128 位元改為256 位元的運作，則需於SSE2 組合語言運作碼前冠以“v”(表向量Vector)並且使用YMM 暫存器而非XMM 暫存器的名稱

3.8 走得更快：部分字平行性與矩陣乘法

- 為了說明部分字平行性對效能的重大影響，我們在 Intel Core i7 上將先不使用 AVX、然後再使用它來執行相同的程式
- 圖3.21 是一個未經最佳化的矩陣乘以矩陣C 程式 DGEMM (Double precision GEneral Matrix Multiply)
- 圖3.22 是圖3.21 中內迴圈的x86 組合語言碼

```
1. void dgemm (int n, double* A, double* B, double* C)
2. {
3.     for (int i = 0; i < n; ++i)
4.         for (int j = 0; j < n; ++j)
5.             {
6.                 double cij = C[i+j*n]; /* cij = C[i][j] */
7.                 for( int k = 0; k < n; k++ )
8.                     cij += A[i+k*n] * B[k+j*n]; /* cij += A[i][k]*B[k][j] */
9.                 C[i+j*n] = cij; /* C[i][j] = cij */
10.            }
11. }
```

圖3.21 一個雙精確度矩陣乘法的C 語言程式，廣為人知的名稱是DGEMM，代表Double-precision GEneral Matrix Multiply (雙精確度一般矩陣乘法)

因為我們將矩陣維度的大小以參數n 來傳遞，這個版本的DGEMM 將矩陣C、A 及B 視為單一維度的矩陣來處理以及進行對位址的運算以求獲得更好的效能，而非使用更為直覺的如同我們在3.5 節中所見的二維陣列的方式。註解可以提醒我們在這種更直覺的方式中對應的陣列元素是什麼

1. vmovsd (%r10),%xmm0	# Load 1 element of C into %xmm0
2. mov %rsi,%rcx	# register %rcx = %rsi
3. xor %eax,%eax	# register %eax = 0
4. vmovsd (%rcx),%xmm1	# Load 1 element of B into %xmm1
5. add %r9,%rcx	# register %rcx = %rcx + %r9
6. vmulsd (%r8,%rax,8),%xmm1,%xmm1	# Multiply %xmm1, element of A
7. add \$0x1,%rax	# register %rax = %rax + 1
8. cmp %eax,%edi	# compare %eax to %edi
9. vaddsd %xmm1,%xmm0,%xmm0	# Add %xmm1, %xmm0
10. jg 30 <dgemm+0x30>	# jump if %eax > %edi
11. add \$0x1,%r11d	# register %r11 = %r11 + 1
12. vmovsd %xmm0,(%r10)	# Store %xmm0 into C element

圖3.22 在圖3.21 中經過最佳化的C 程式其巢狀迴圈的本體經編譯後所產出的x86 組合語言(程式)

雖然其所處理的是只有64 位元的數據，編譯器應該是推測其可能需要在一道指令中使用到三個位址而不只兩個(見3.7 節中的仔細深思)，因此使用了AVX 版本而非SSE2 中的指令

3.8 走得更快：部分字平行性與矩陣乘法

▣ 圖3.23 列出圖3.21 的程式經改良後的版本

```
1. #include <x86intrin.h>
2. void dgemm (int n, double* A, double* B, double* C)
3. {
4.     for ( int i = 0; i < n; i+=4 )
5.         for ( int j = 0; j < n; j++ ) {
6.             __m256d c0 = _mm256_load_pd(C+i+j*n); /* c0 = C[i][j] */
7.             for( int k = 0; k < n; k++ )
8.                 c0 = _mm256_add_pd(c0, /* c0 += A[i][k]*B[k][j] */
9.                                     _mm256_mul_pd(_mm256_load_pd(A+i+k*n),
10.                                                    _mm256_broadcast_sd(B+k+j*n)));
11.             _mm256_store_pd(C+i+j*n, c0); /* C[i][j] = c0 */
12.         }
13. }
```

圖3.23 DGEMM 經過最佳化的C 程式透過C 的編譯器內建函式以便產出x86 的具部分字平行性的AVX 指令

圖3.24 顯示編譯器對內迴圈所產出的組合語言(程式)

3.8 走得更快：部分字平行性與矩陣乘法

- 圖3.23 第6 行的宣告使用了 `_m256d` 數據型態，其告知編譯器該變數包含4 個雙精確度的浮點值
- 也在第6 行的 `_mm256_load_pd()` 使用 AVX 指令將4 個雙精確度浮點數由矩陣C 平行地(`_pd`)載入 `c0` 中
- 在第11 行的最後一步以 `_mm256_store_pd()` 將4 個雙精確度浮點數由 `c0` 儲存回矩陣C 中

3.8 走得更快：部分字平行性與矩陣乘法

- 圖3.24列出使用-O3 層次最佳化的gcc 編譯後產出並加上註解的x86 碼
- 五個AVX 指令—它們均以v 開頭且五個中有四個使用pd 的平行雙精確度能力—其相當於上述的各個C 固有功能

3.8 走得更快：部分字平行性與矩陣乘法

1. vmovapd (%r11),%ymm0	# Load 4 elements of C into %ymm0
2. mov %rbx,%rcx	# register %rcx = %rbx
3. xor %eax,%eax	# register %eax = 0
4. vbroadcastsd (%rax,%r8,1),%ymm1	# Make 4 copies of B element
5. add \$0x8,%rax	# register %rax = %rax + 8
6. vmulpd (%rcx),%ymm1,%ymm1	# Parallel mul %ymm1,4 A elements
7. add %r9,%rcx	# register %rcx = %rcx + %r9
8. cmp %r10,%rax	# compare %r10 to %rax
9. vaddpd %ymm1,%ymm0,%ymm0	# Parallel add %ymm1, %ymm0
10. jne 50 <dgemm+0x50>	# jump if not %r10 != %rax
11. add \$0x1,%esi	# register % esi = % esi + 1
12. vmovapd %ymm0,(%r11)	# Store %ymm0 into 4 C elements

圖3.24 在圖3.23 中經過最佳化的C 程式其巢狀迴圈的本體經編譯後所產生的x86 組合語言(程式)

注意其與圖3.22 的相似性，而主要的不同是現在五個浮點運作會透過YMM 暫存器以使用pd 形式的指令來進行平行的雙精確度運作，而非sd 形式的純量雙精確度運作

3.8 走得更快：部分字平行性與矩陣乘法

▣ 以32x32 的兩個矩陣而言

- 圖3.21 中未經最佳化的DGEMM 在2.6 GHz 的Intel Core i7 (Sandy Bridge)一個核上可跑出1.7 GigaFLOPS
- 圖3.23 中的最佳化後的碼可得6.4 GigaFLOPS
- AVX 的版本有3.85 倍快
 - 非常接近因使用部分字的平行性(subword parallelism)而一次執行4 倍數量的運作所應獲得的4 倍加速

3.9 謬誤與陷阱

- 謬誤：如同左移指令可以代替 2 的冪次方的整數乘法一樣，右移指令會等同於 2 的冪次方的整數除法
 - 解決方法是使用算術右移(arithmetic right shift)來延伸符號位元而不是移入 0 來補位
 - -5_{10} 的算術右移二位元會產生：
1111 1111 1111 1111 1111 1111 1111 1110₂
 - 結果是 -2_{10} 而不是 -1_{10} ；接近了，但仍舊不對

3.9 謬誤與陷阱

- ▣ 陷阱：浮點加法運算不具有結合性
- ▣ 謬誤：可用於整數數據型態的平行執行策略亦可用於浮點數據型態
 - 若你累加一百萬個數字，不論使用1個或1000個處理器都應該會得到相同的結果
 - 然而因為浮點加法不具結合性，該假設對浮點數並不成立
 - 更令人困擾的情況發生在對這個程序使用不同數目的處理器
 - 此即使對相同的輸入執行相同程式也會得到稍有不同的答案

3.9 謬誤與陷阱

- ▣ 陷阱：MIPS 的 add immediate unsigned (addiu) 指令對其 16 位元立即值欄位做符號延伸
- ▣ 謬誤：只有理論數學家在意浮點數的精確度
 - 1994 年 11 月的報紙頭條證明該敘述是個謬誤(見圖 3.25)



圖3.25 從1994年11月起的一些報紙與雜誌文章採樣

包括 *New York Times*、*San Jose Mercury News*、*San Francisco Chronicle* 以及 *Infoworld*。Pentium 浮點除法的錯誤甚至登上 *David Letterman Late Show* 電視節目的十大新聞榜。Intel 最後花了三億元來置換這些錯誤的晶片

3.10 總結評論

- 今天市售的計算機都使用2的補數二進制整數(按：整數應作定點)算術，而且如果它也支援浮點運作，它就會提供IEEE 754的二進浮點算術
- 有限的精確度帶來「溢位」或「短值」的異常事件
 - 不精確與受限制的表示法帶來的各種挑戰是形成數值分析這個領域的部分原因
 - 邇來平行度的運用會再度引起對數值分析的重視

3.10 總結評論

- ▣ 數據階層平行性、特別是部分字平行性，提供一種提升效能的簡易方法
- ▣ 圖3.26 列出在本章與第二章中所介紹的MIPS 指令
 - 圖中左手邊的一組指令為*MIPS* 核心指令(MIPS core)
 - 右手邊的指令則稱為*MIPS* 算術核心指令(MIPS arithmetic core)

3.10 總結評論

- 在圖3.27 左邊的是MIPS 處理器所執行而圖3.26 中沒列出的指令
 - 我們稱完整的硬體指令組為*MIPS-32*
 - 在圖3.27 右邊的是組譯器可以接受但並不屬於MIPS-32 的指令。我們稱這組指令為*MIPS* 虛擬指令(Pseudo MIPS)

3.11 歷史觀點與進一步閱讀

- ▣ 這一節檢視浮點數自從von Neumann 以來的歷史